



# COMPUTER NETWORKS

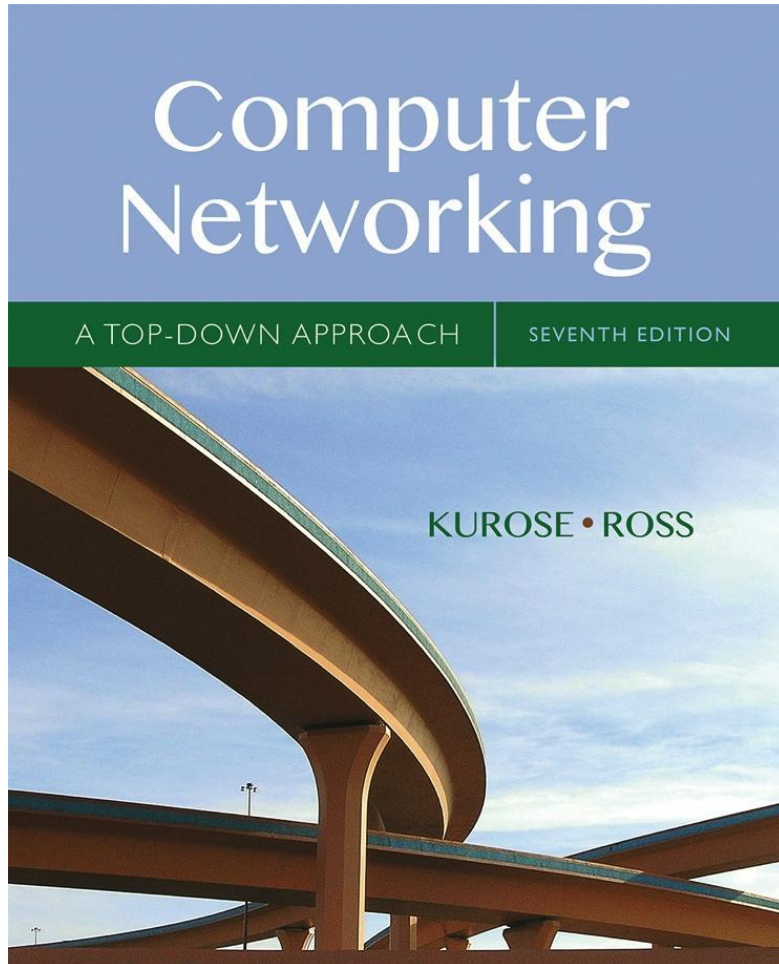
## Computer Networks and the Internet

---

**Team Networks**

Department of Computer Science and Engineering

## *Text Book*



## *Computer Networking: A Top-Down Approach*

Jim Kurose, Keith Ross

Pearson, 2017

7<sup>th</sup> Edition

# COMPUTER NETWORKS

---

## Computer Networks and the Internet

### Team Networks

Department of Computer Science and Engineering

## Unit – 1 Computer Networks and the Internet

### 1.1 Introduction to Computer Networks

### 1.2 What is the Internet?

- A nuts-and-bolts and Services description, Protocol

### 1.3 Network edge

- End systems, Access networks, Physical media

### 1.4 Network core

- Packet switching, Circuit switching, Network structure

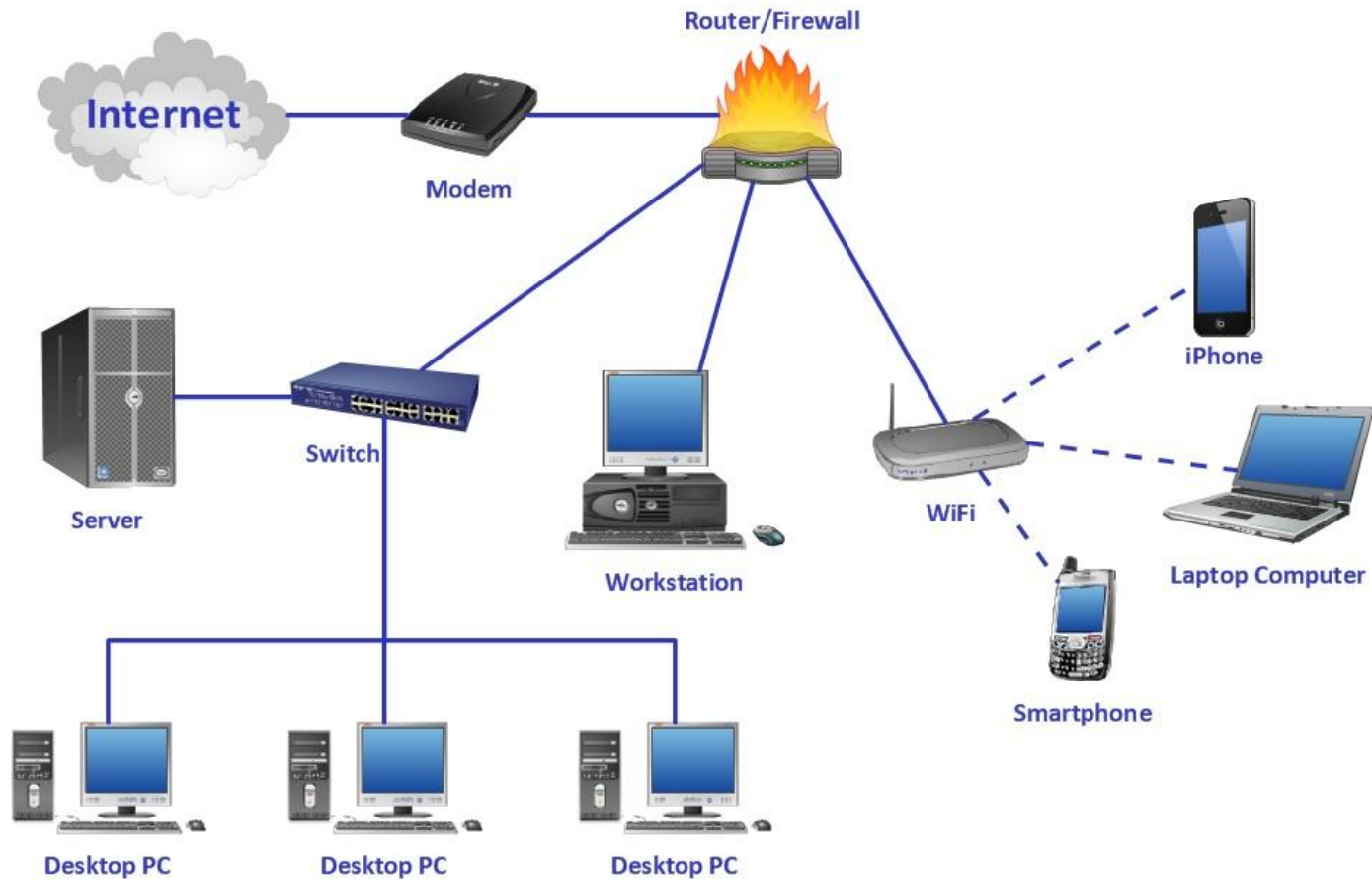
### 1.5 Delay, Loss & Throughput in networks

### 1.6 Protocol layers, Service models

- OSI model and TCP/IP protocol suite

# COMPUTER NETWORKS

## Introduction to Computer Networks



- Two or more devices connected together.
- Communicate with each other, share data or resources

- A massive network of networks.
- A computer network that interconnects billions of computing devices throughout the world.
- Traditional devices – PCs, Workstations, Servers – web pages, emails, etc.
- Internet “things” – laptops, PDAs, TVs, gaming consoles, home security systems, home appliances, watches, cars, traffic control systems, etc.,

# COMPUTER NETWORKS

## The Internet: A “Nuts and Bolts” View



Billions of connected computing *devices*:

- *hosts* = end systems
- running *network apps* at Internet's “edge”



*Packet switches*: forward packets (chunks of data)

- *routers, switches*



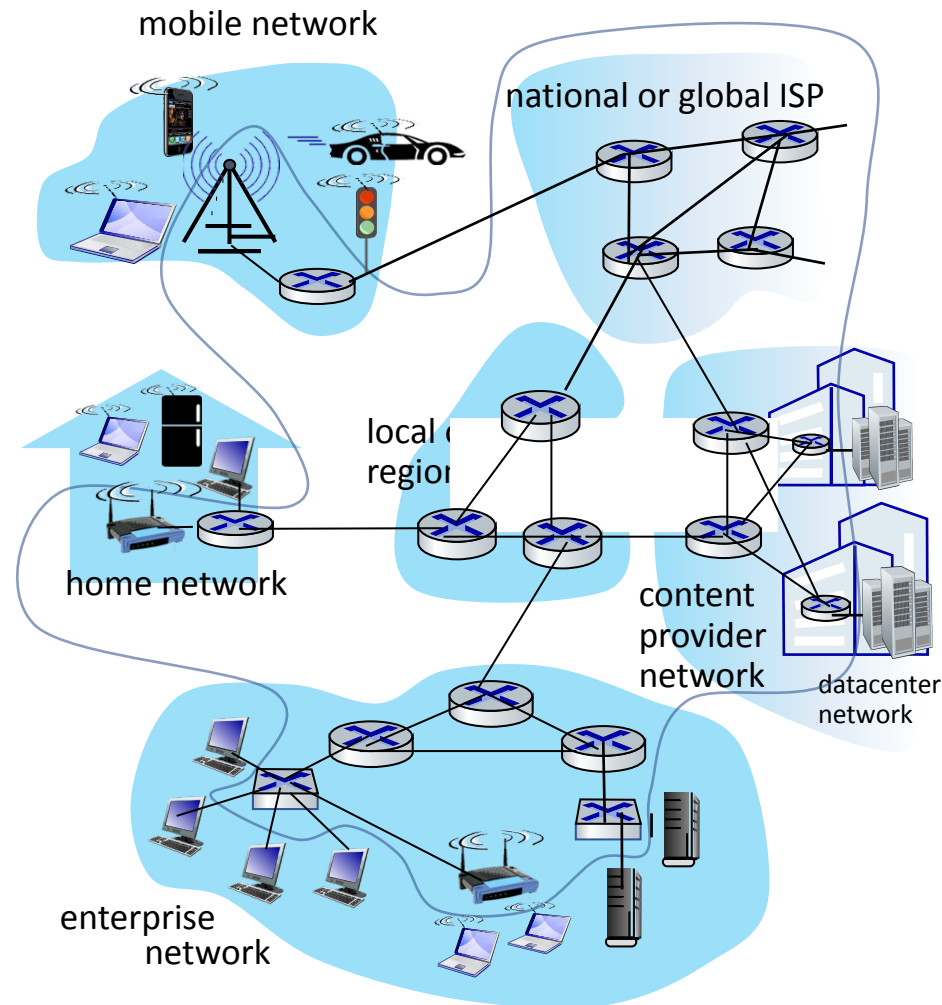
*Communication links*

- fiber, copper, radio, satellite
- transmission rate: *bandwidth*



*Networks*

- collection of devices, routers, links: managed by an organization



# COMPUTER NETWORKS

## “Fun” Inter-connected Devices



Amazon Echo



Internet refrigerator



IP picture frame



Tweet-a-watt:  
monitor energy  
use



Web-enabled toaster +  
weather forecaster



Security Camera



Slingbox: remote  
control cable TV



AR devices

Internet phones



sensorized,  
bed  
mattress



Fitbit

*Others?*

**There will be 41 Billion  
IoT devices by 2027\***

\* <https://www.businessinsider.com/internet-of-things-report?IR=T>

- *Internet: “network of networks”*

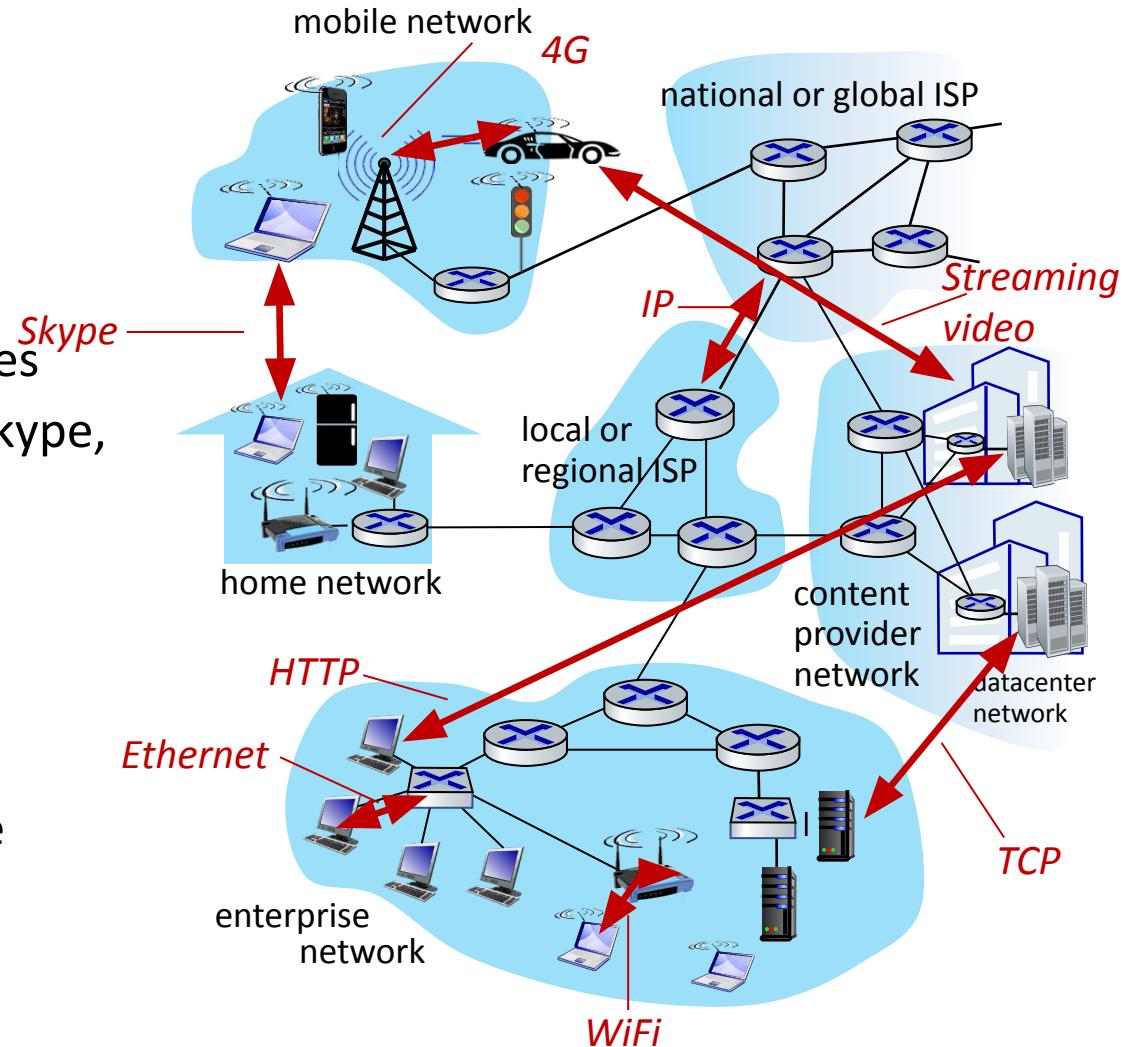
- Interconnected ISPs

- *Protocols are everywhere*

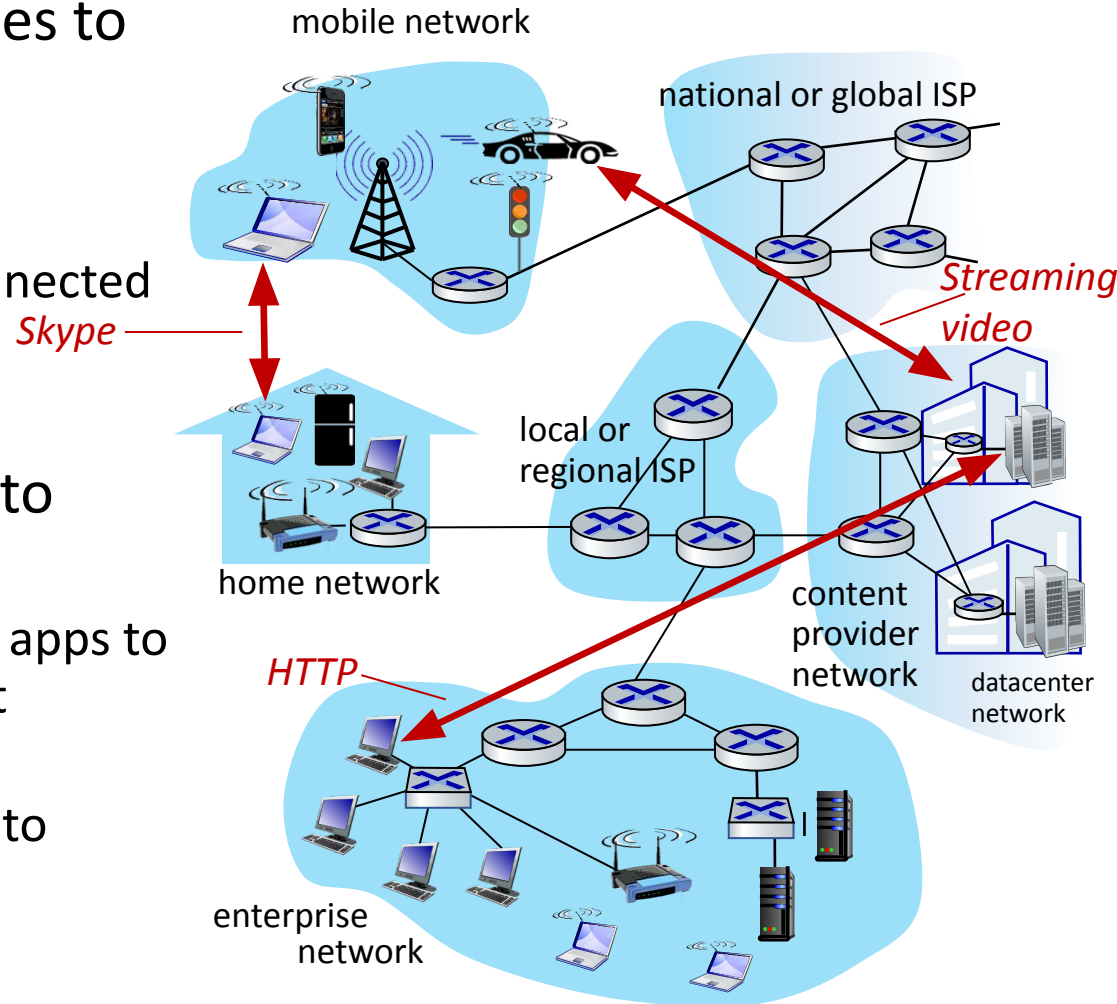
- control sending, receiving of messages
- e.g., HTTP (Web), streaming video, Skype, TCP, IP, WiFi, 4G/5G, Ethernet

- *Internet standards*

- RFC: Request for Comments
- IETF: Internet Engineering Task Force



- *Infrastructure* that provides services to applications:
  - Web, streaming video, multimedia teleconferencing, email, games, e-commerce, social media, inter-connected appliances, ...
- provides *programming interface* to distributed applications:
  - “hooks” allowing sending/receiving apps to “connect” to, use Internet transport service
  - provides service options, analogous to postal service



### Human protocols:

- “what’s the time?”
- “I have a question”
- introductions

... specific messages sent

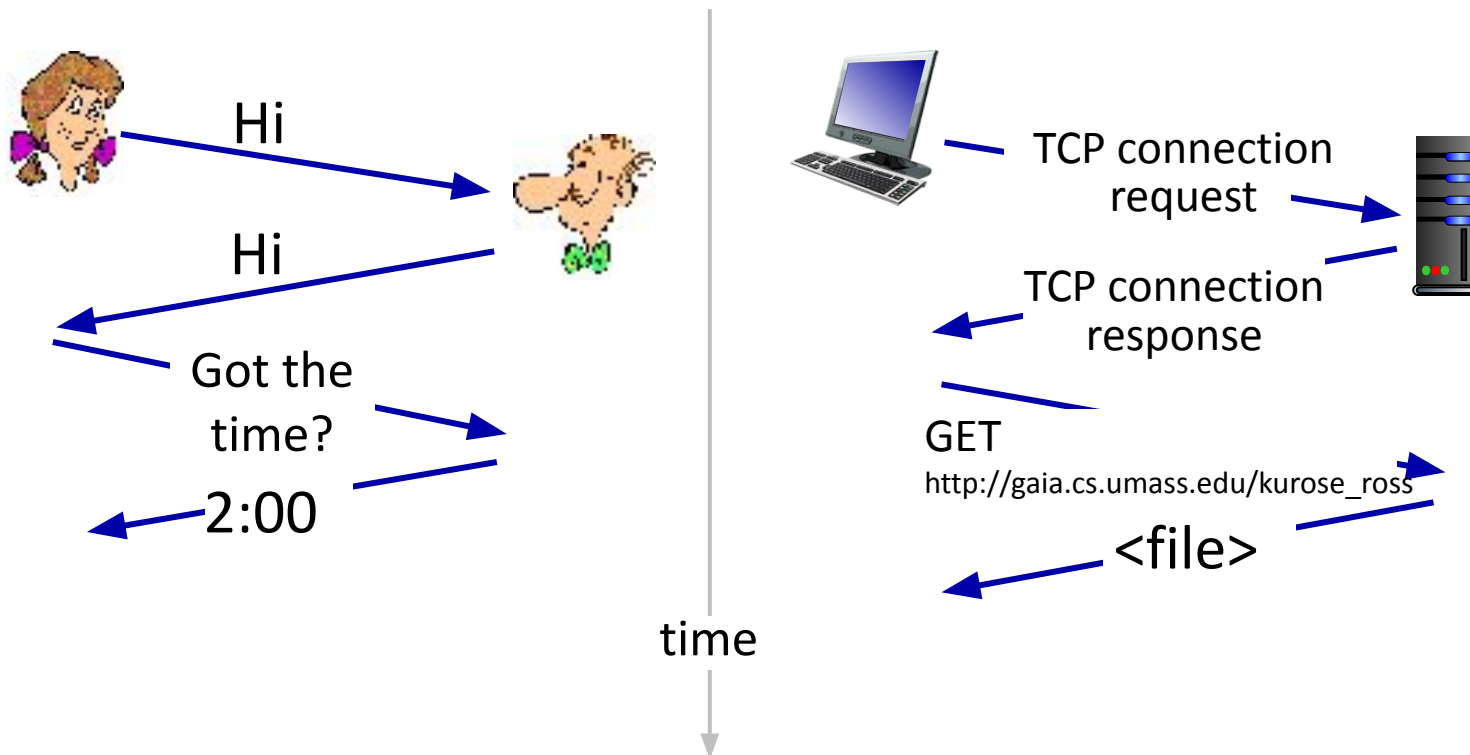
... specific actions taken when  
message received, or other  
events

### Network protocols:

- computers (devices) rather than humans
- all communication activity in Internet  
governed by protocols

*Protocols define the **format, order** of  
**messages sent and received** among  
network entities, and **actions taken**  
on msg transmission, receipt.*

A human protocol and a computer network protocol:



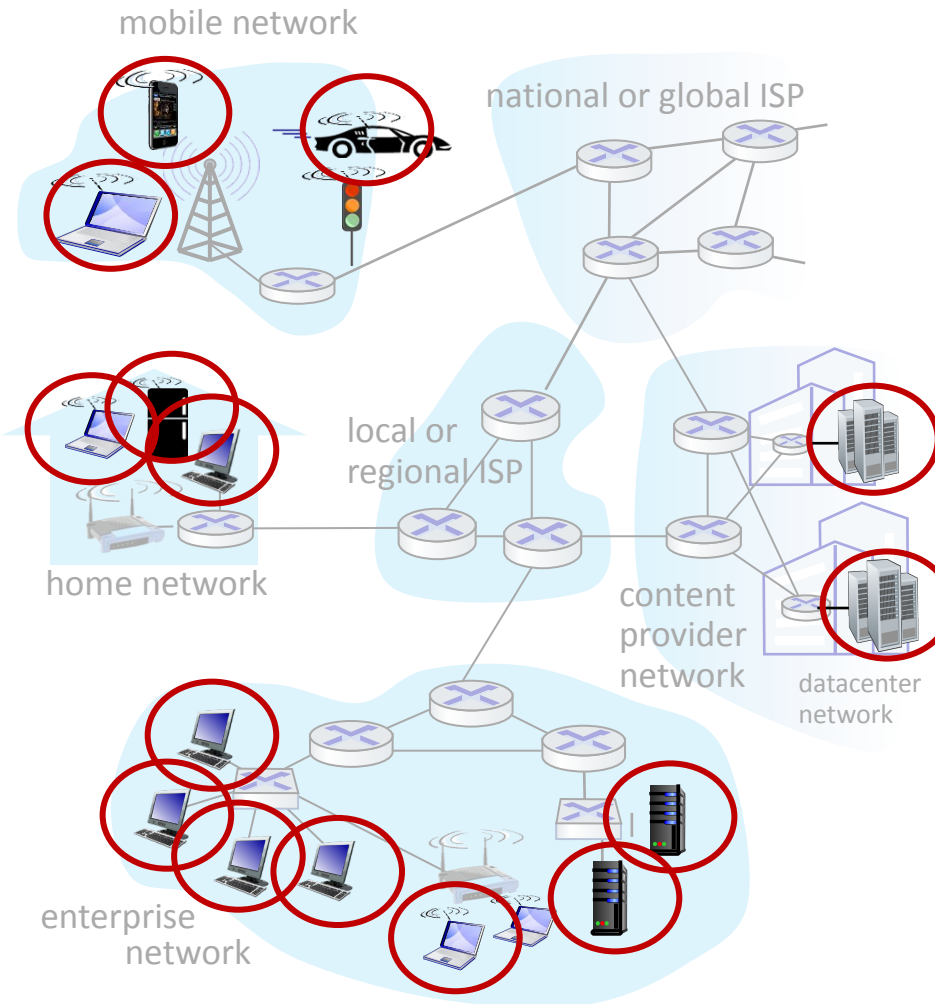
**Q:** other human protocols?

# COMPUTER NETWORKS

## Network Edge: A closer look at network structure

### Network edge:

- Hosts: clients & servers
- Servers in data centers



# COMPUTER NETWORKS

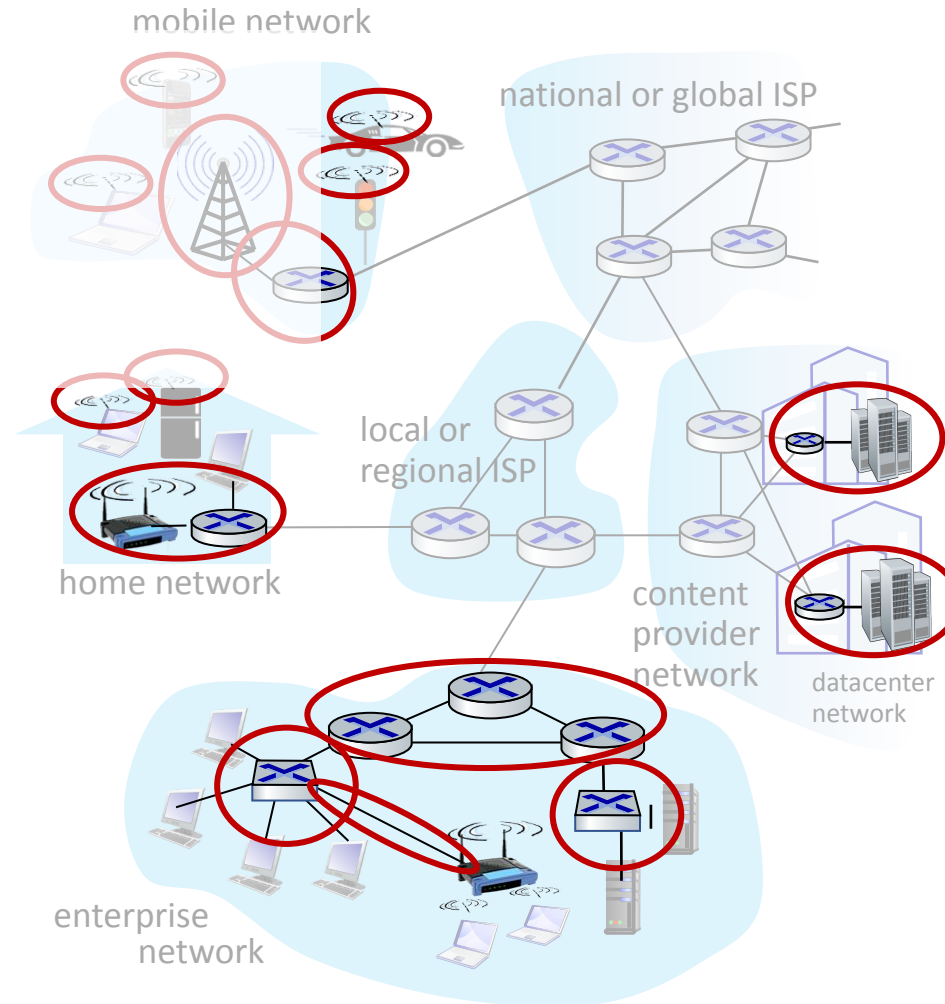
## Network Edge: A closer look at network structure

### Network edge:

- Hosts: clients & servers
- Servers in data centers

### Access networks, physical media:

- wired, wireless communication links



# COMPUTER NETWORKS

## Network Edge: A closer look at network structure

### Network edge:

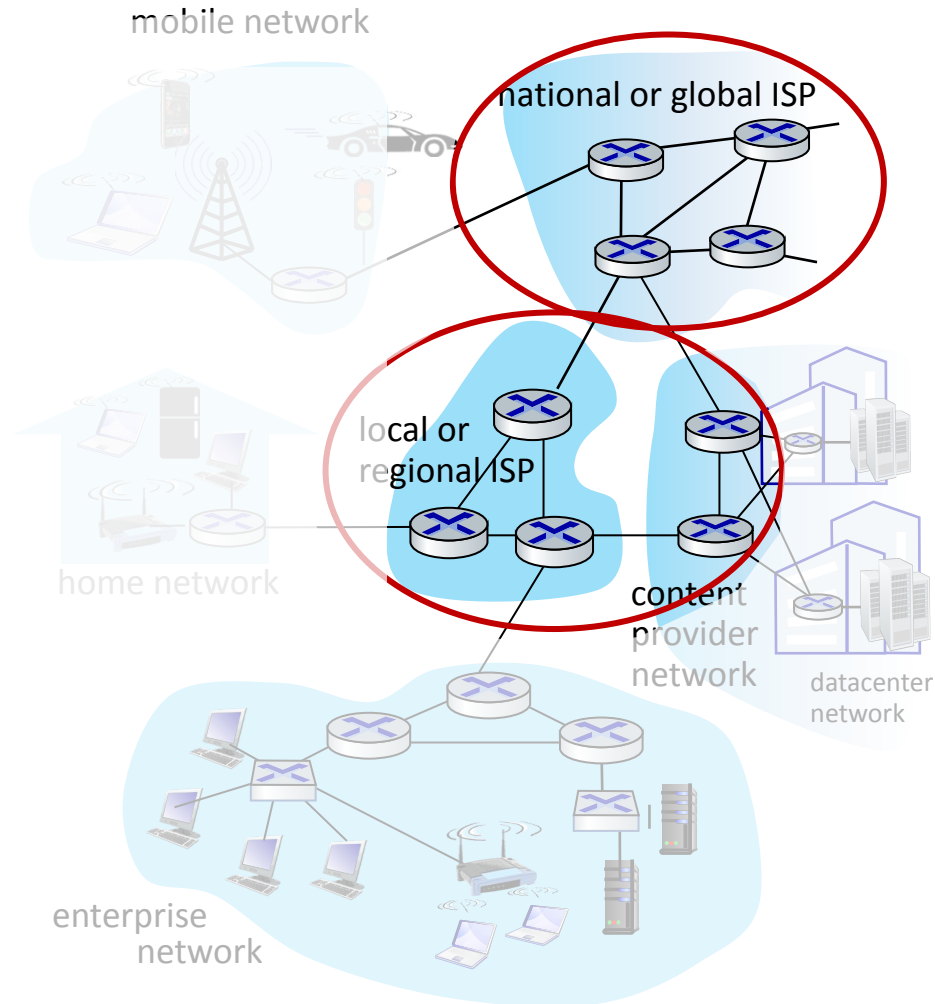
- Hosts: clients and servers
- Servers in data centers

### Access networks, physical media:

- wired, wireless communication links

### Network core:

- interconnected routers
- network of networks



# COMPUTER NETWORKS

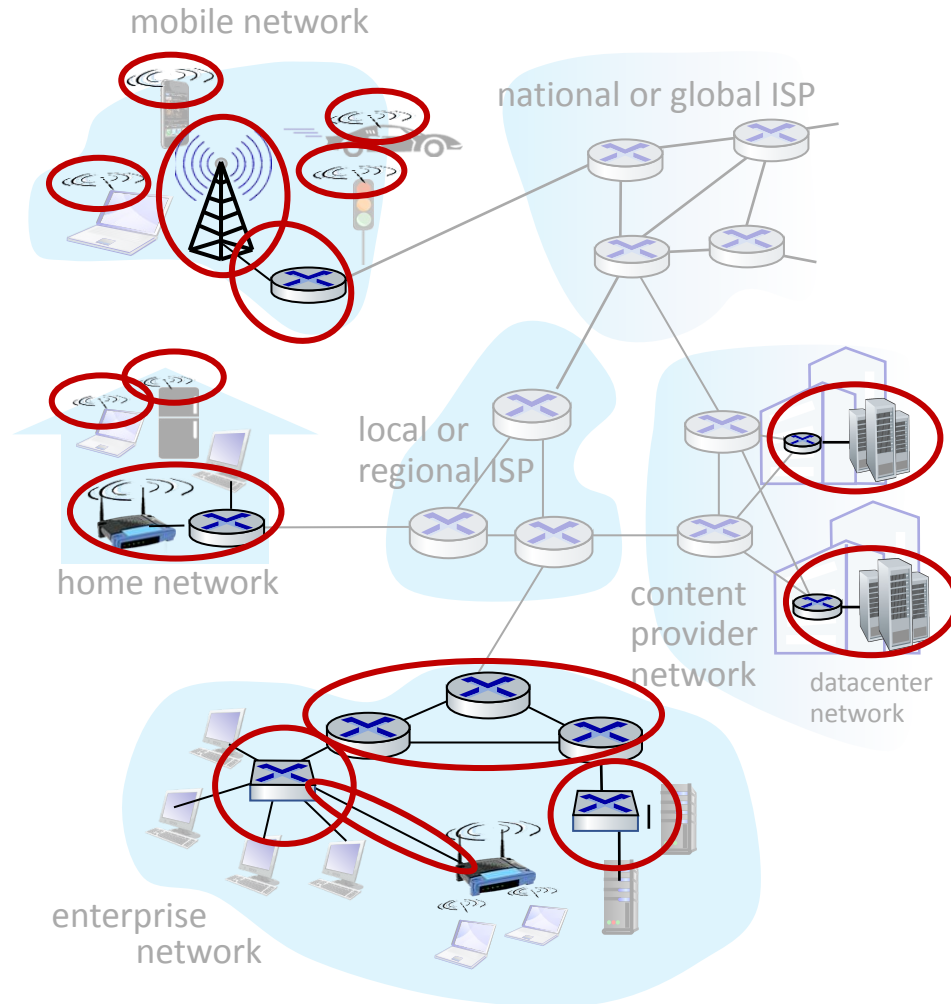
## Network Edge: Access networks and Physical media

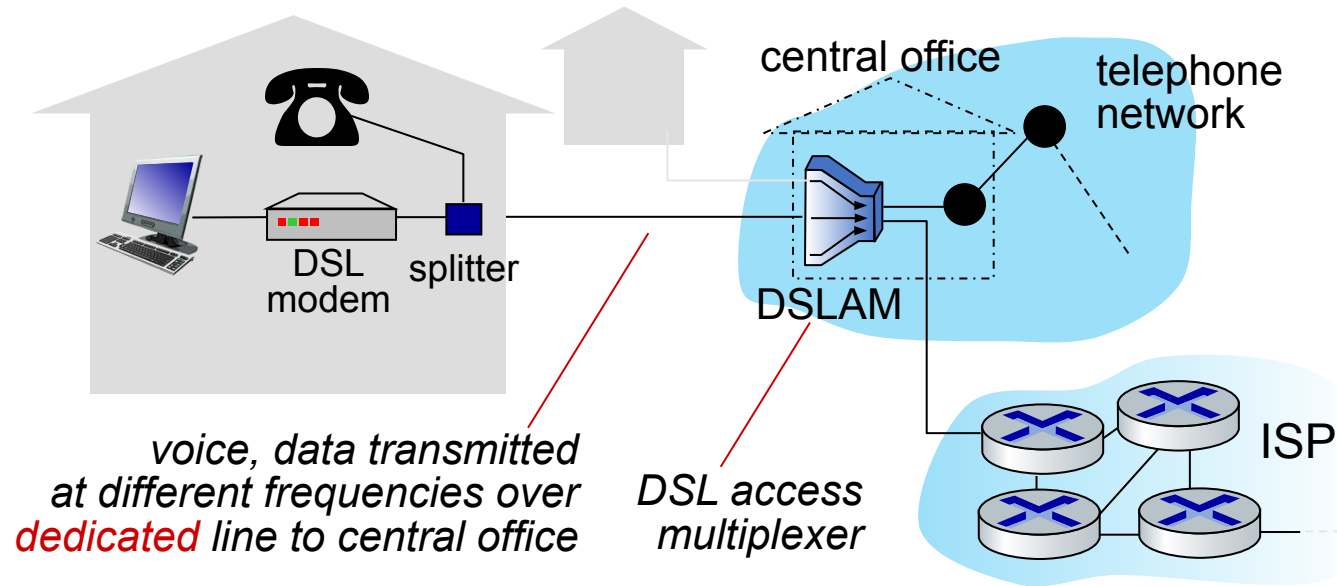
*Q: How to connect end systems to edge router?*

- Residential access networks
- Institutional access networks (school, company)
- Mobile access networks (WiFi, 4G/5G)

*What to look for:*

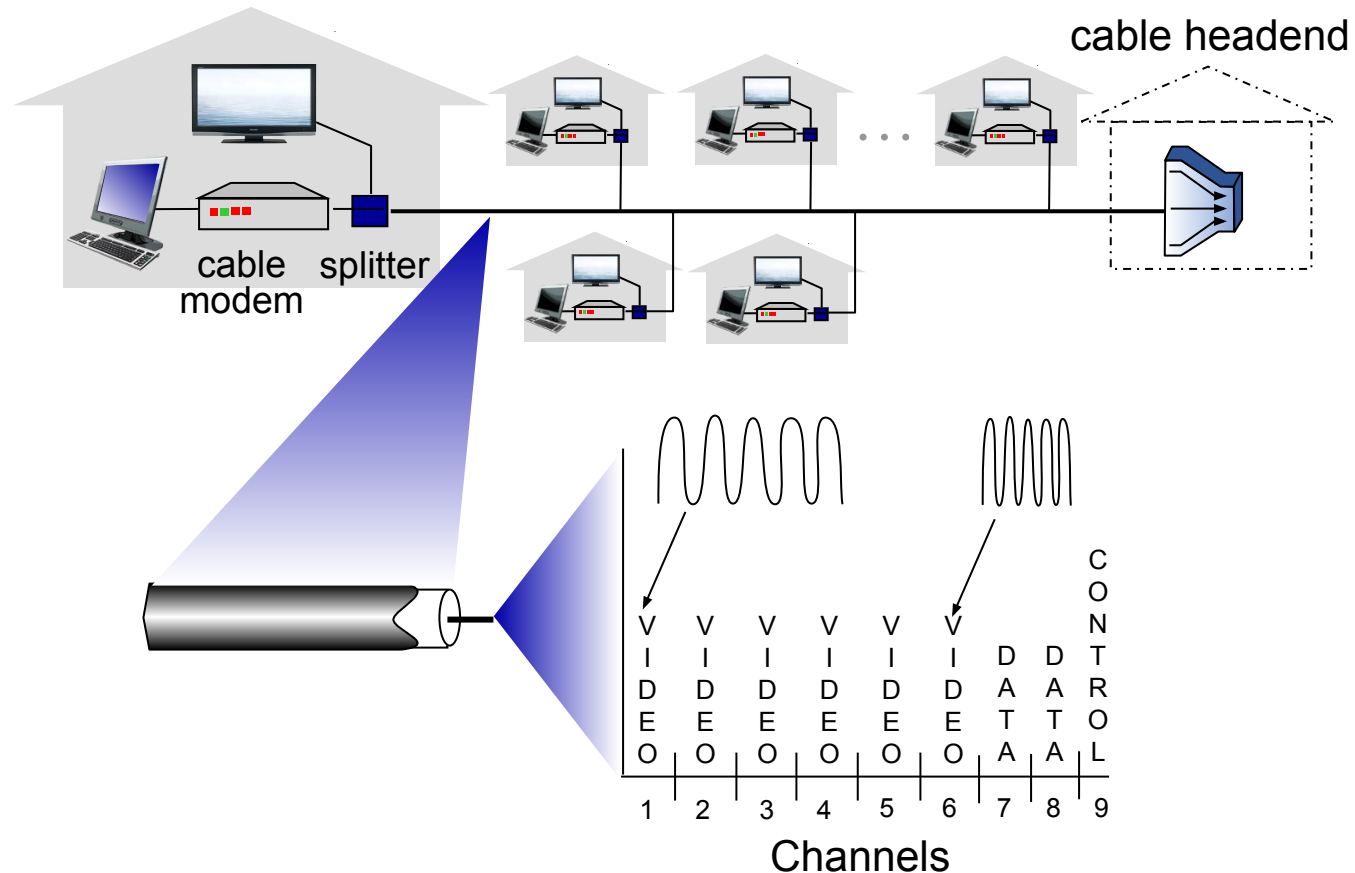
- Transmission rate (bits per second) of access network?
- Shared or dedicated access among users?





- 24-52 Mbps – downstream transmission rate
- 3.5-16 Mbps – upstream transmission rate
- Asymmetric access

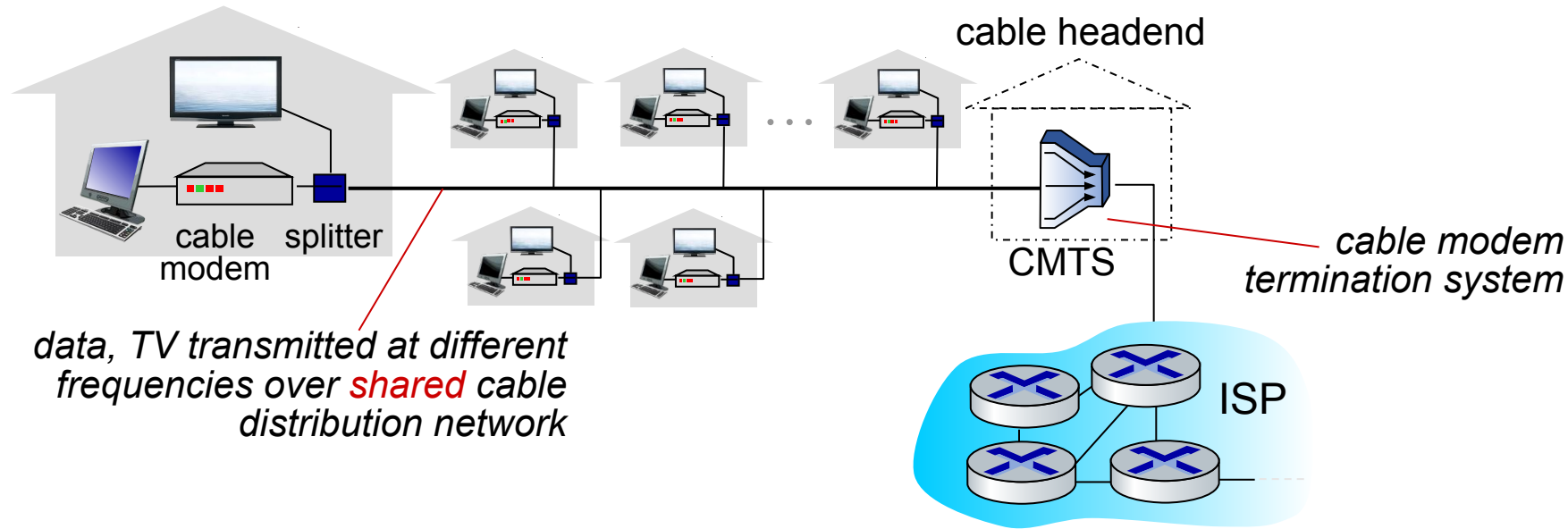
- use **existing** telephone line to central office DSLAM
  - **data** over DSL phone line goes to Internet
  - **voice** over DSL phone line goes to telephone net
- A high-speed downstream channel, in the 50 kHz to 1 MHz band
- A medium-speed upstream channel, in the 4 kHz to 50 kHz band
- An ordinary two-way telephone channel, in the 0 to 4 kHz band



**Frequency division multiplexing (FDM):** different channels transmitted in different frequency bands

# COMPUTER NETWORKS

## Network Edge: Access Networks: Cable-based access



### ■ HFC: hybrid fiber coax

- Asymmetric:

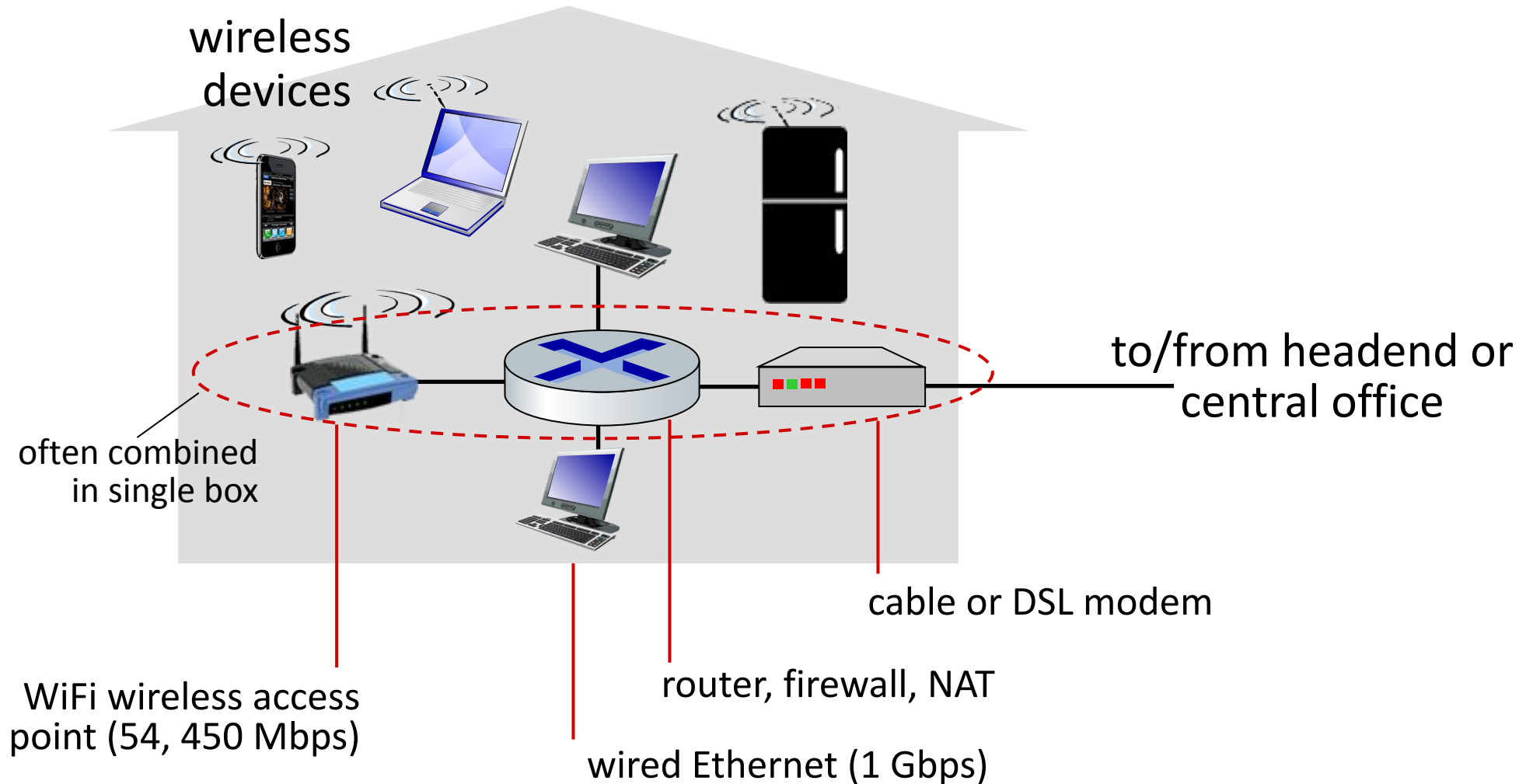
- up to 40 Mbps – 1.2 Gbs downstream transmission rate,
  - 30-100 Mbps upstream transmission rate

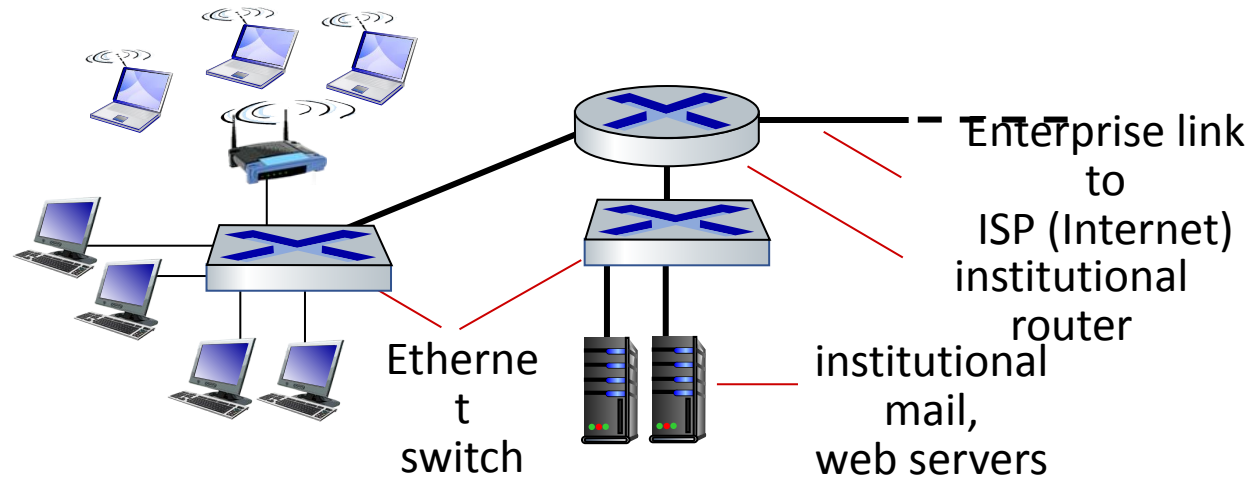
### ■ Network of cable, fiber attaches homes to ISP router

- homes *share access network* to cable headend

# COMPUTER NETWORKS

## Network Edge: Access Networks – Home access





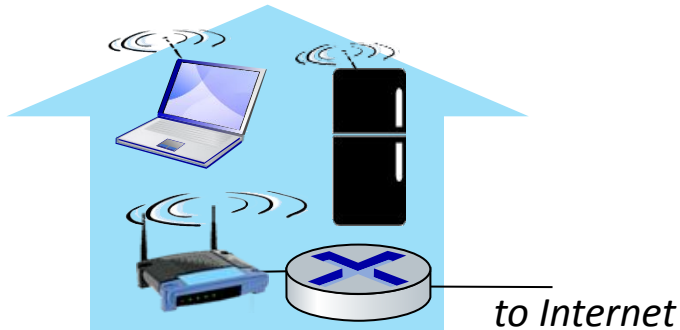
- companies, universities, etc.
- mix of wired, wireless link technologies, connecting a mix of switches and routers (we'll cover differences shortly)
  - Ethernet: wired access at 100Mbps, 1Gbps, 10Gbps
  - WiFi: wireless access points at 11, 54, 450 Mbps

Shared *wireless* access network connects end system to router

- via base station aka “access point”

### Wireless local area networks (WLANs)

- typically within or around building (~100 ft)
- 802.11b/g/n (WiFi): 11, 54, 450 Mbps transmission rate



### Wide-area cellular access networks

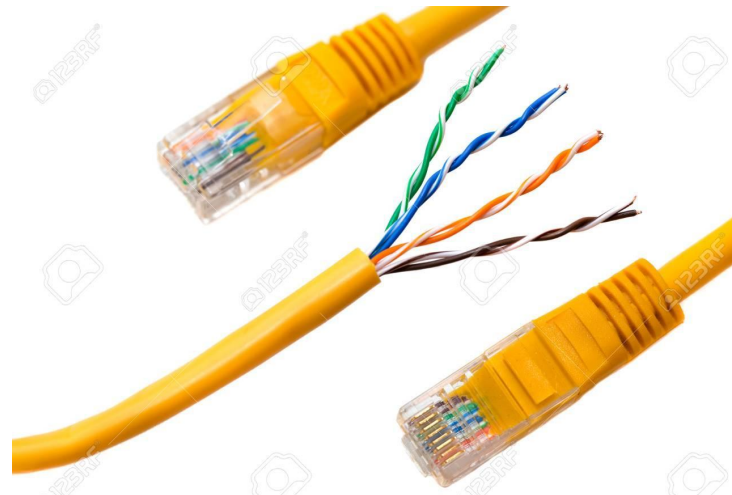
- provided by mobile, cellular network operator (10's km)
- 10's Mbps
- 4G/5G cellular networks (6G coming)



- **bit:** propagates between transmitter/receiver pairs
- **physical link:** what lies between transmitter & receiver
- **guided media:**
  - signals propagate in solid media: copper, fiber, coax
- **unguided media:**
  - signals propagate freely, e.g., radio

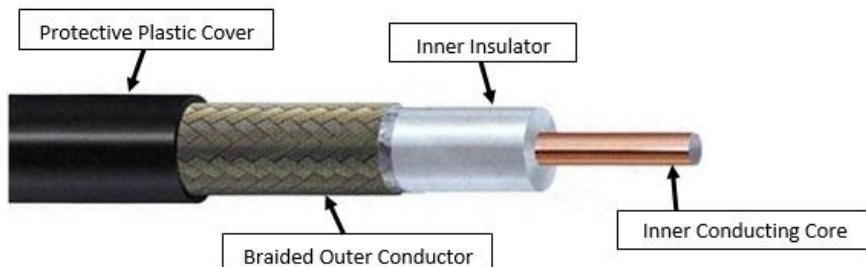
### Twisted pair (TP)

- two insulated copper wires (STP & UTP)
  - Category 5: 100 Mbps, 1 Gbps Ethernet
  - Category 6: 10Gbps Ethernet



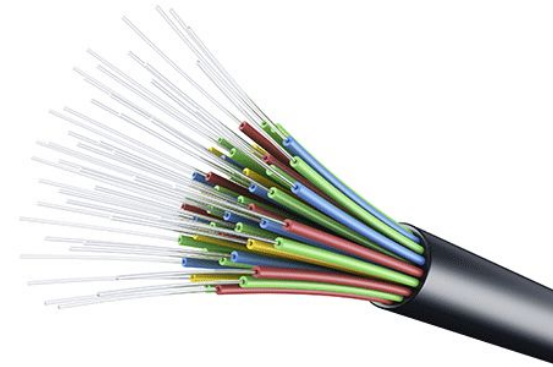
### Coaxial cable:

- two concentric copper conductors
- concentric rather than parallel
- bidirectional
- broadband:
  - multiple frequency channels on cable
  - 100's Mbps per channel



### Fiber optic cable:

- glass fiber carrying light pulses, each pulse a bit
- high-speed operation:
  - high-speed point-to-point transmission (10's-100's Gbps)
- low error rate:
  - repeaters spaced far apart
  - immune to electromagnetic noise

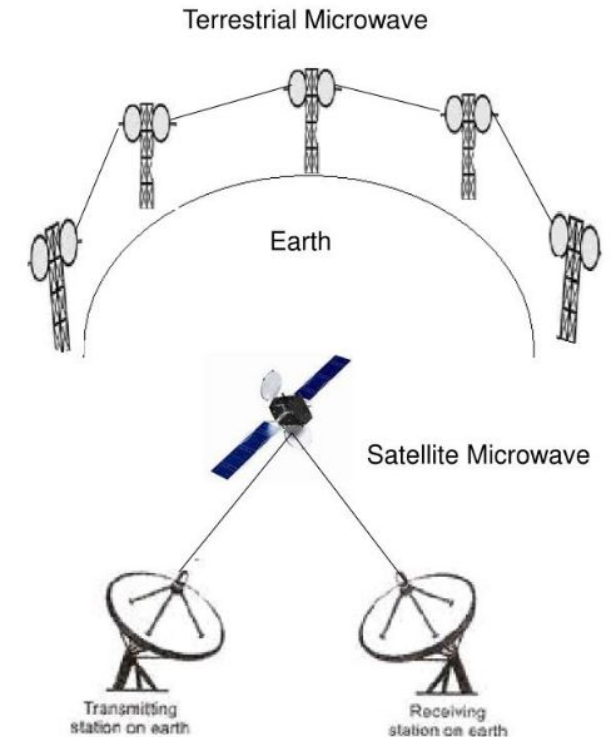


### Wireless radio

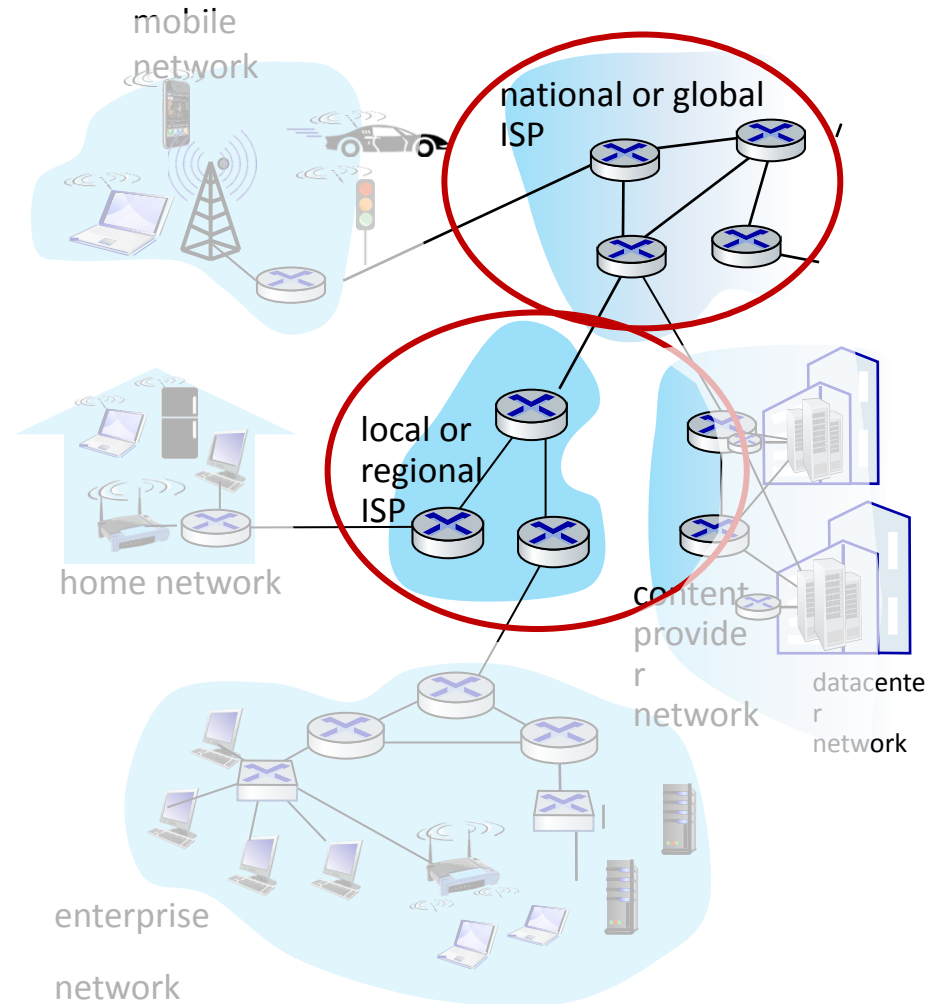
- signal carried in electromagnetic spectrum
- no physical “wire”
- broadcast and “half-duplex” (sender to receiver)
- propagation environment effects:
  - reflection
  - obstruction by objects
  - interference

### Radio link types:

- **terrestrial microwave**
  - up to 45 Mbps channels
- **Wireless LAN (WiFi)**
  - Up to 100's Mbps
- **wide-area** (e.g., cellular)
  - 4G cellular: ~ 10's Mbps
- **satellite**
  - up to 45 Mbps per channel
  - 280 msec end-end delay
  - geosynchronous vs. low-earth-orbit

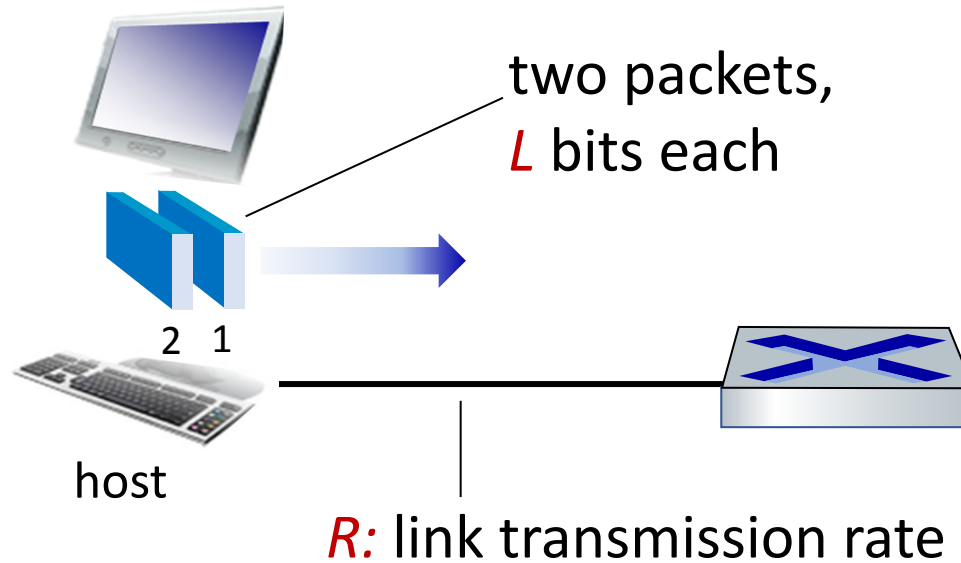


- mesh of interconnected routers
- **packet-switching**: hosts break application-layer messages into *packets*
  - forward packets from one router to the next, across links on path from source to destination
  - each packet transmitted at full link capacity



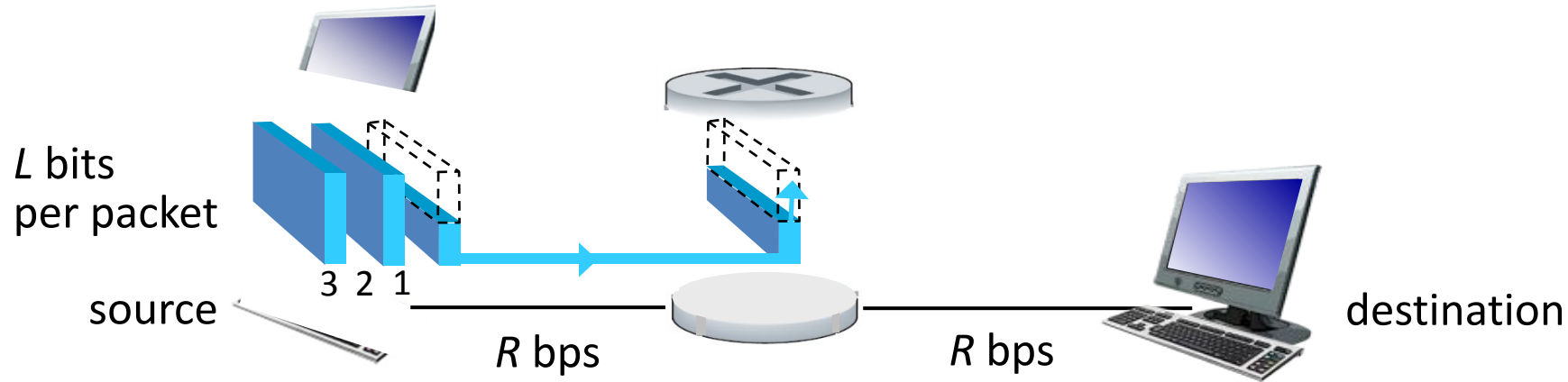
Host sending function:

- takes application message
- breaks into smaller chunks, known as *packets*, of length  $L$  bits
- transmits packet into access network at *transmission rate  $R$* 
  - link transmission rate, aka link *capacity, aka link bandwidth*



$$\begin{array}{l} \text{packet} \\ \text{transmission} \\ \text{delay} \end{array} = \begin{array}{l} \text{time needed to} \\ \text{transmit } L\text{-bit} \\ \text{packet into link} \end{array} = \frac{L \text{ (bits)}}{R \text{ (bits/sec)}}$$

## Network Core: Packet Switching: store-and-forward

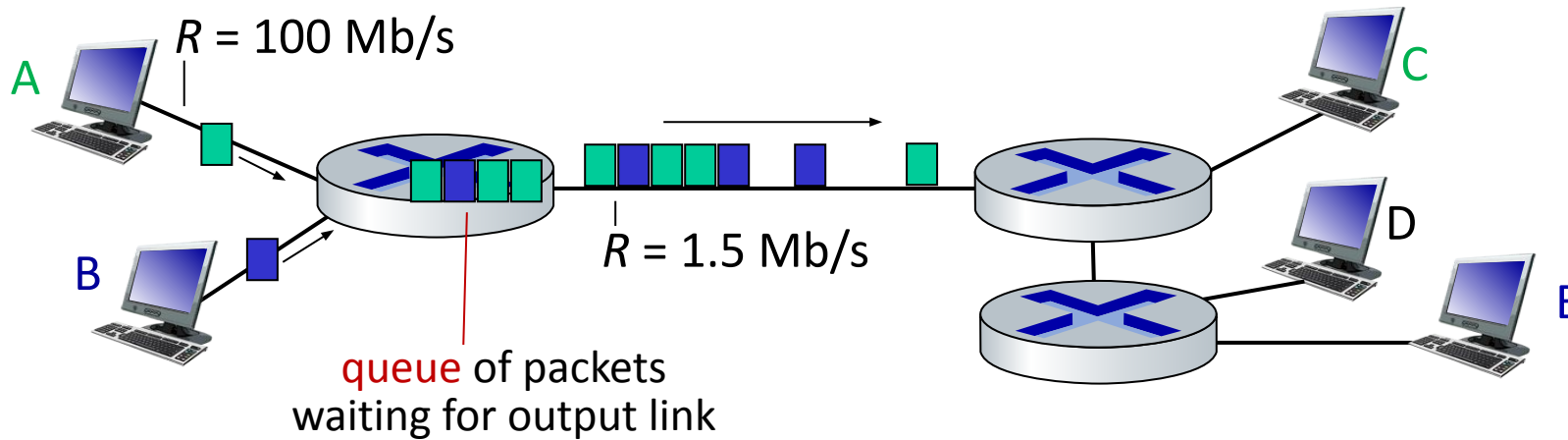


- **Transmission delay:** takes  $L/R$  seconds to transmit (push out)  $L$ -bit packet into link at  $R$  bps
- **Store and forward:** entire packet must arrive at router before it can be transmitted on next link
- **End-end delay:**  $2L/R$  (above), assuming zero propagation delay (more on delay shortly)

### *One-hop numerical example:*

- $L = 10$  Kbits
- $R = 100$  Mbps
- one-hop transmission delay = 0.1 msec

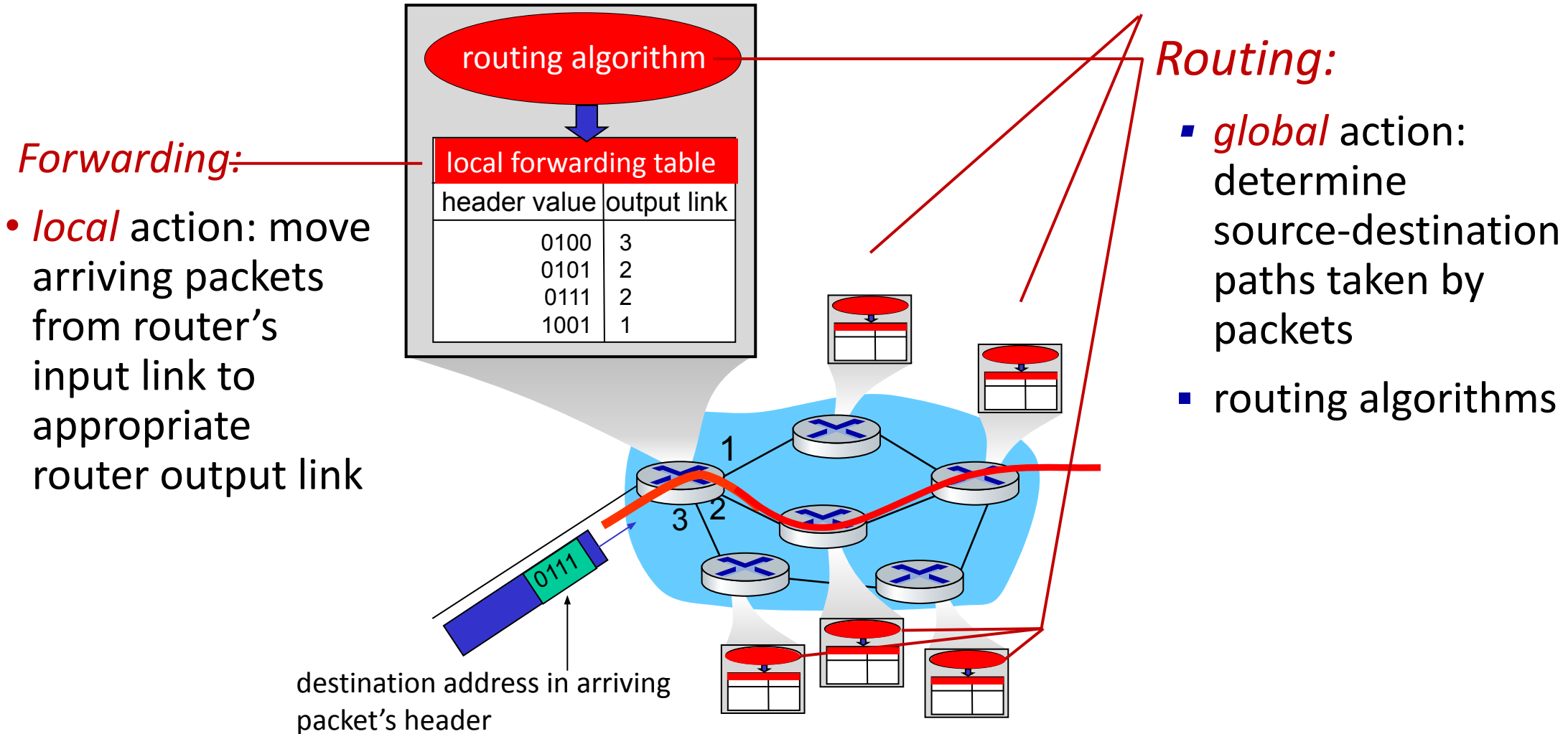
## Network Core: Packet Switching: queuing delay, loss



***Packet queuing and loss:*** if arrival rate (in bps) to link exceeds transmission rate (bps) of link for a period of time:

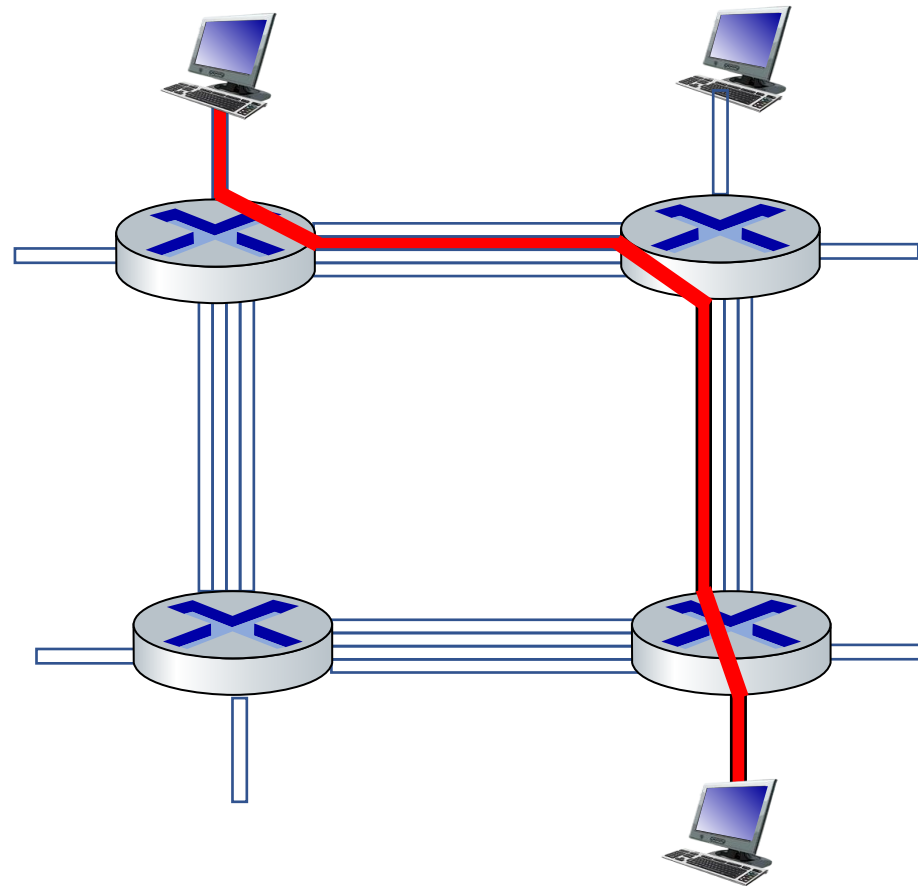
- packets will queue, waiting to be transmitted on output link
- packets can be dropped (lost) if memory (buffer) in router fills up

## Network Core: Two Key Network Core Functions



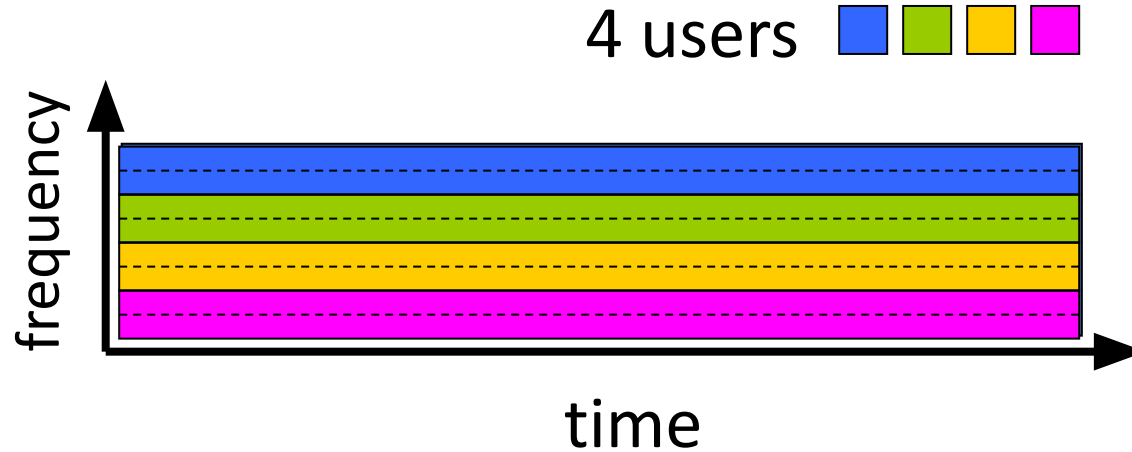
end-end resources allocated to, reserved for “call” between source and destination (eg: telephone)

- in diagram, each link has four circuits.
  - call gets 2<sup>nd</sup> circuit in top link and 1<sup>st</sup> circuit in right link.
- dedicated resources: no sharing
  - circuit-like (guaranteed) performance
- circuit segment idle if not used by call (no sharing)
- commonly used in traditional telephone networks



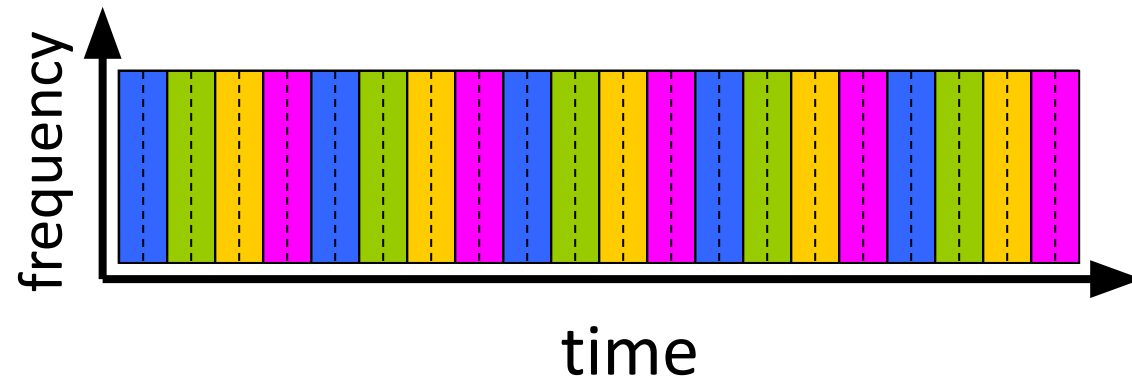
### Frequency Division Multiplexing (FDM)

- optical, electromagnetic frequencies divided into (narrow) frequency bands
- each call allocated its own band, can transmit at max rate of that narrow band



### Time Division Multiplexing (TDM)

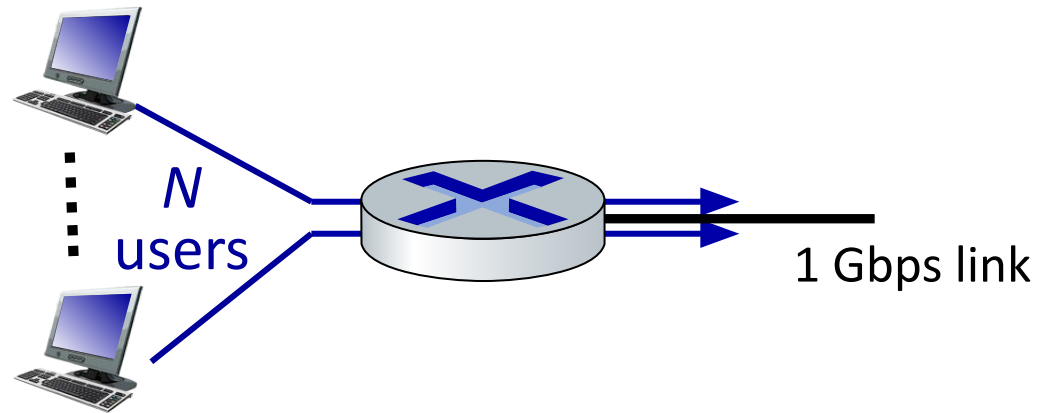
- time divided into frames -> slots
- each call allocated periodic slot(s), can transmit at maximum rate of (wider) frequency band, but only during its time slot(s)



*packet switching allows more users to use network!*

Example:

- 1 Gb/s link
- each user:
  - 100 Mb/s when “active”
  - active 10% of time



- *circuit-switching*: 10 users
- *packet switching*: with 35 users,
  - probability > 10 active users at same time is less than .0004 \*
  - 10 or few active users, probability 0.9996

*Q*: how did we get value 0.0004?

*Q*: what happens if > 35 users ?

\* Check out the online interactive exercises for more examples: [http://gaia.cs.umass.edu/kurose\\_ross/interactive](http://gaia.cs.umass.edu/kurose_ross/interactive)

### Is packet switching a “slam dunk winner”?

- great for “bursty” data – sometimes has data to send, but at other times not
  - resource sharing
  - simpler, no call setup
- **excessive congestion possible:** packet delay and loss due to buffer overflow
  - protocols needed for reliable data transfer, congestion control
- **Q: How to provide circuit-like behavior?**
  - bandwidth guarantees traditionally used for audio/video applications

**Q:** human analogies of reserved resources (circuit switching) versus on-demand allocation (packet switching)?

## Packet Switching vs Circuit Switching – Numerical Example

---

- How long does it take to send a file of 640,000 bits (1 byte = 8 bits) from host A to host B over a circuit-switched network?
  - All links are 1.536 Mbps
  - Each link uses TDM with 24 slots/sec
  - 500 msec to establish end-to-end circuit

**Let's work it out!**



- How long does it take to send a file of 640,000 bits (1 byte = 8 bits) from host A to host B over a circuit-switched network?
  - All links are 1.536 Mbps
  - Each link uses TDM with 24 slots/sec
  - 500 msec to establish end-to-end circuit

**Let's work it out!**

**Solution:**

- Each circuit has a transmission rate of  $(1.536 \text{ Mbps})/24 = 64 \text{ kbps}$
- It takes  $(640,000 \text{ bits})/(64 \text{ kbps}) = 10 \text{ seconds}$  to transmit the file
- To this 10 seconds we add the circuit establishment time, giving 10.5 seconds to send the file



## Network Core: Packet Switching vs Circuit Switching

---

- Connectionless
  - Designed for data
  - Flexible
  - Out of order, assembled at the dest
  - Forward, Store & Fwd
  - Network layer
  - Bandwidth is saved (dynamic)
  - Transmission of data – Source, routers
  - Transmission delay
- Connection oriented
  - Designed for voice
  - Inflexible
  - Message received in same order
  - FDM & TDM
  - Physical layer
  - Bandwidth is wasted (fixed)
  - Transmission of data – source
  - Call setup delay

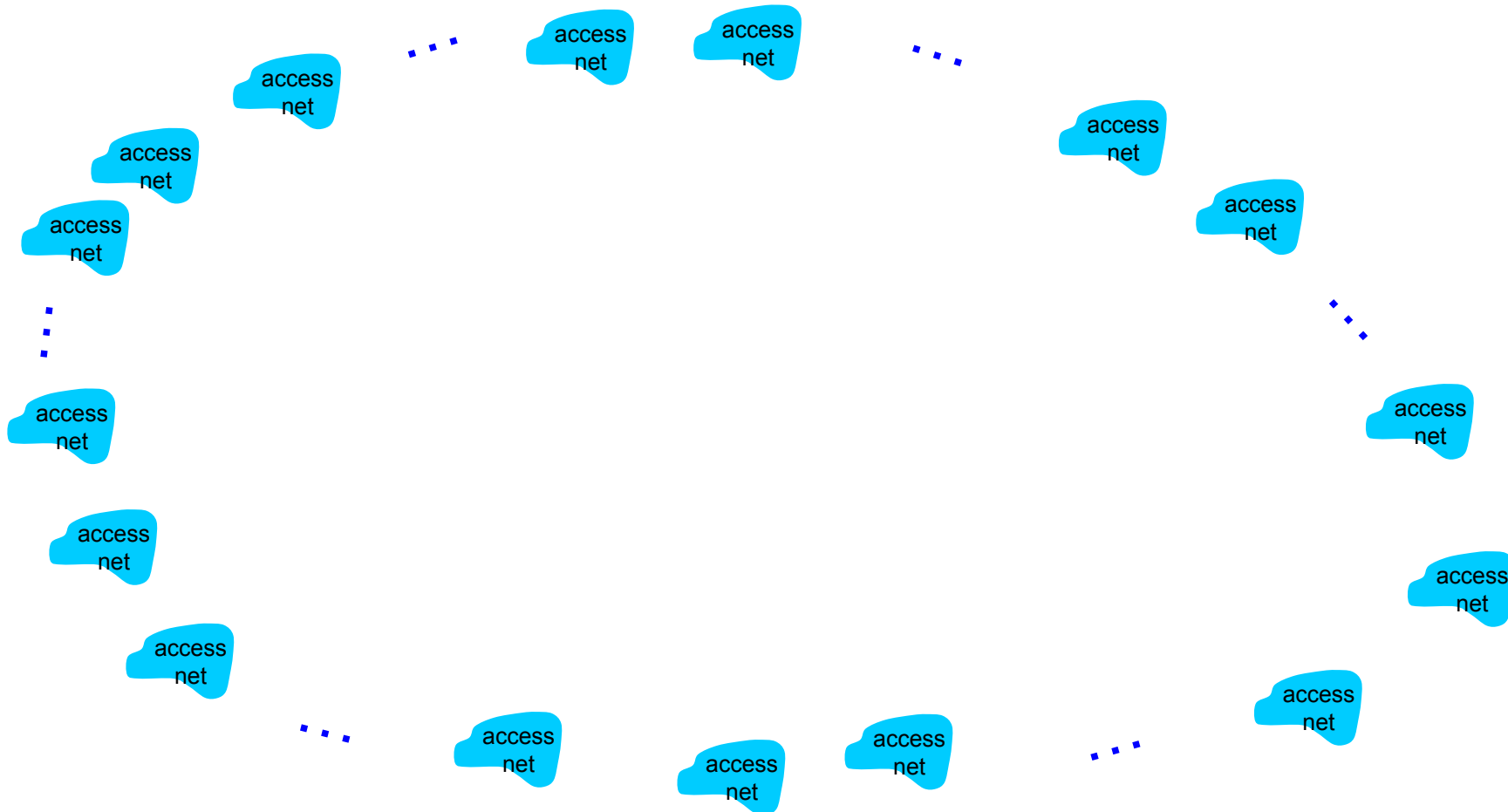
## Internet Structure: a “network of networks”

---

- ❖ End systems connect to Internet via **access ISPs** (Internet Service Providers)
  - Residential, company and university ISPs
- ❖ Access ISPs in turn must be interconnected.
  - ❖ So that any two hosts can send packets to each other
- ❖ Resulting network of networks is very complex
  - ❖ Evolution was driven by **economics** and **national policies**
- ❖ Let's take a stepwise approach to describe current Internet structure

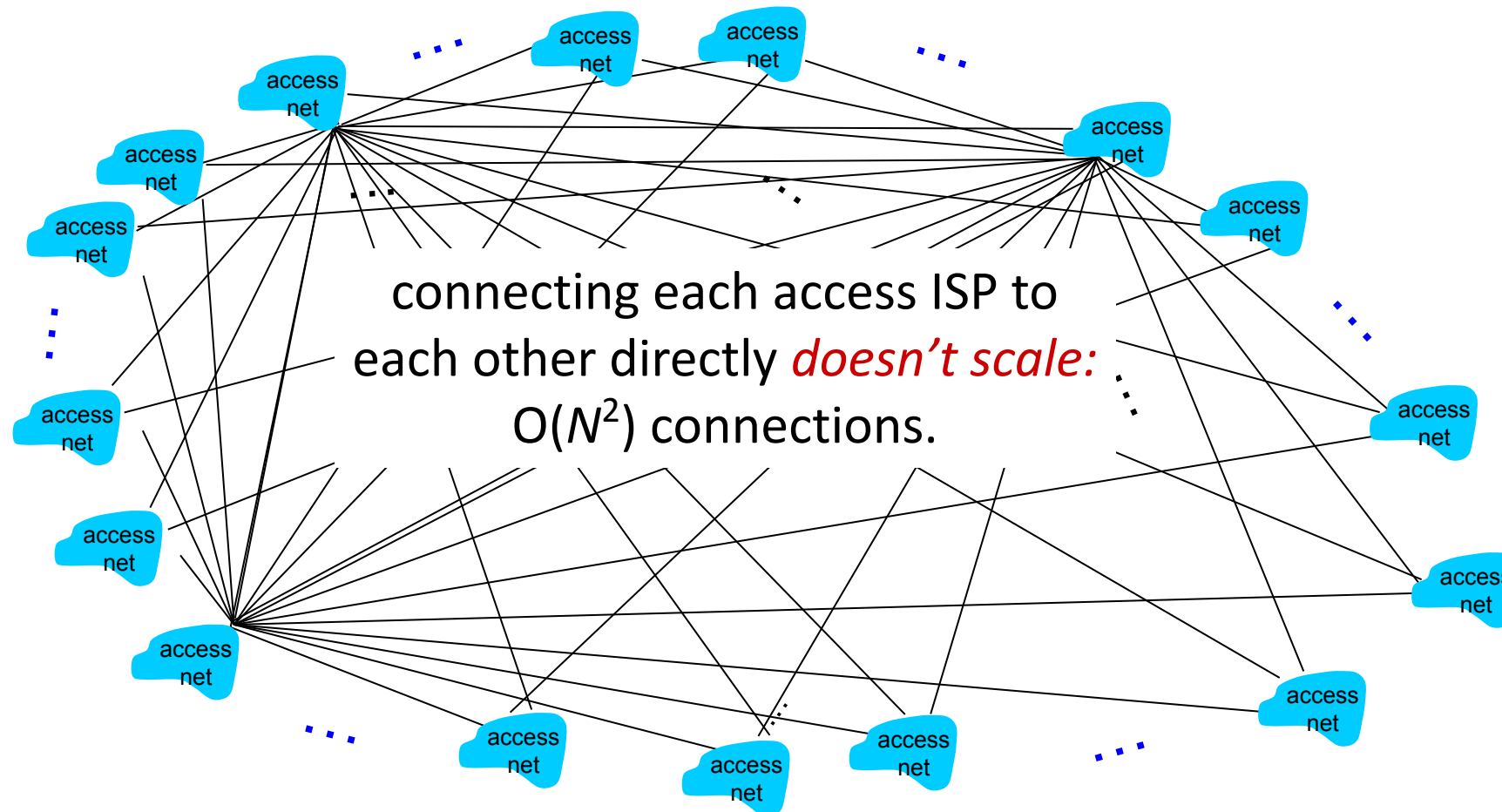
## Internet Structure: a “network of networks”

*Question:* given *millions* of access ISPs, how to connect them together?



## Internet Structure: a “network of networks”

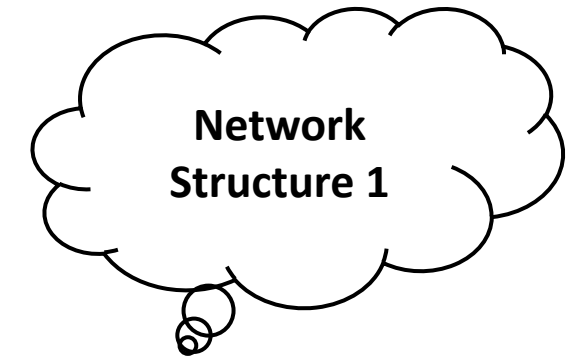
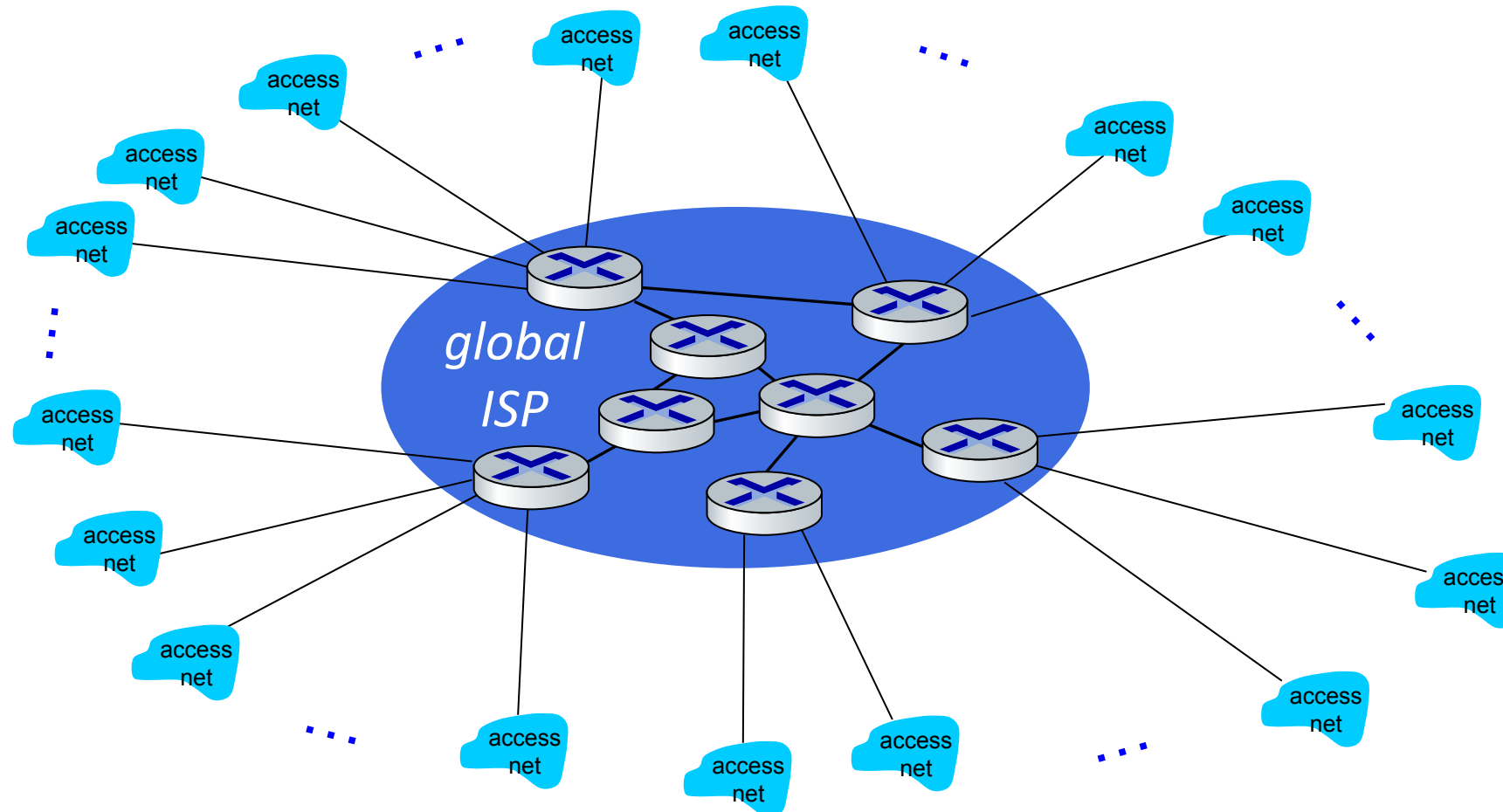
*Question:* given *millions* of access ISPs, how to connect them together?



## Internet Structure: a “network of networks”

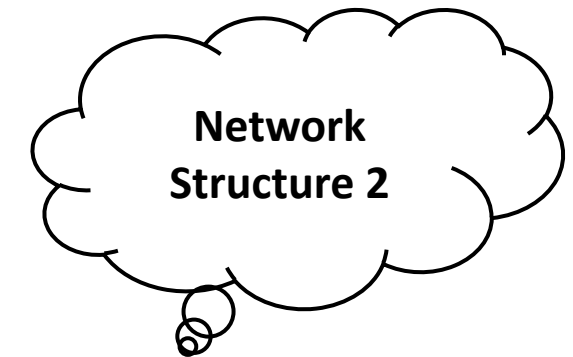
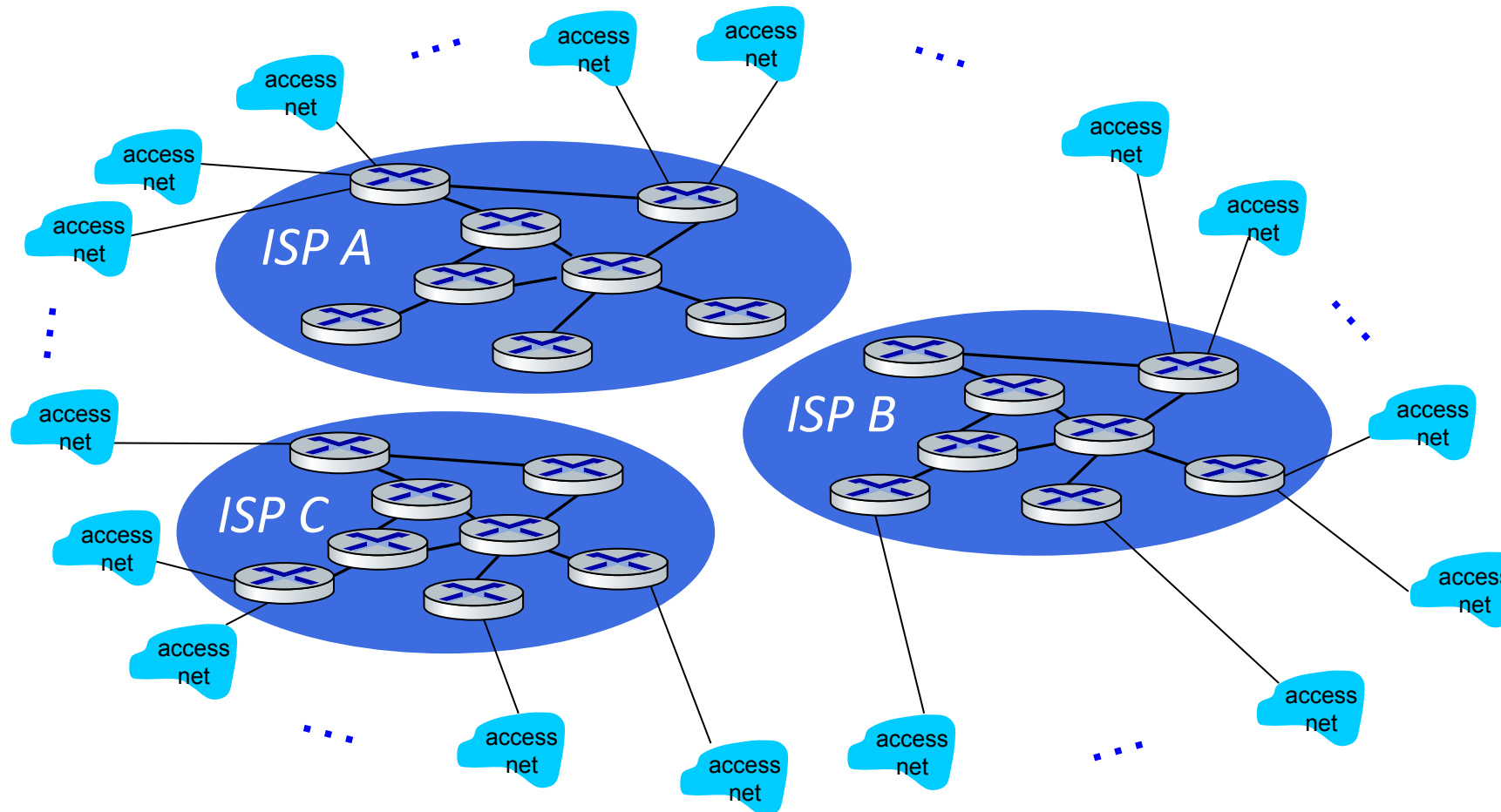
*Option: connect each access ISP to one global transit ISP?*

*Customer and provider ISPs have economic agreement.*



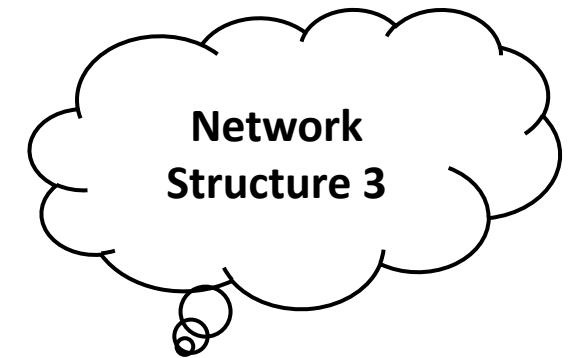
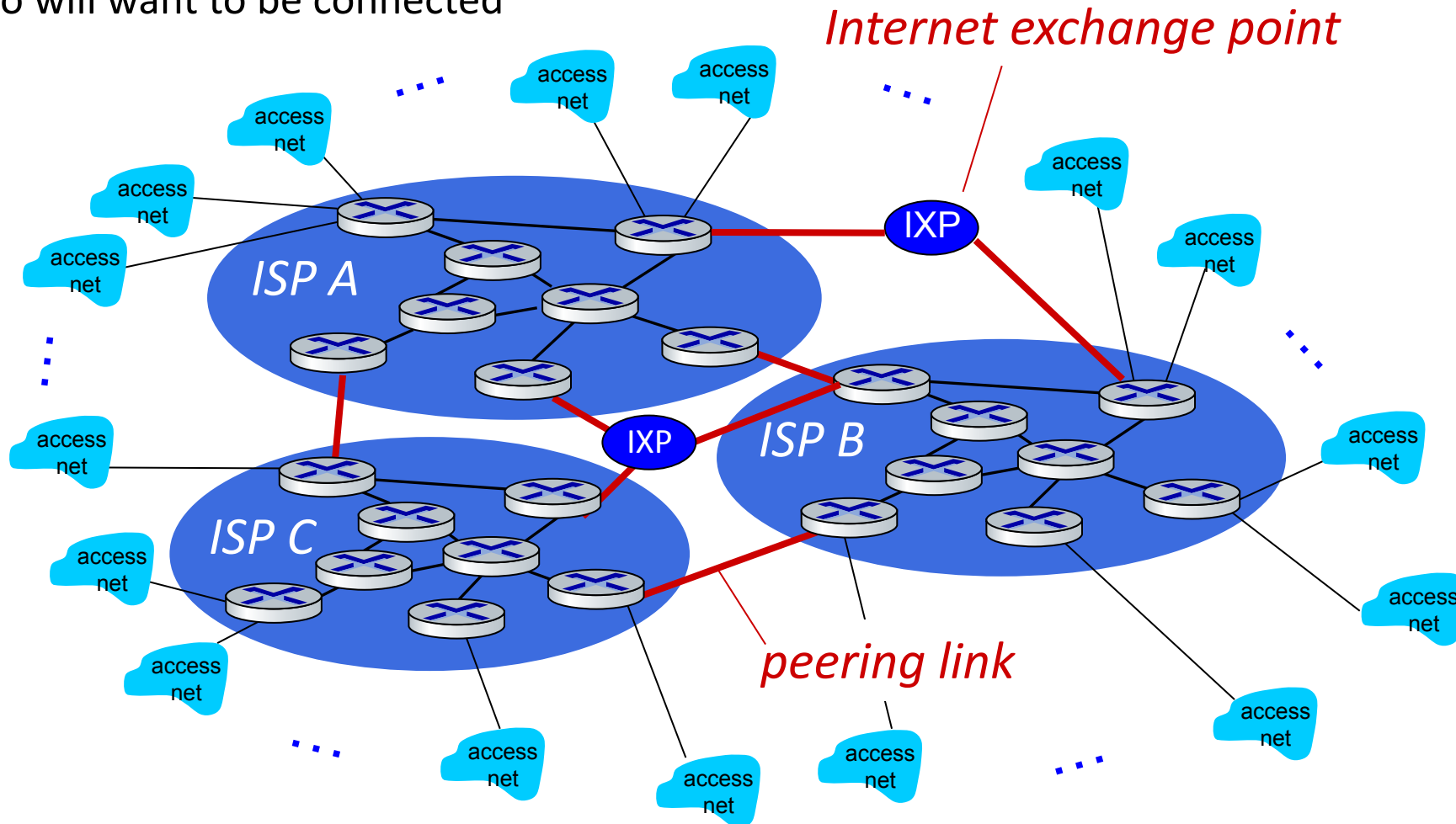
## Internet Structure: a “network of networks”

But if one global ISP is viable business, there will be competitors ....



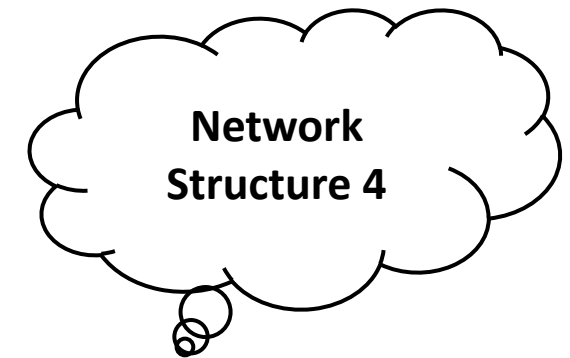
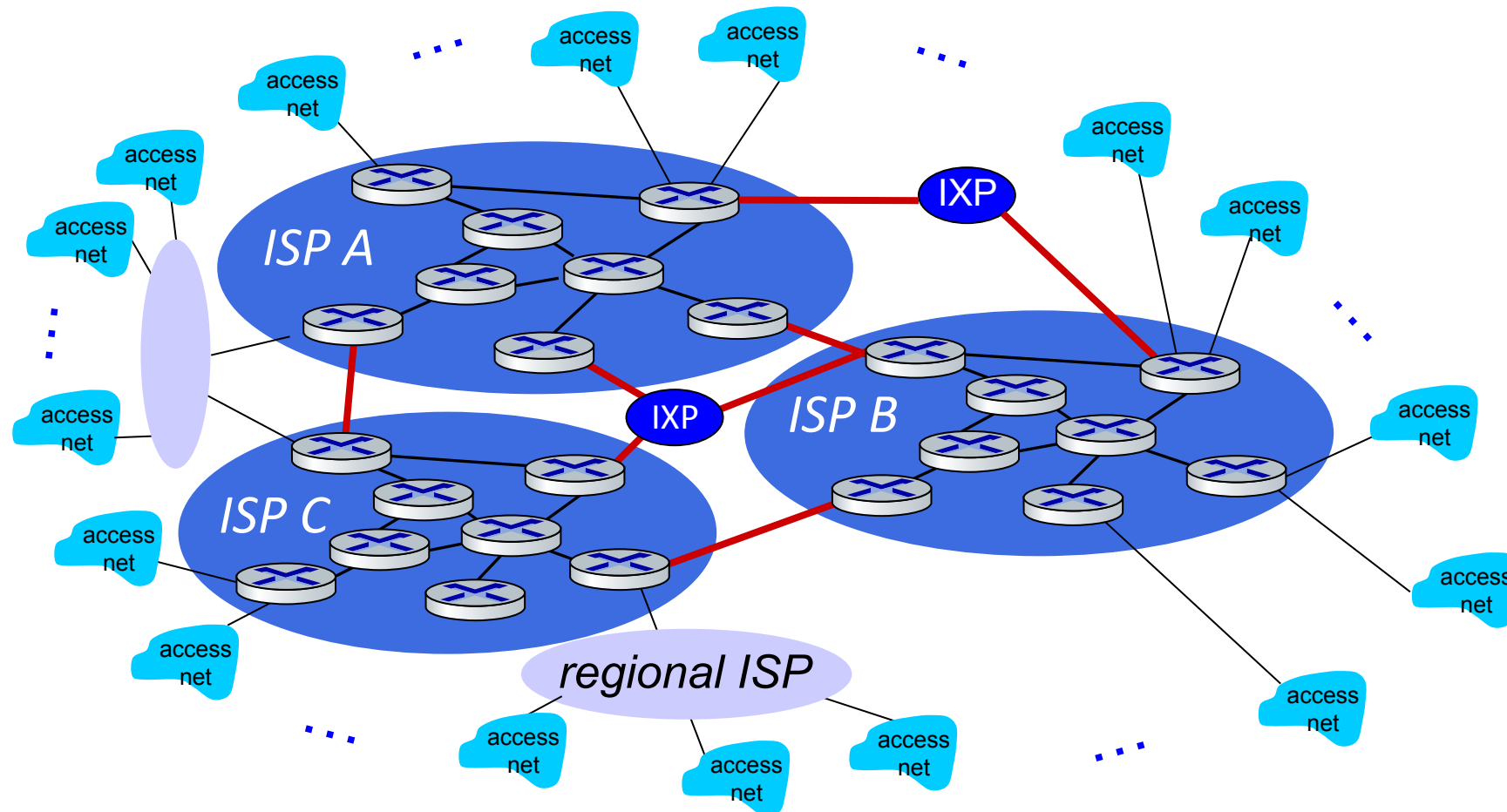
## Internet Structure: a “network of networks”

But if one global ISP is viable business, there will be competitors ....  
who will want to be connected



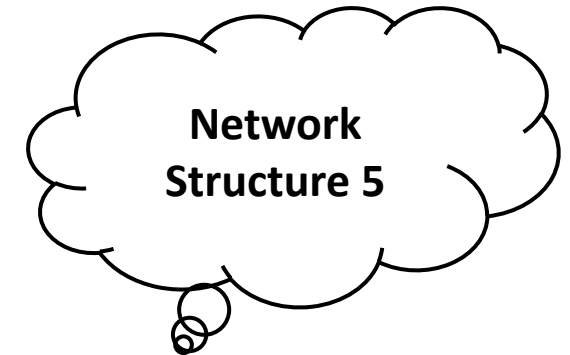
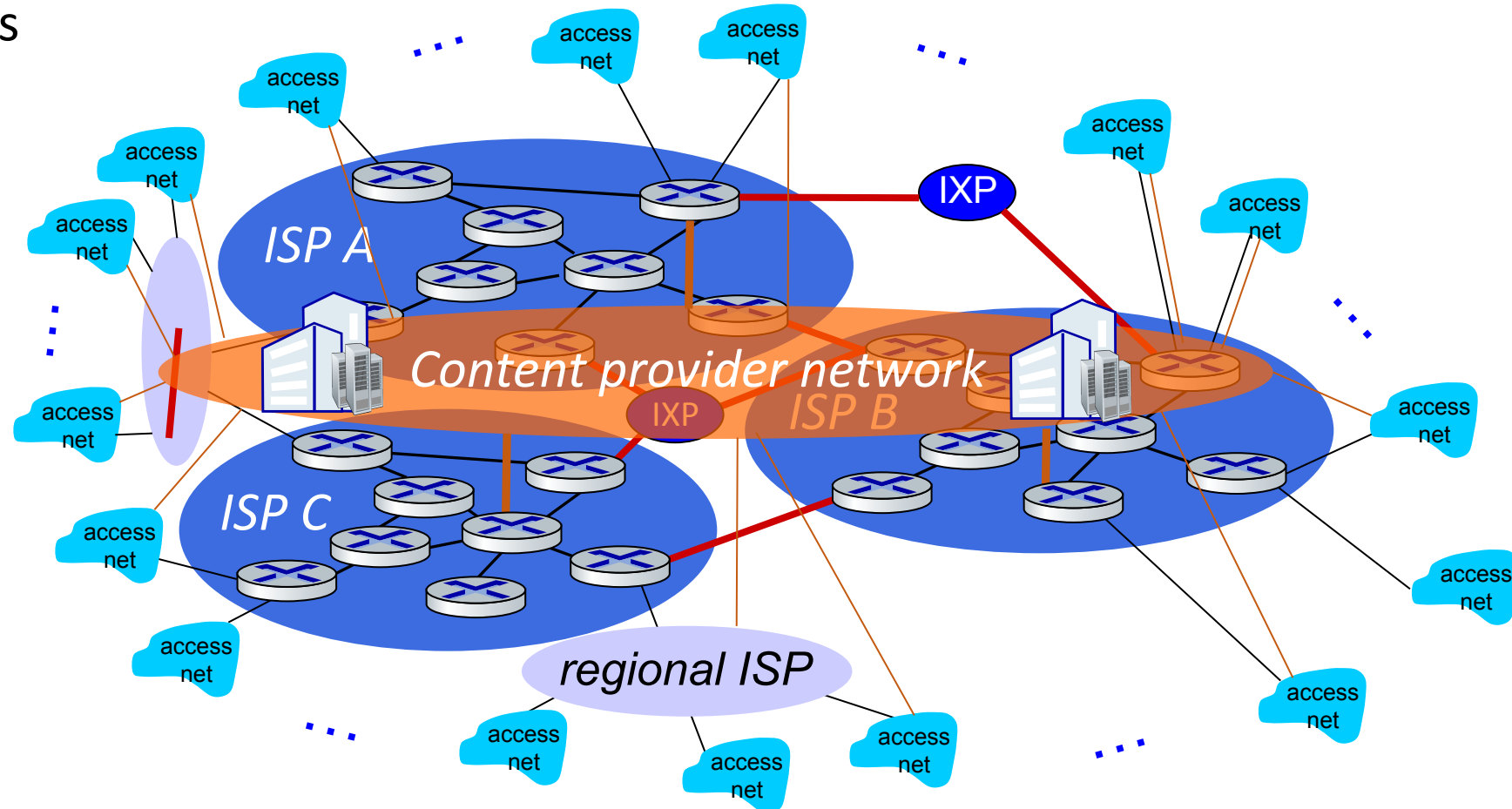
## Internet Structure: a “network of networks”

... and regional networks may arise to connect access nets to ISPs

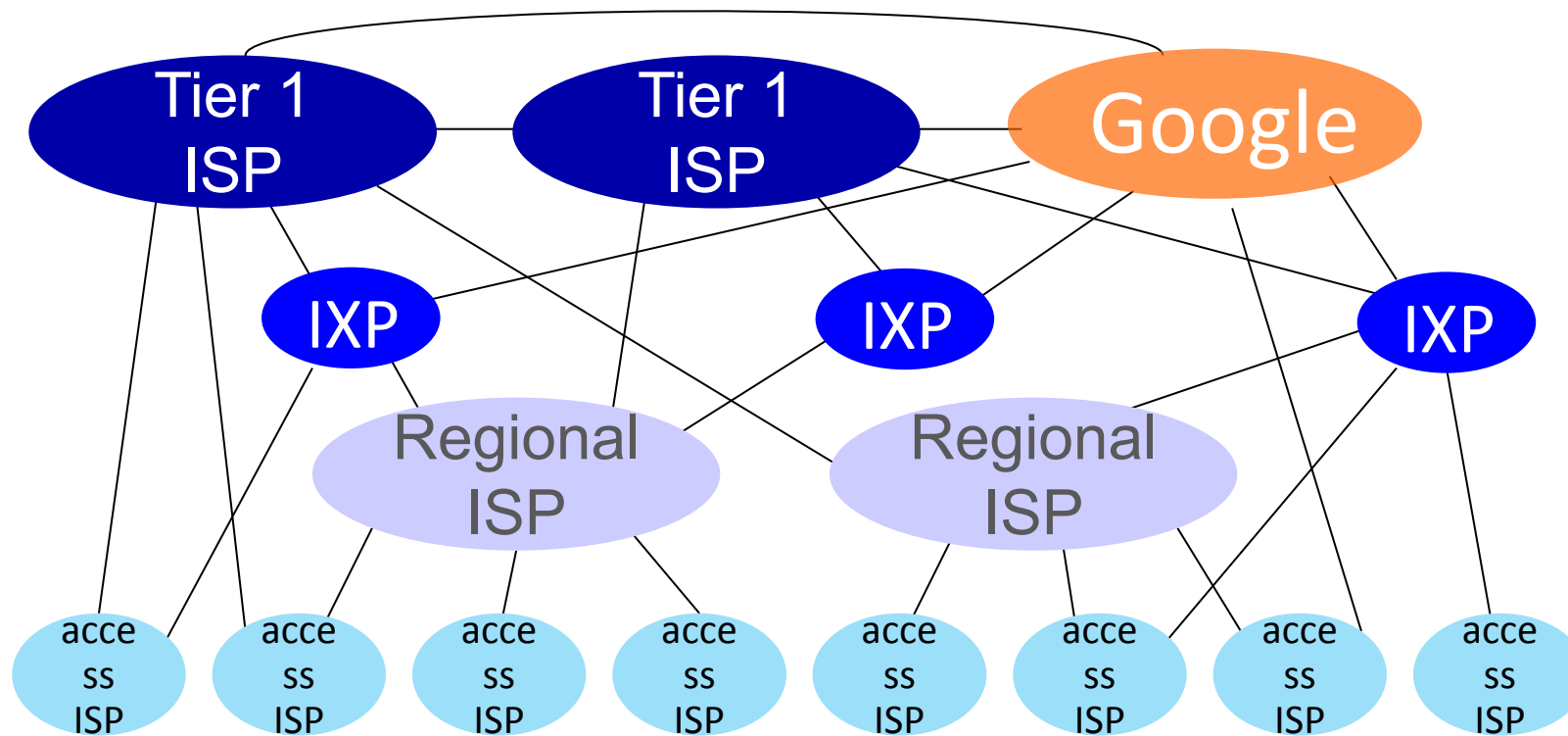


## Internet Structure: a “network of networks”

... and content provider networks (e.g., Google, Microsoft, Akamai) may run their own network, to bring services, content close to end users



## Internet Structure: a “network of networks”

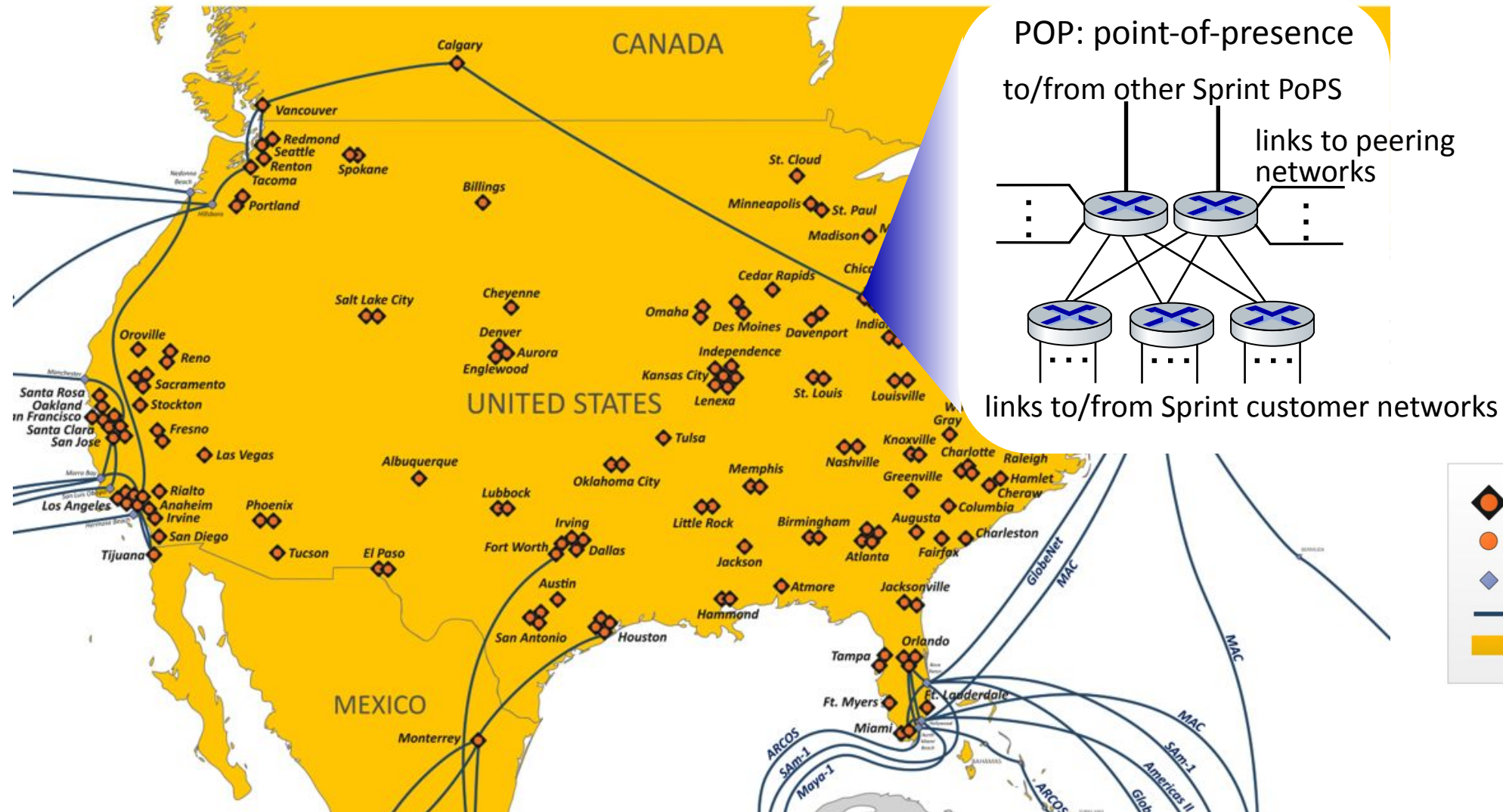


At “center”: small # of well-connected large networks

- **“tier-1” commercial ISPs** (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage
- **content provider networks** (e.g., Google, Facebook): private network that connects its data centers to Internet, often bypassing tier-1, regional ISPs

# COMPUTER NETWORKS

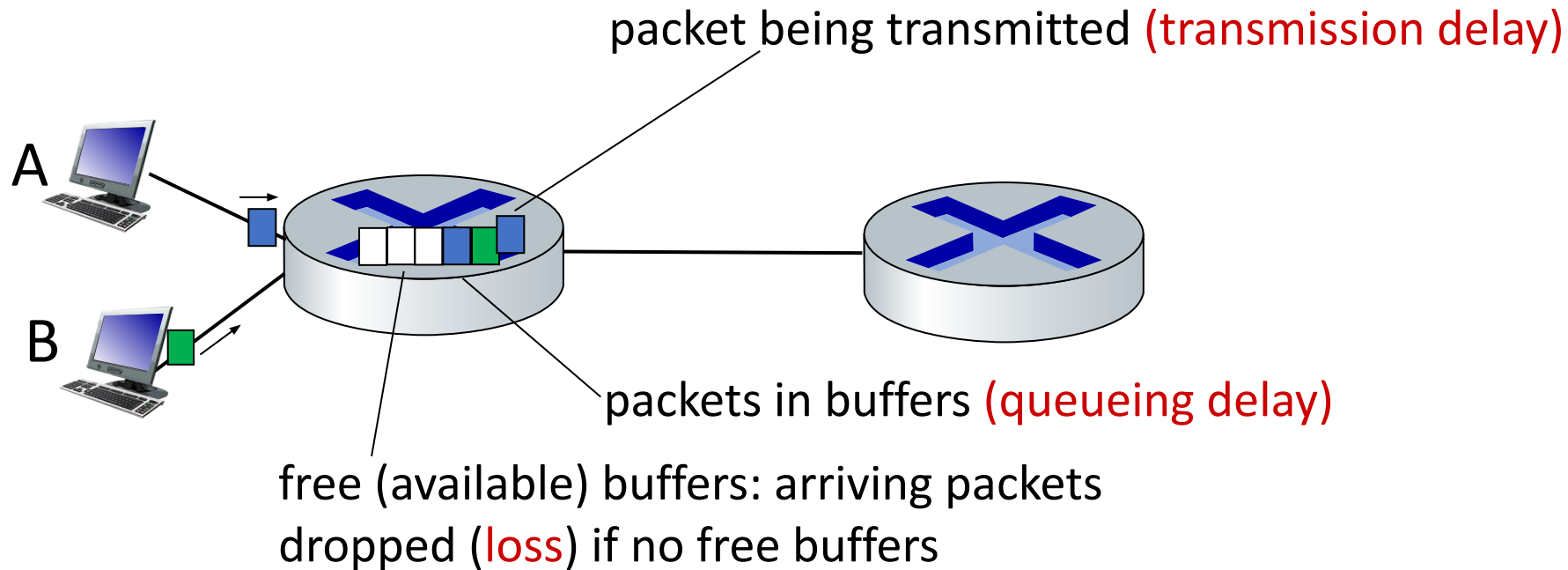
## Network Core: Tier 1 ISP Network Map: Sprint 2019

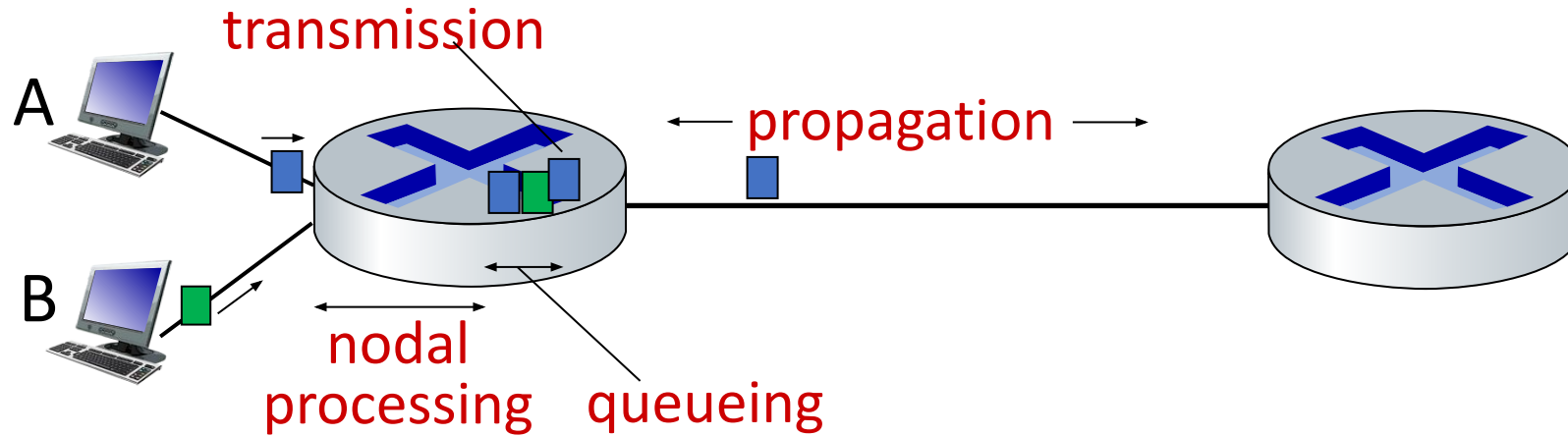


### How do packet loss and delay occurs?

packets *queue* in router buffers

- packets queue, wait for turn
- arrival rate to link (temporarily) exceeds output link capacity: packet loss





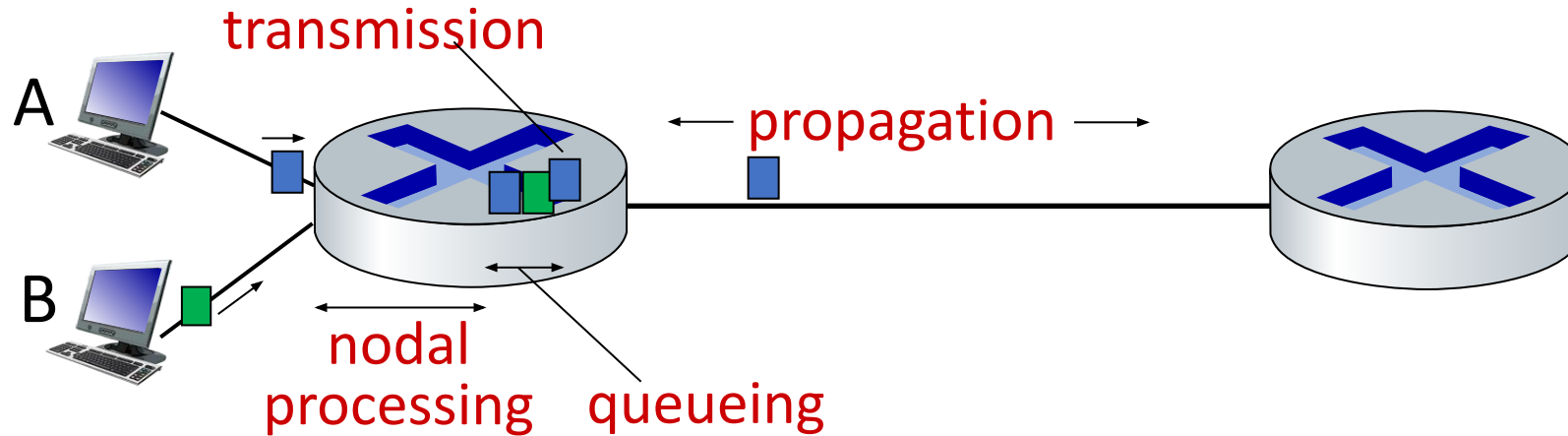
$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

$d_{\text{proc}}$ : nodal processing

- check bit errors
- determine output link
- typically < msec

$d_{\text{queue}}$ : queueing delay

- time waiting at output link for transmission
- depends on congestion level of router
- microseconds to milliseconds



\* Check out the online interactive exercises:  
[http://gaia.cs.umass.edu/kurose\\_ross](http://gaia.cs.umass.edu/kurose_ross)

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

$d_{\text{trans}}$ : transmission delay:

- $L$ : packet length (bits)
- $R$ : link transmission rate (bps)

- $d_{\text{trans}} = L/R$

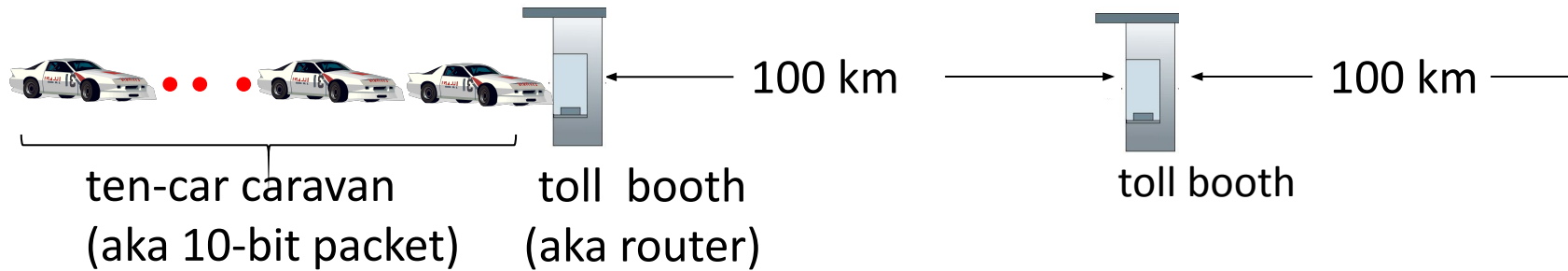
$d_{\text{prop}}$ : propagation delay:

- $d$ : length of physical link
- $s$ : propagation speed ( $\sim 2 \times 10^8$  m/sec)

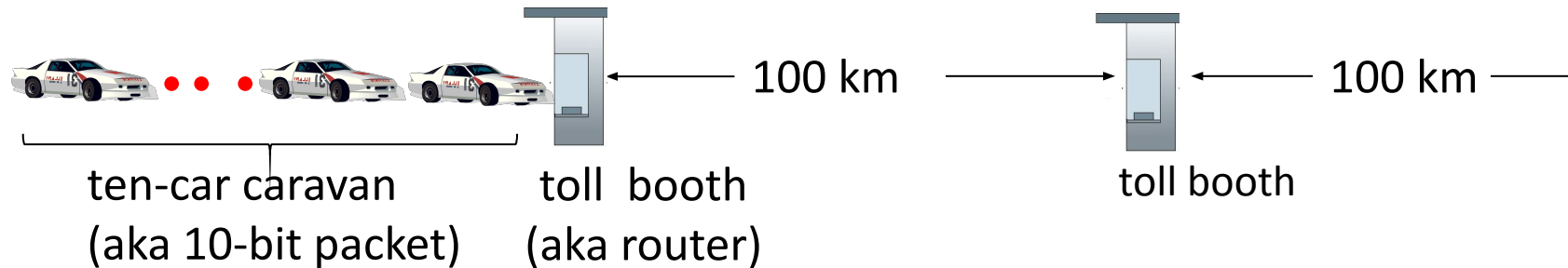
- $d_{\text{prop}} = d/s$

$d_{\text{trans}}$  and  $d_{\text{prop}}$   
very different

| Transmission Delay                                                                     | Propagation Delay                                                            |
|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Time required for the router to push out the packet.                                   | Time it takes a bit to propagate from one router to the next.                |
| A function of the <b>packet's length</b> and the <b>transmission rate</b> of the link. | A function of the <b>distance</b> between the two routers.                   |
| $d_{trans} = L/R$                                                                      | $d_{prop} = d/s$                                                             |
| Nothing to do with the distance between the two routers.                               | Nothing to do with the packet's length or the transmission rate of the link. |



- cars “propagate” at 100 km/hr
- toll booth takes 12 sec to service car (bit transmission time)
- car ~ bit; caravan ~ packet
- **Q: How long until caravan is lined up before 2nd toll booth?**
- time to “push” entire caravan through toll booth onto highway =  $12 \times 10 = 120$  sec
- time for last car to propagate from 1st to 2nd toll booth:  
 $100\text{km} / (100\text{km/hr}) = 1$  hr
- **A: 62 minutes**



- suppose cars now “propagate” at 1000 km/hr
- and suppose toll booth now takes one min to service a car
- **Q: Will cars arrive to 2nd booth before all cars serviced at first booth?**

**A: Yes!** after 7 min, first car arrives at second booth; three cars still at first booth

Unlike other delays ( $d_{proc}$ ,  $d_{trans}$ ,  $d_{prop}$ ),  $d_{queue}$  is interesting.

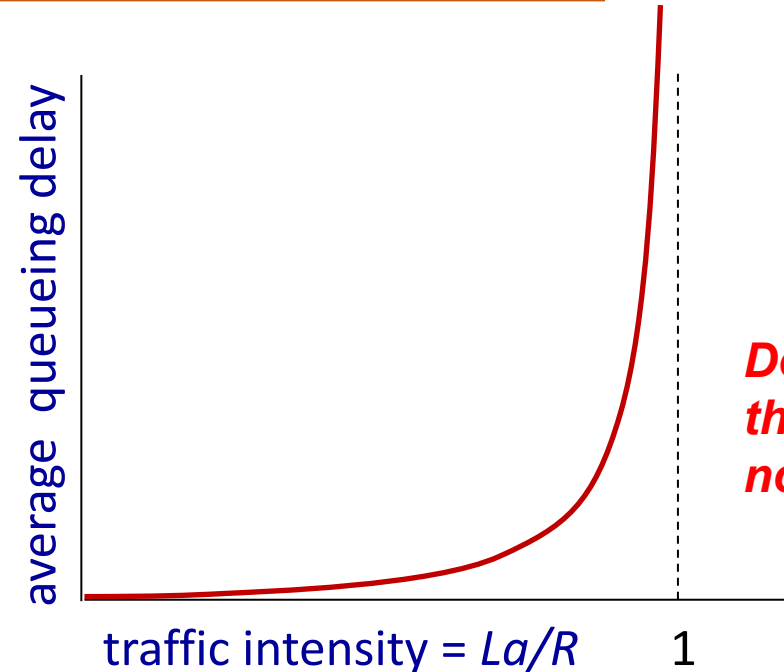
- Can vary from packet to packet.
- Characterize  $d_{queue}$  -> average, variance, probability that it exceeds some specified value.

When is the queuing delay large and when is it insignificant?

- Rate at which traffic arrives at the queue,
- Transmission rate of the link,
- Nature of the arriving traffic – periodically or in bursts

- $R$ : link bandwidth (bps)
- $L$ : packet length (bits)
- $a$ : average packet arrival rate (pps)
- $La$ : avg. rate at which bits arrive at the queue
- $La/R > 1$ : more “work” arriving is more than can be serviced - average delay infinite!
- $La/R \leq 1$ : nature of arriving traffic
- $La/R \sim 0$ : avg. queueing delay small

$La/R > 1$ : Average rate at which bits arrive at the queue exceeds the rate at which the bits can be transmitted from the queue.



***Design your system so that the traffic intensity is no greater than 1.***

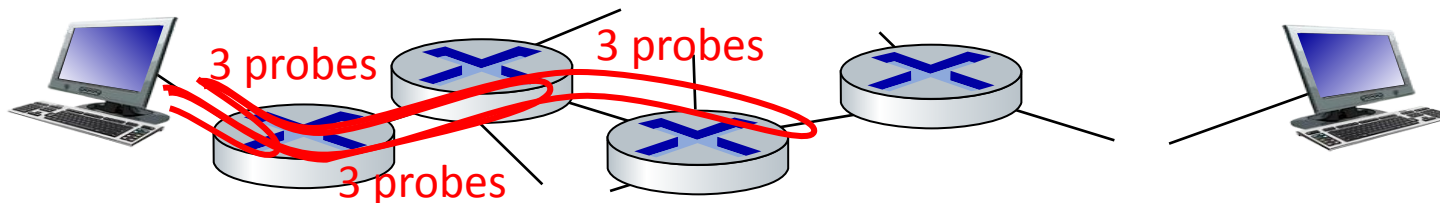


$La/R \sim 0$



$La/R \rightarrow 1$

- what do “real” Internet delay & loss look like?
- **traceroute** program: provides delay measurement from source to router along end-end Internet path towards destination.  
For all  $i$ :
  - sends three packets that will reach router  $i$  on path towards destination (with time-to-live field value of  $i$ )
  - router  $i$  will return packets to sender
  - sender measures time interval between transmission and reply



**traceroute:** gaia.cs.umass.edu to www.eurecom.fr

3 delay measurements from  
gaia.cs.umass.edu to cs-gw.cs.umass.edu

```
1 cs-gw (128.119.240.254) 1 ms 1 ms 2 ms
2 border1-rt-fa5-1-0.gw.umass.edu (128.119.3.145) 1 ms 1 ms 2 ms
3 cht-vbns.gw.umass.edu (128.119.3.130) 6 ms 5 ms 5 ms
4 jn1-at1-0-0-19.wor.vbns.net (204.147.132.129) 16 ms 11 ms 13 ms
5 jn1-so7-0-0-0.wae.vbns.net (204.147.136.136) 21 ms 18 ms 18 ms
6 abilene-vbns.abilene.ucaid.edu (198.32.11.9) 22 ms 18 ms 22 ms
7 nycm-wash.abilene.ucaid.edu (198.32.8.46) 22 ms 22 ms 22 ms
8 62.40.103.253 (62.40.103.253) 104 ms 109 ms 106 ms
9 de2-1.de1.de.geant.net (62.40.96.129) 109 ms 102 ms 104 ms
10 de.fr1.fr.geant.net (62.40.96.50) 113 ms 121 ms 114 ms
11 renater-gw.fr1.fr.geant.net (62.40.103.54) 112 ms 114 ms 112 ms
12 nio-n2.cssi.renater.fr (193.51.206.13) 111 ms 114 ms 116 ms
13 nice.cssi.renater.fr (195.220.98.102) 123 ms 125 ms 124 ms
14 r3t2-nice.cssi.renater.fr (195.220.98.110) 126 ms 126 ms 124 ms
15 eurecom-valbonne.r3t2.ft.net (193.48.50.54) 135 ms 128 ms 133 ms
16 194.214.211.25 (194.214.211.25) 126 ms 128 ms 126 ms
17 * * *
18 * * *
```

3 delay measurements  
to border1-rt-fa5-1-0.gw.umass.edu

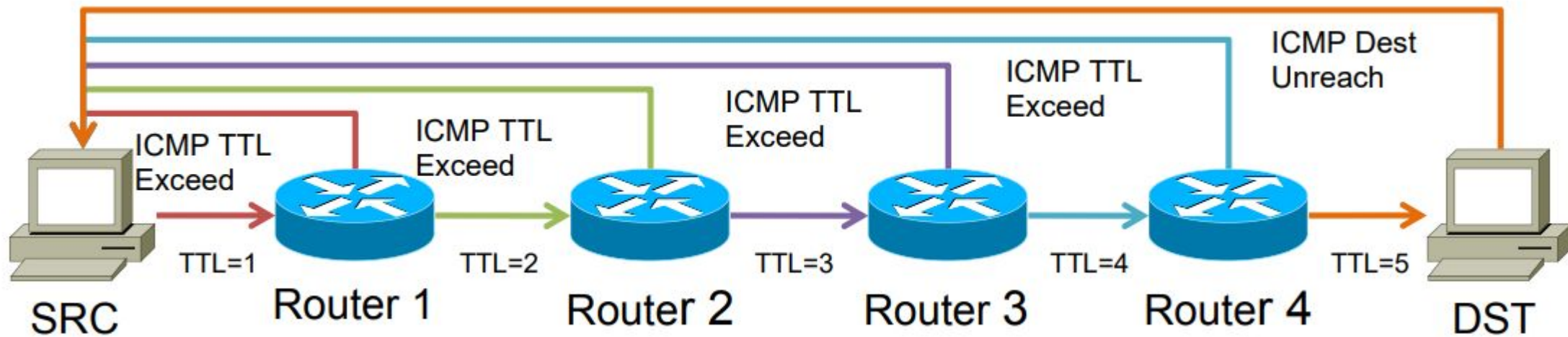
trans-oceanic link

looks like delays  
decrease! Why?

\* means no response (probe lost, router not replying)

19 fantasia.eurecom.fr (193.55.113.142) 132 ms 128 ms 136 ms

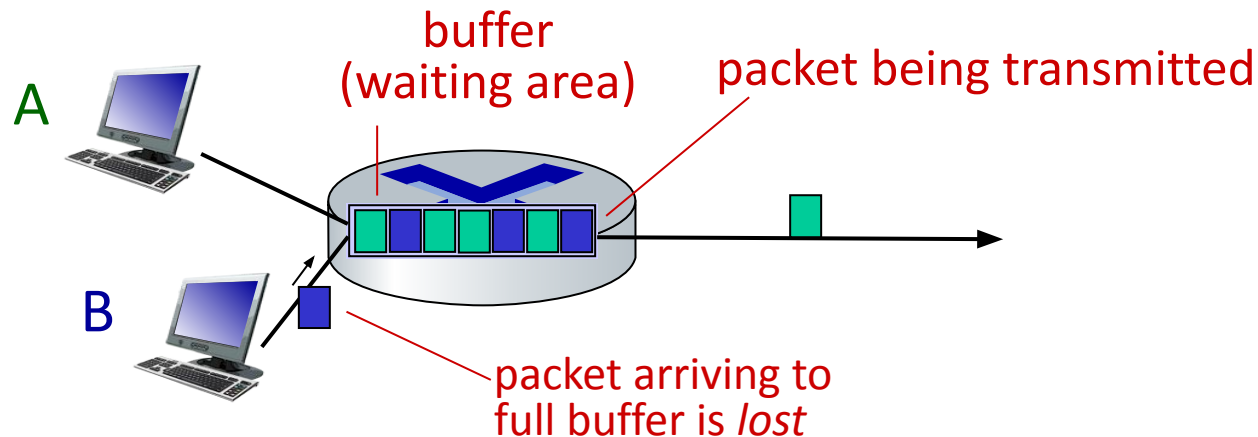
\* Do some traceroutes from exotic countries at [www.traceroute.org](http://www.traceroute.org)



Refer RFC 1393, **Traceroute Using an IP Option**

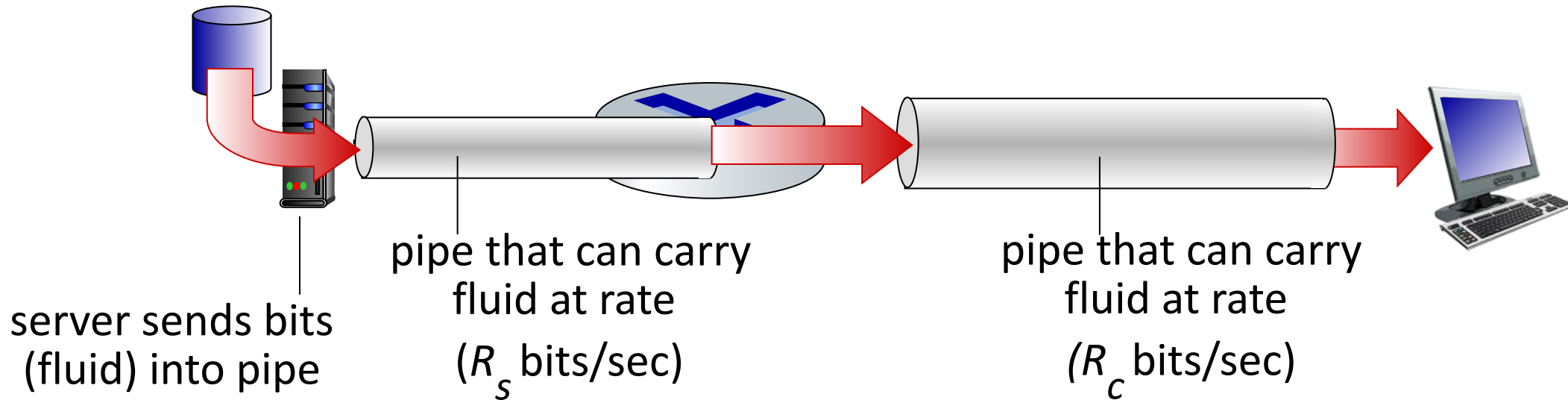
**Don't Trust Traceroute (Completely)**

- queue (aka buffer) preceding link in buffer has finite capacity
- packet arriving to full queue dropped (aka lost)
- lost packet may be retransmitted by previous node, by source end system, or not at all

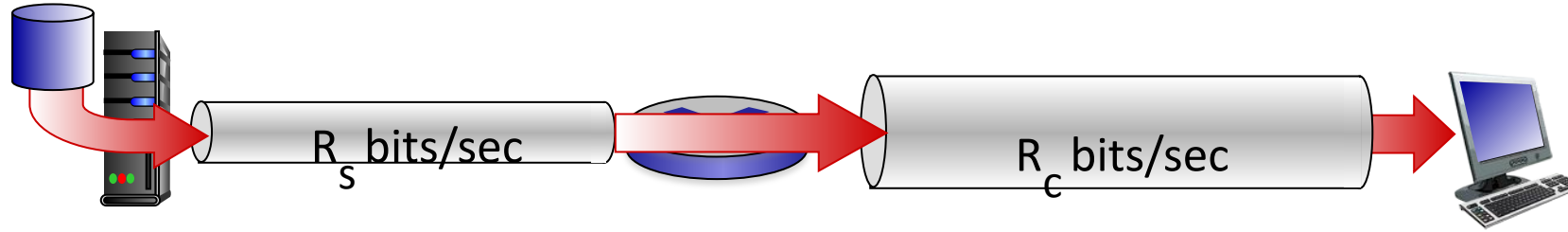


\* Check out the Java applet for an interactive animation on queuing and loss

- **throughput:** rate (bits/time unit) at which bits are being sent from sender to receiver
  - **instantaneous:** rate at given point in time
  - **average:** rate over longer period of time

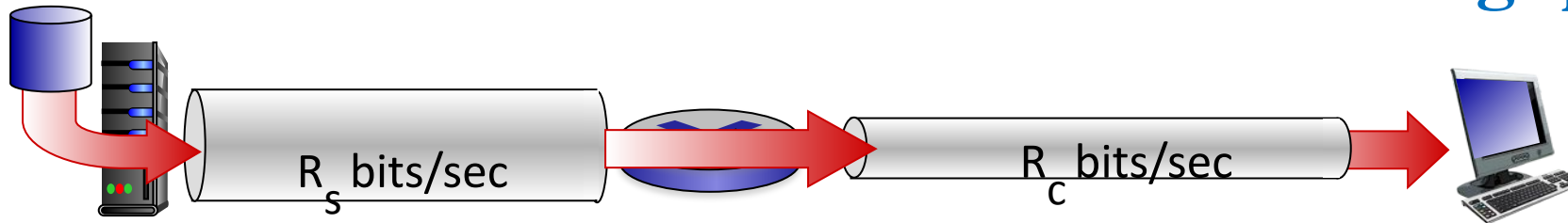


$R_s < R_c$  What is average end-end throughput?



$R_s > R_c$  What is average end-end throughput?

**Throughput =  $\min\{R_s, R_c\}$**



*bottleneck link*

link on end-end path that constrains end-end throughput.

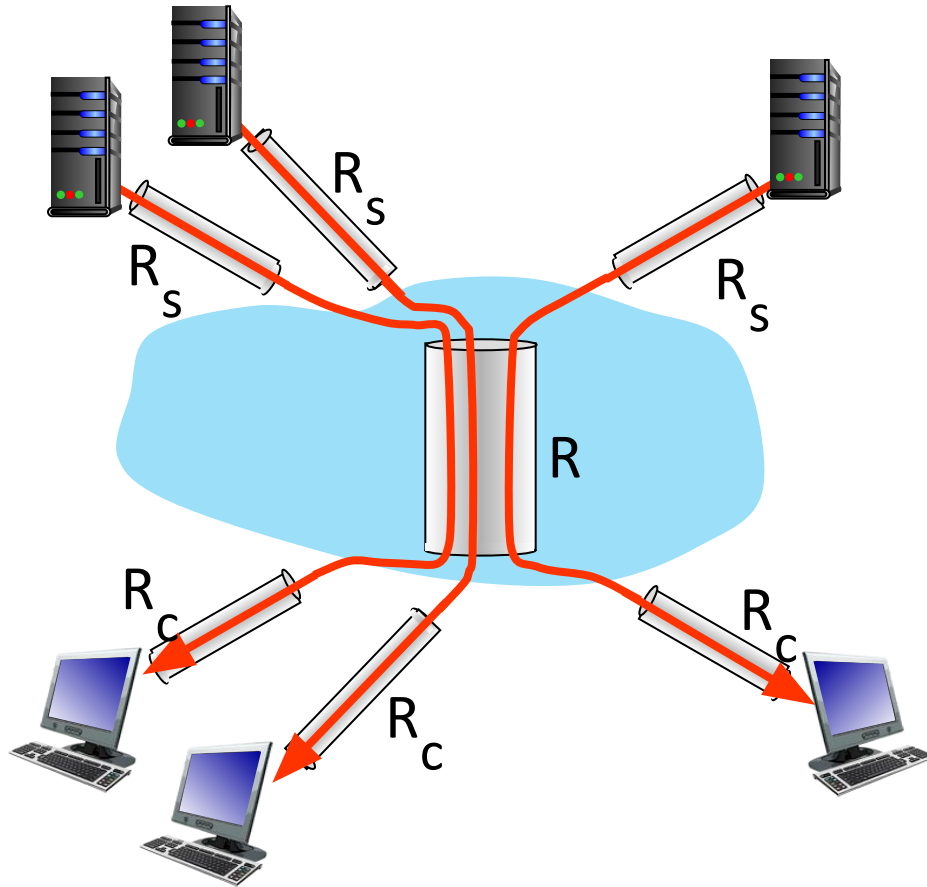
- Suppose you are downloading an MP3 file of  $F = 32$  million bits.
- The server has a transmission rate of  $R_s = 2$  Mbps and you have an access link of  $R_c = 1$  Mbps.
- What is the time needed to transfer the file?

**Let's work it out!**

**Solution:**

- 32 seconds!





10 connections (fairly) share backbone bottleneck link  $R$  bits/sec

- per-connection end-end throughput:  $\min(R_c, R_s, R/10)$
- in practice:  $R_c$  or  $R_s$  is often bottleneck

\* Check out the online interactive exercises for more examples:  
[http://gaia.cs.umass.edu/kurose\\_ross/](http://gaia.cs.umass.edu/kurose_ross/)

- Suppose  $R_s = 2$  Mbps,  $R_c = 1$  Mbps,  $R = 5$  Mbps
- 10 clients from 10 servers = 10 downloads

End-to-end throughput for each download is now reduced to 500 kbps.



# Computer Networks and the Internet

---

## Protocol Layers

### Team Networks

Department of Computer Science and Engineering

Networks are complex,  
with many “pieces”:

- hosts
- routers
- links of various media
- applications
- protocols
- hardware, software

*Question:*

is there any hope of  
*organizing* structure of  
network?

.... or at least our *discussion*  
of networks?

# COMPUTER NETWORKS

## Example: Organization of Air Travel



ticket (purchase)

baggage (check)

gates (load)

runway takeoff

airplane routing

ticket (complain)

baggage (claim)

gates (unload)

runway landing

airplane routing

airplane routing

airline travel: a series of steps, involving many services



**layers:** each layer implements a service

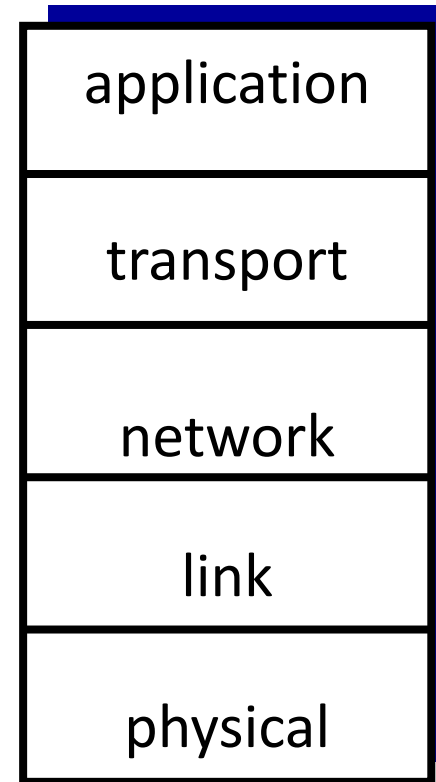
- via its own internal-layer actions
- relying on services provided by layer below

**Q:** describe in words  
the service provided  
in each layer above

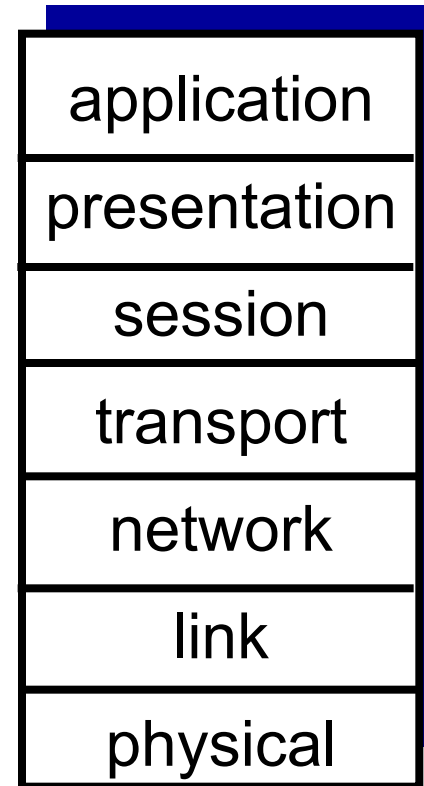
dealing with complex systems:

- explicit structure allows identification, relationship of complex system's pieces
  - layered *reference model* for discussion
- modularization eases maintenance, updating of system
  - change in layer's service *implementation*: transparent to rest of system
  - e.g., change in gate procedure doesn't affect rest of system
- layering considered harmful?
- layering in other complex systems?

- **application:** supporting network applications (access to network resources)
  - IMAP, SMTP, HTTP
- **transport:** process-process data transfer (segmentation & reassembly, sockets, connection, flow and error control)
  - TCP, UDP
- **network:** routing of datagrams from source to destination (addressing, routing)
  - IP, routing protocols
- **link:** data transfer between neighboring network elements (framing, addressing, flow & error control)
  - Ethernet, 802.11 (WiFi), PPP
- **physical:** bits “on the wire”



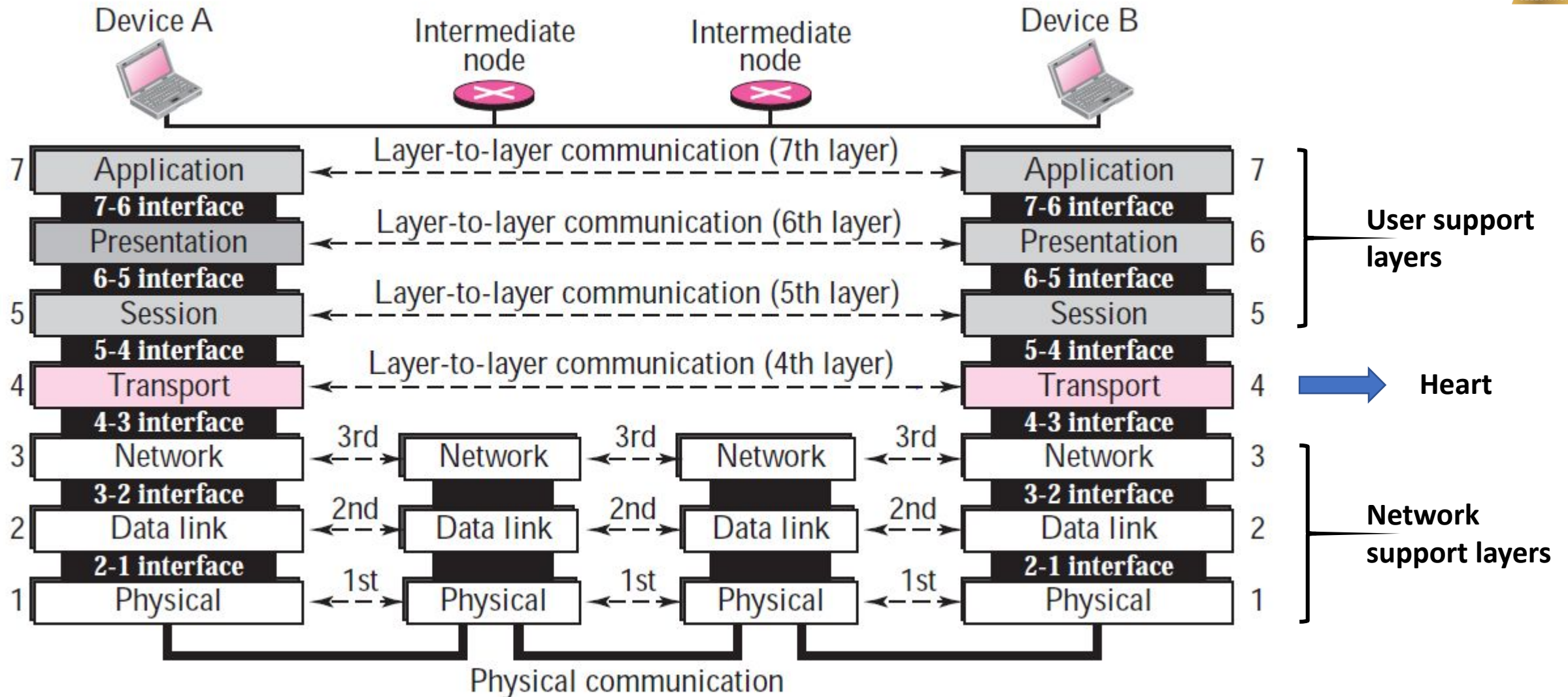
- *presentation*: allow applications to interpret meaning of data, (e.g., encryption, compression, machine-specific conventions)
- *session*: synchronization, checkpointing, recovery of data exchange
- Internet stack “missing” these layers!
  - these services, *if needed*, must be implemented in application
  - needed?



Open Systems Interconnection (OSI) model – introduced in late 1970s by ISO.

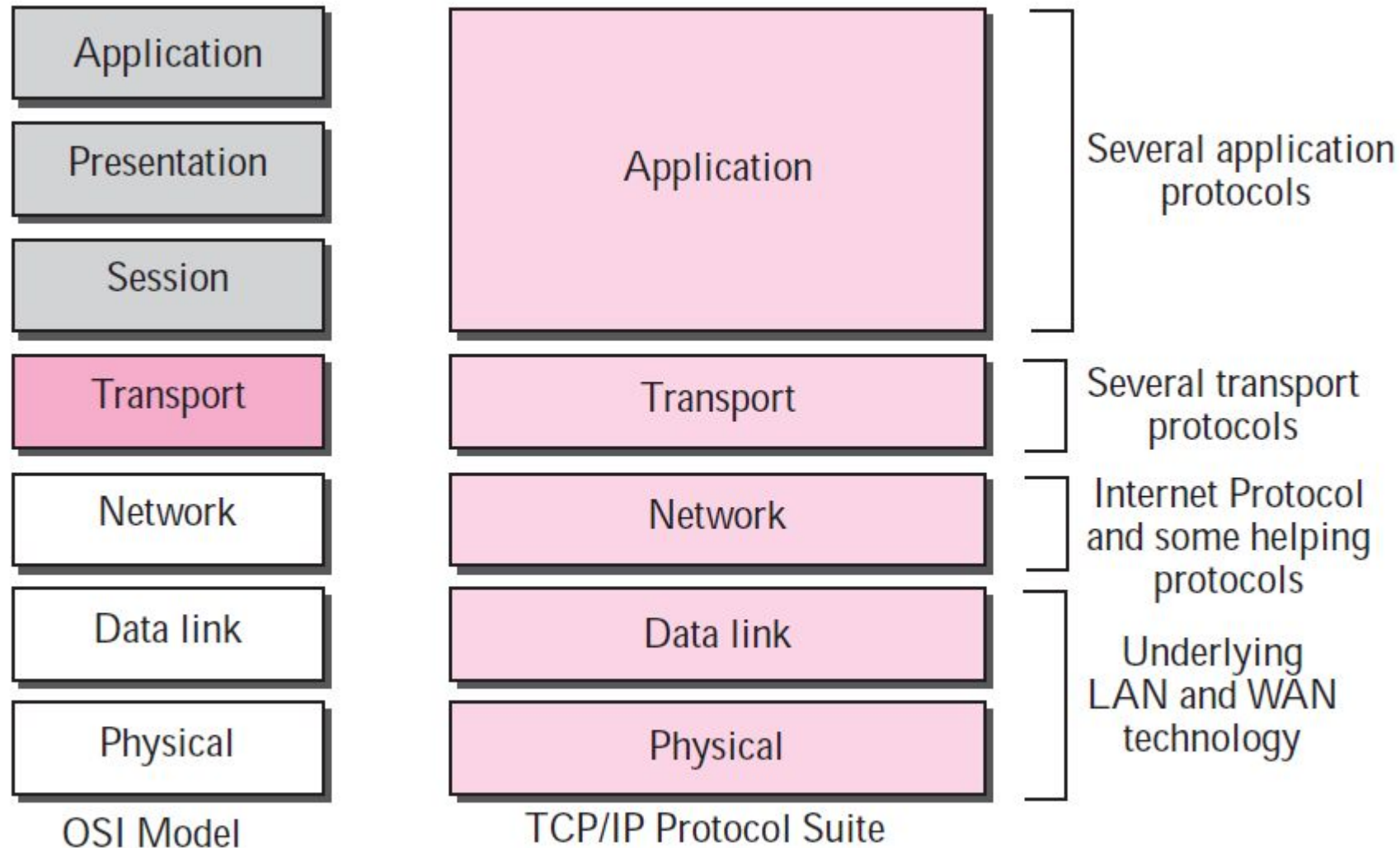
# COMPUTER NETWORKS

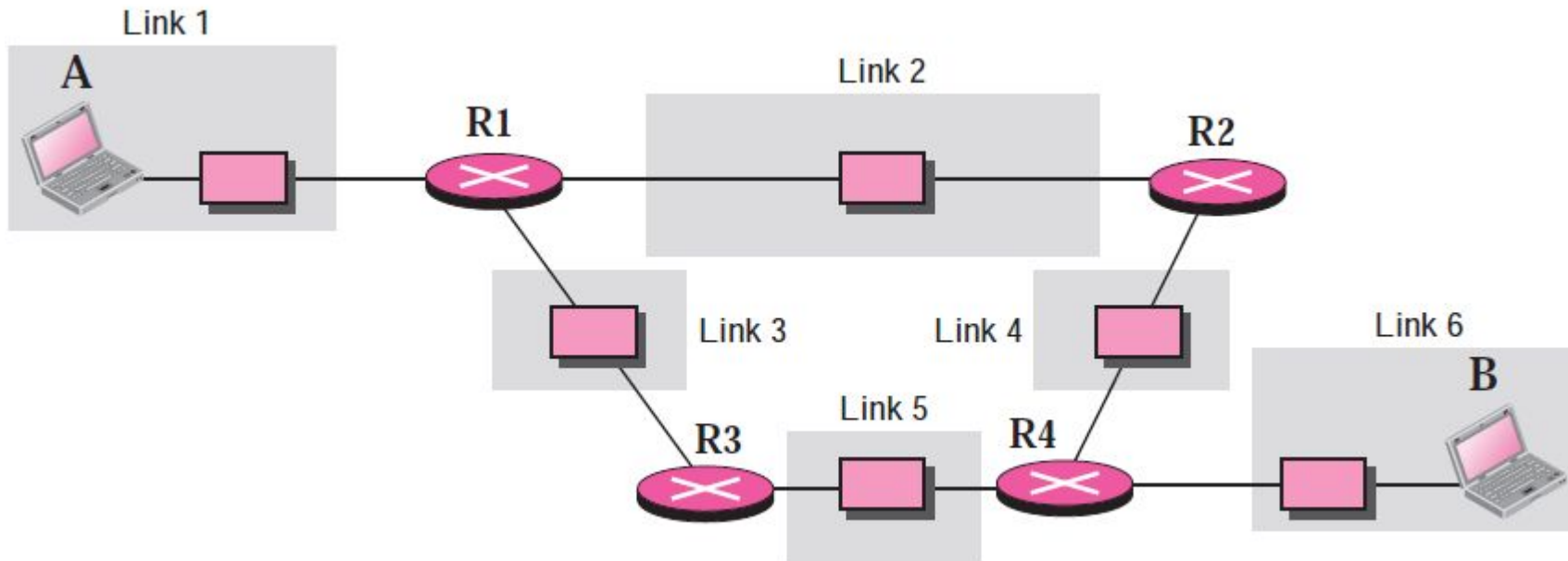
## OSI reference model (more)



# COMPUTER NETWORKS

## TCP/IP vs OSI reference model

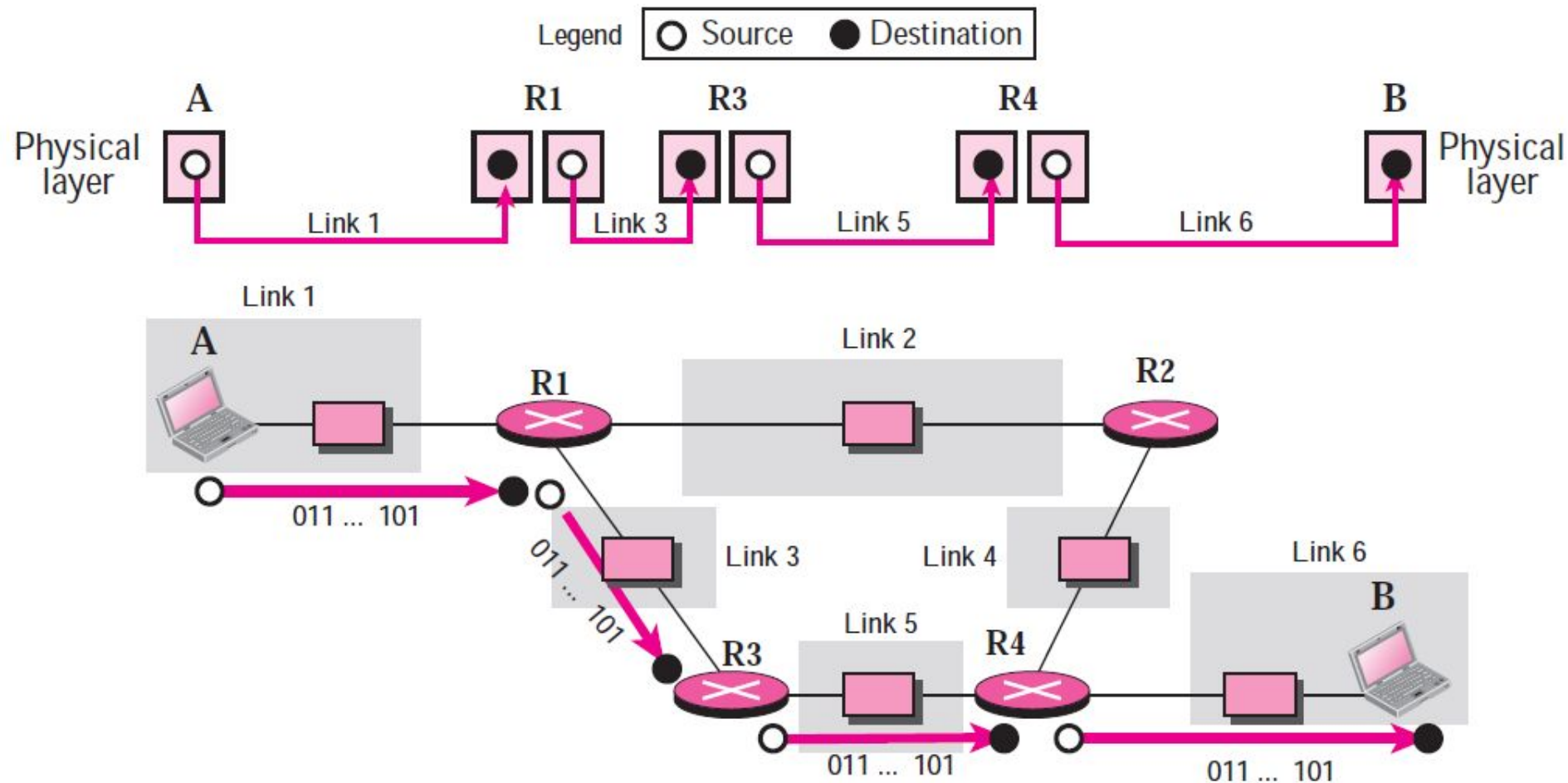




**A private internet**

# COMPUTER NETWORKS

## Layers in the TCP/IP Protocol Suite (more)

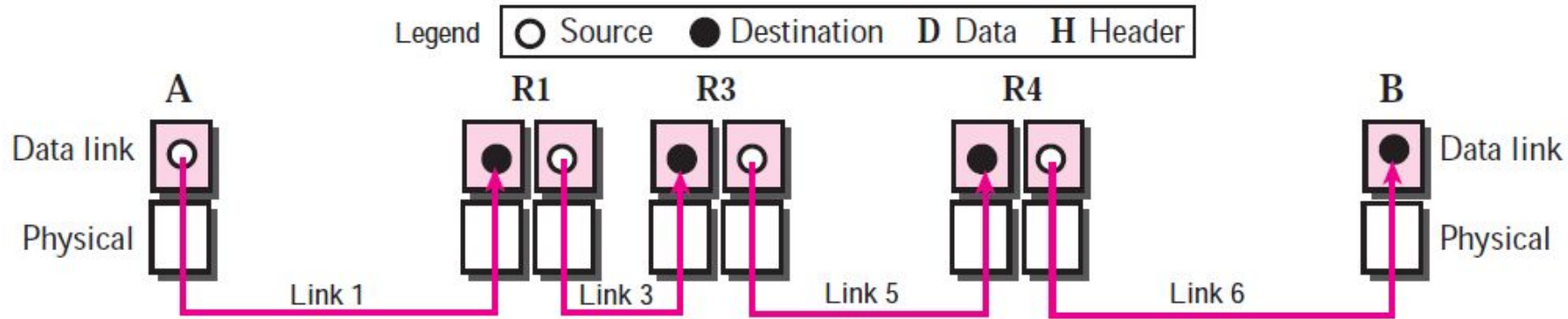


Communication at  
the physical layer

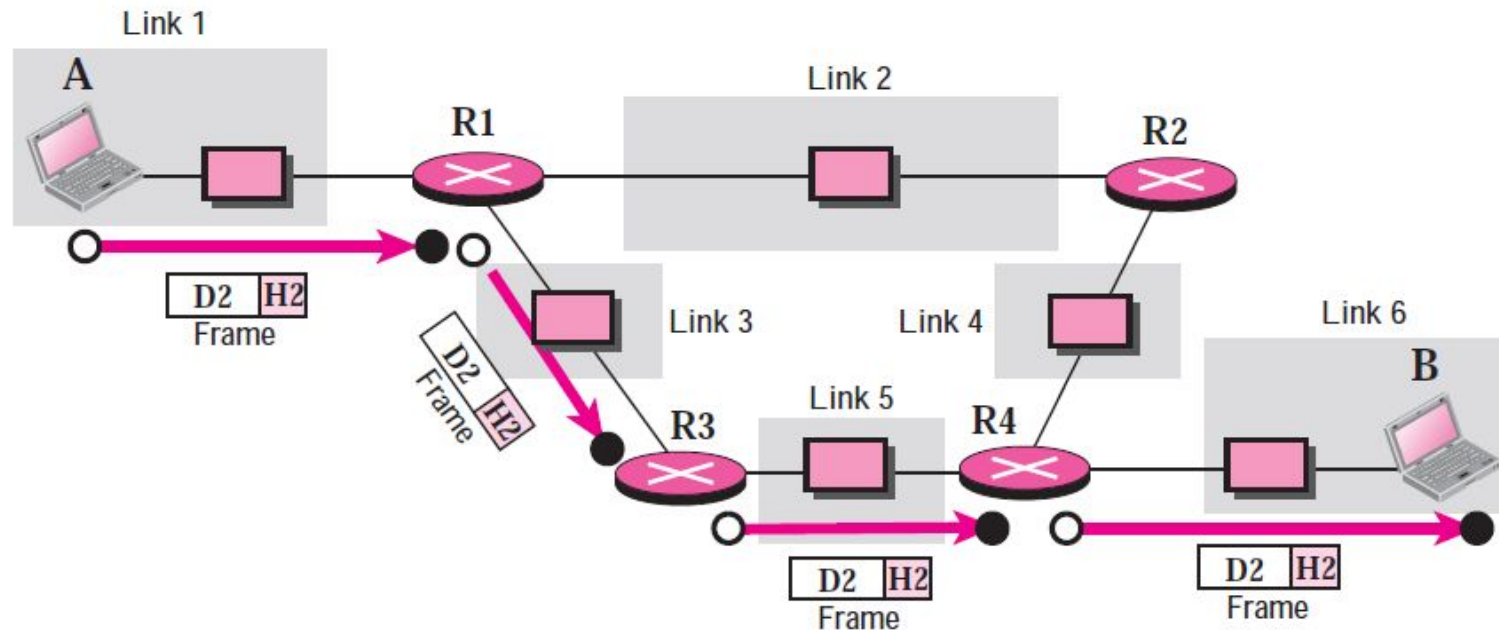
Unit of Communication  
– **bit**

# COMPUTER NETWORKS

## Layers in the TCP/IP Protocol Suite (more)



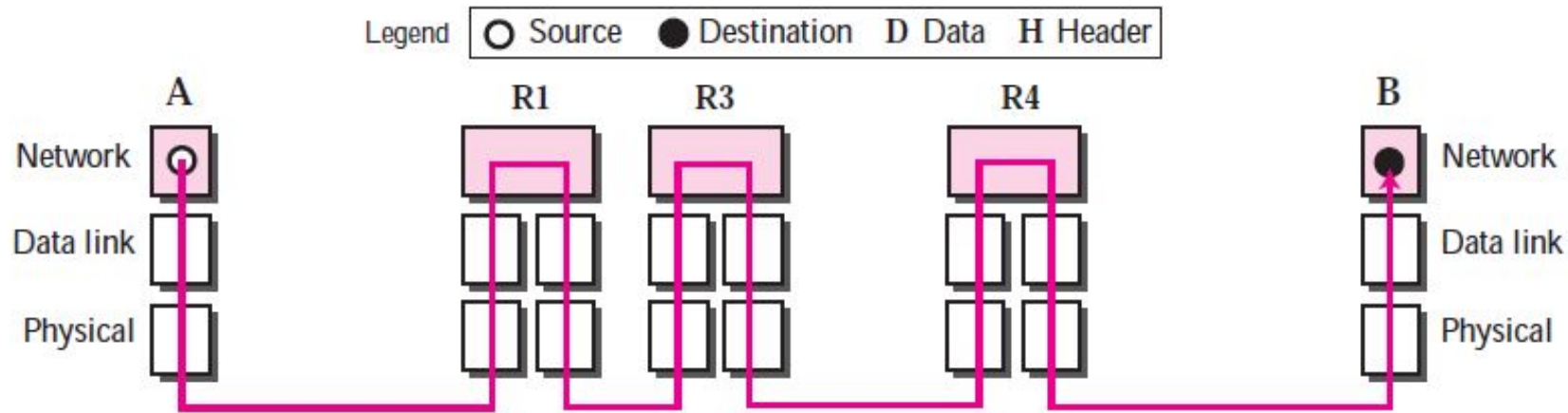
Communication at the data link layer



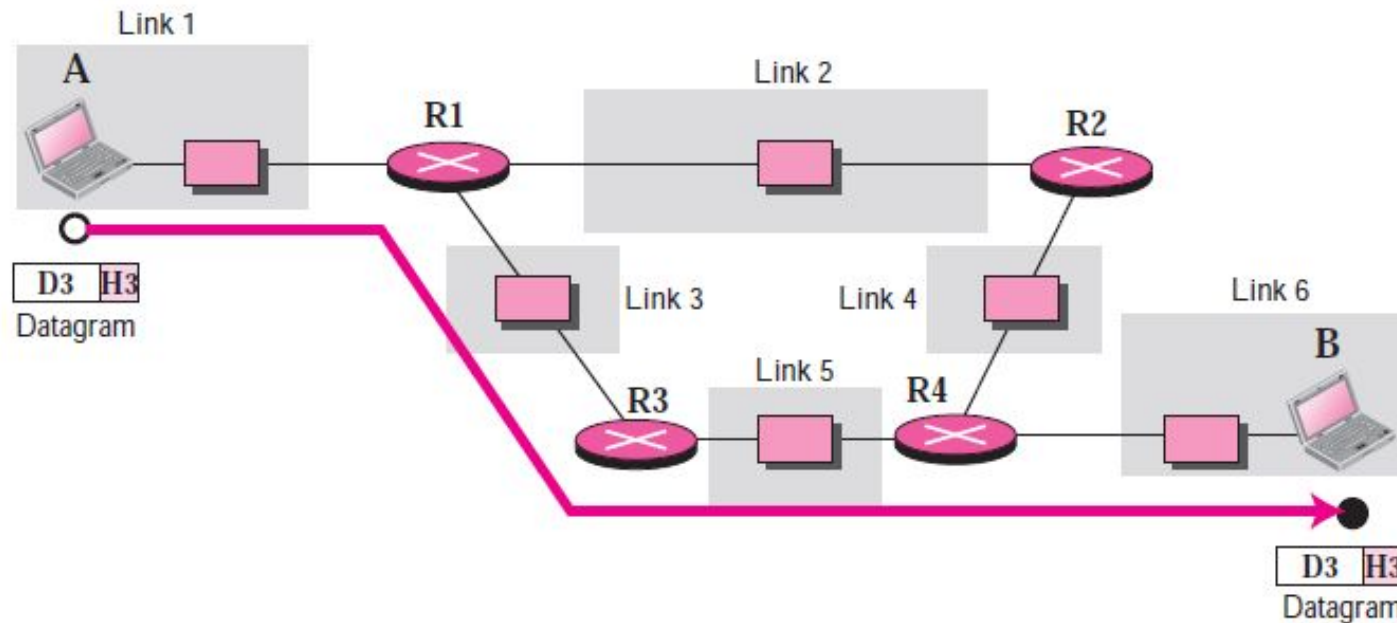
Unit of Communication –  
**frame**

# COMPUTER NETWORKS

## Layers in the TCP/IP Protocol Suite (more)



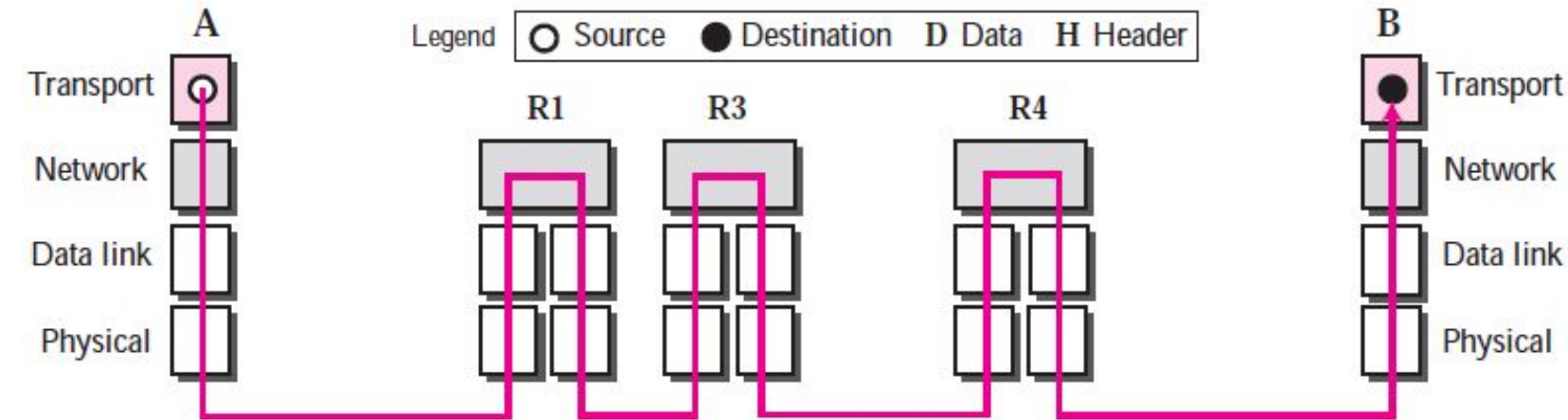
Communication at the  
network layer



Unit of Communication –  
**datagram**

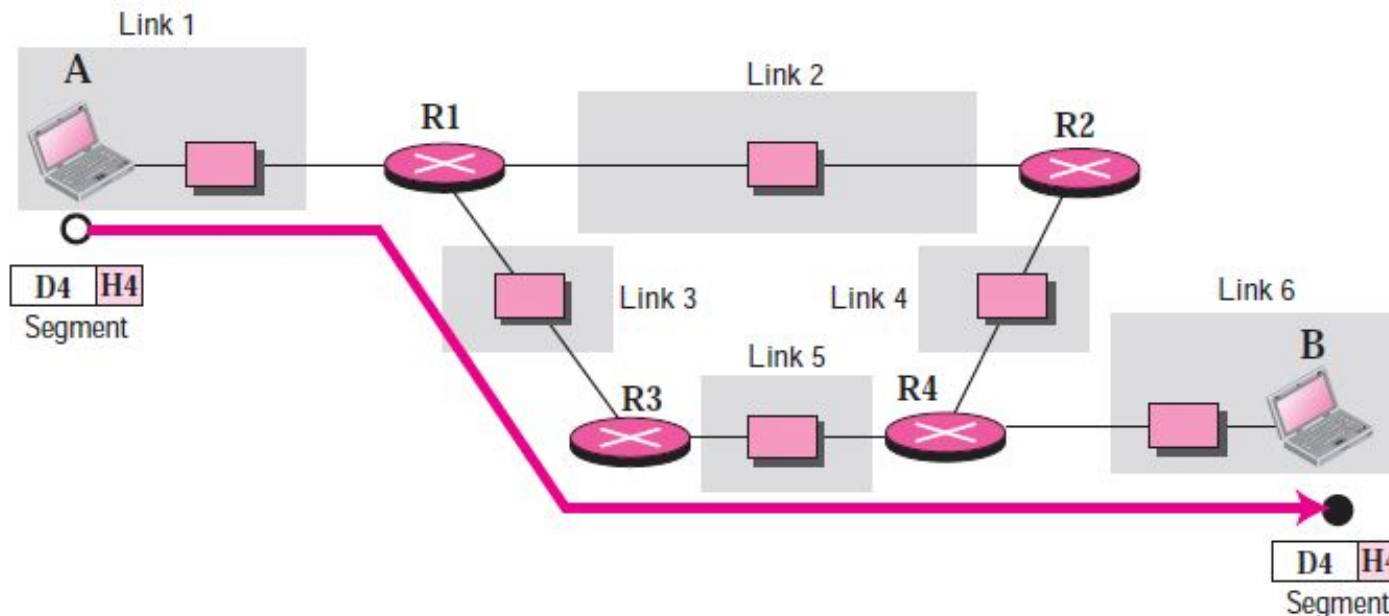
# COMPUTER NETWORKS

## Layers in the TCP/IP Protocol Suite (more)



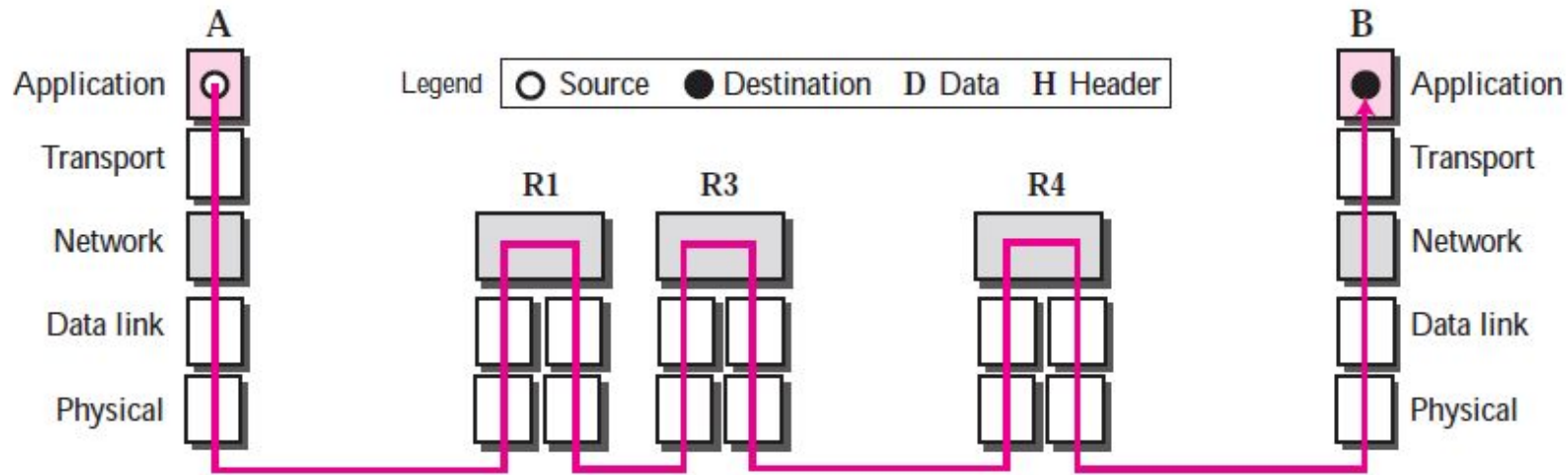
Communication at the transport layer

Unit of Communication  
– **segment/packet**



# COMPUTER NETWORKS

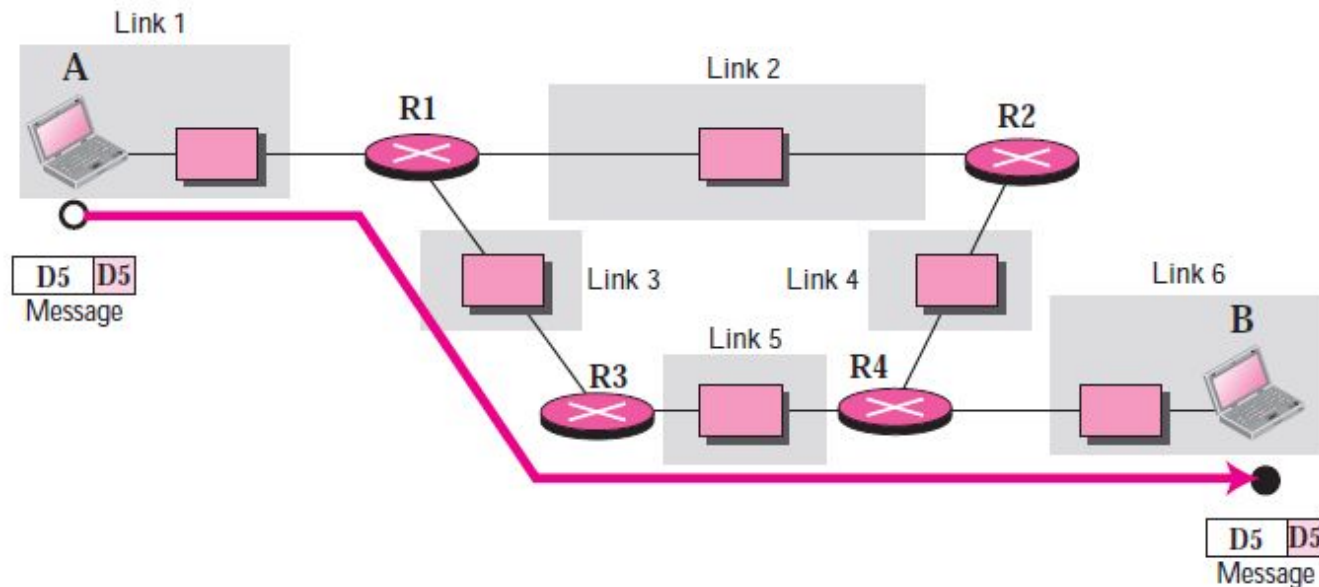
## Layers in the TCP/IP Protocol Suite (more)



Communication at the application layer

Unit of Communication

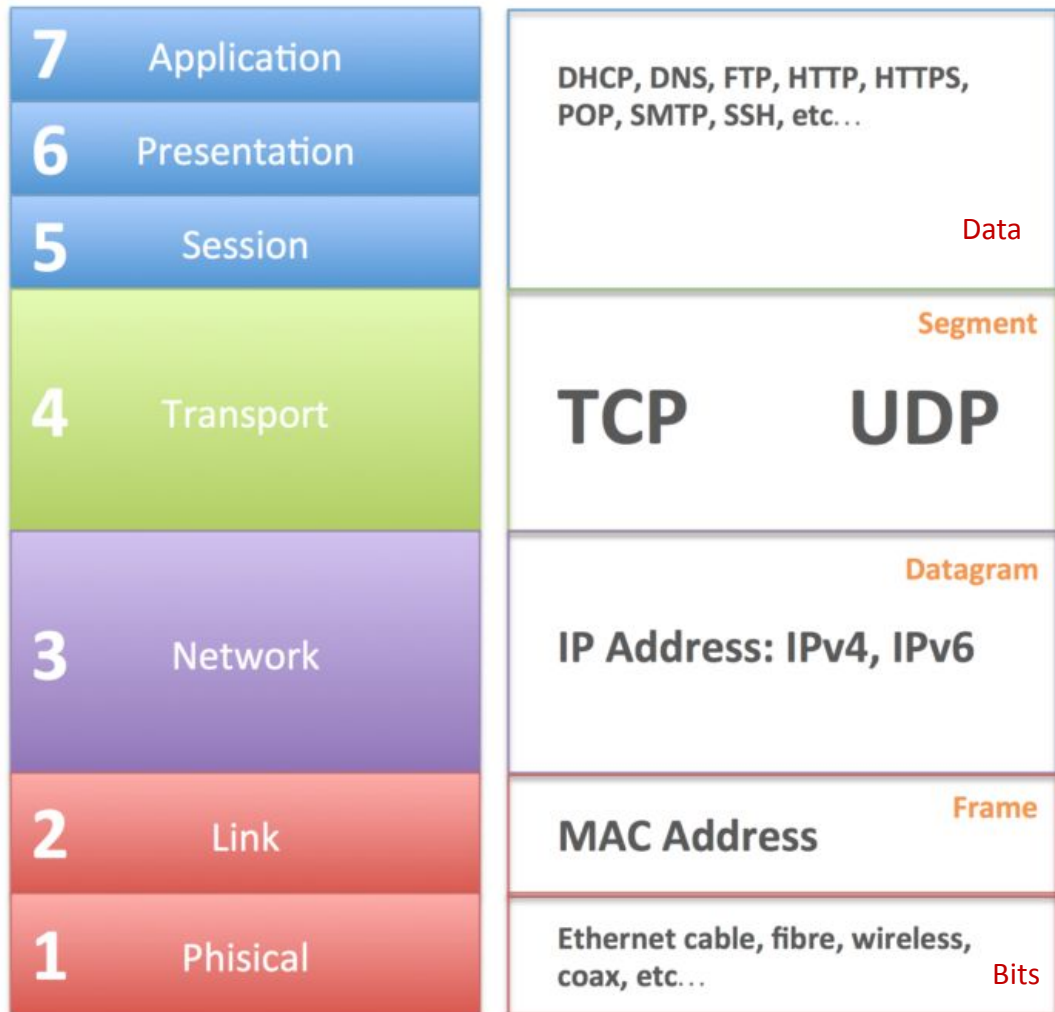
– **message**



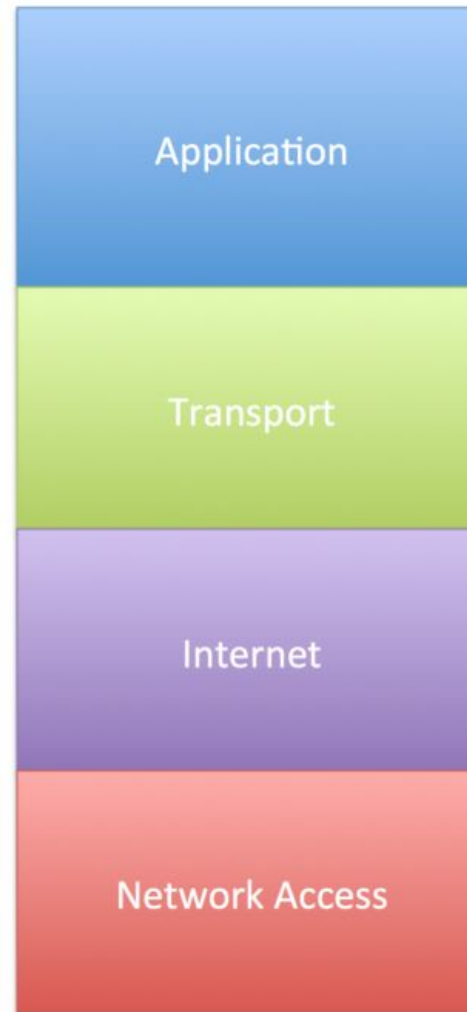
# COMPUTER NETWORKS

## OSI vs TCP/IP Protocol Suite

### The OSI Model

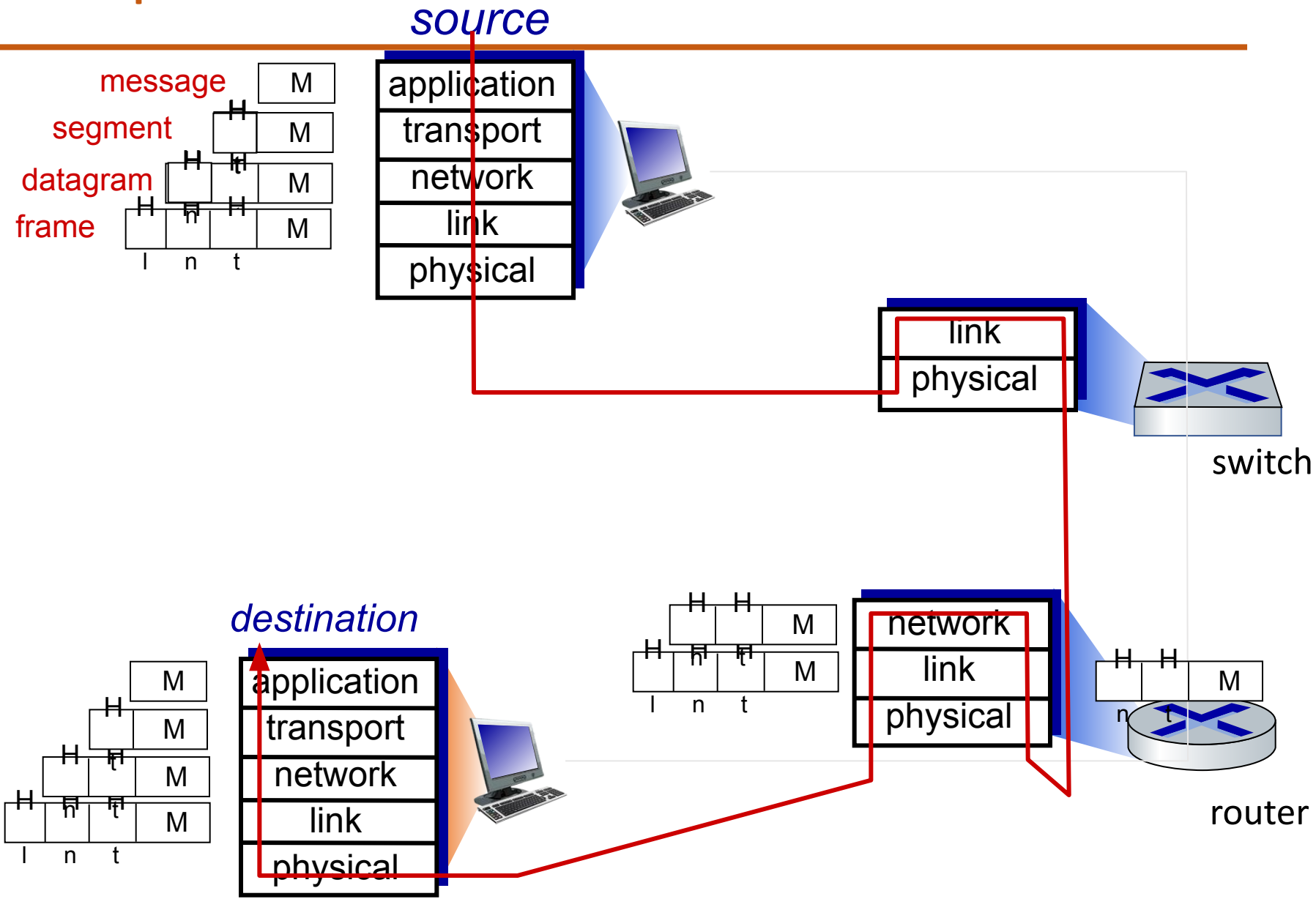


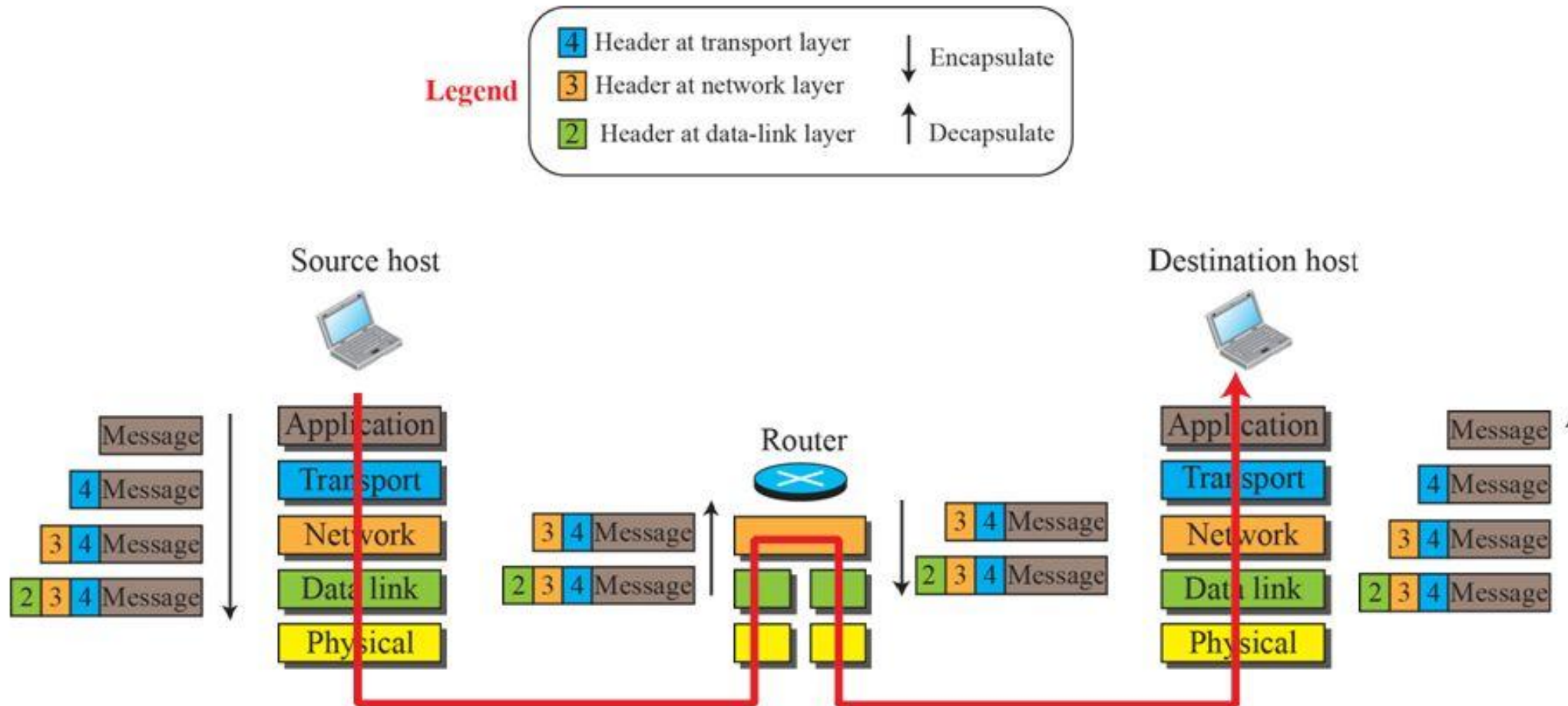
### The TCP/IP Model



# COMPUTER NETWORKS

## Encapsulation – Data Communication in Protocol Stack

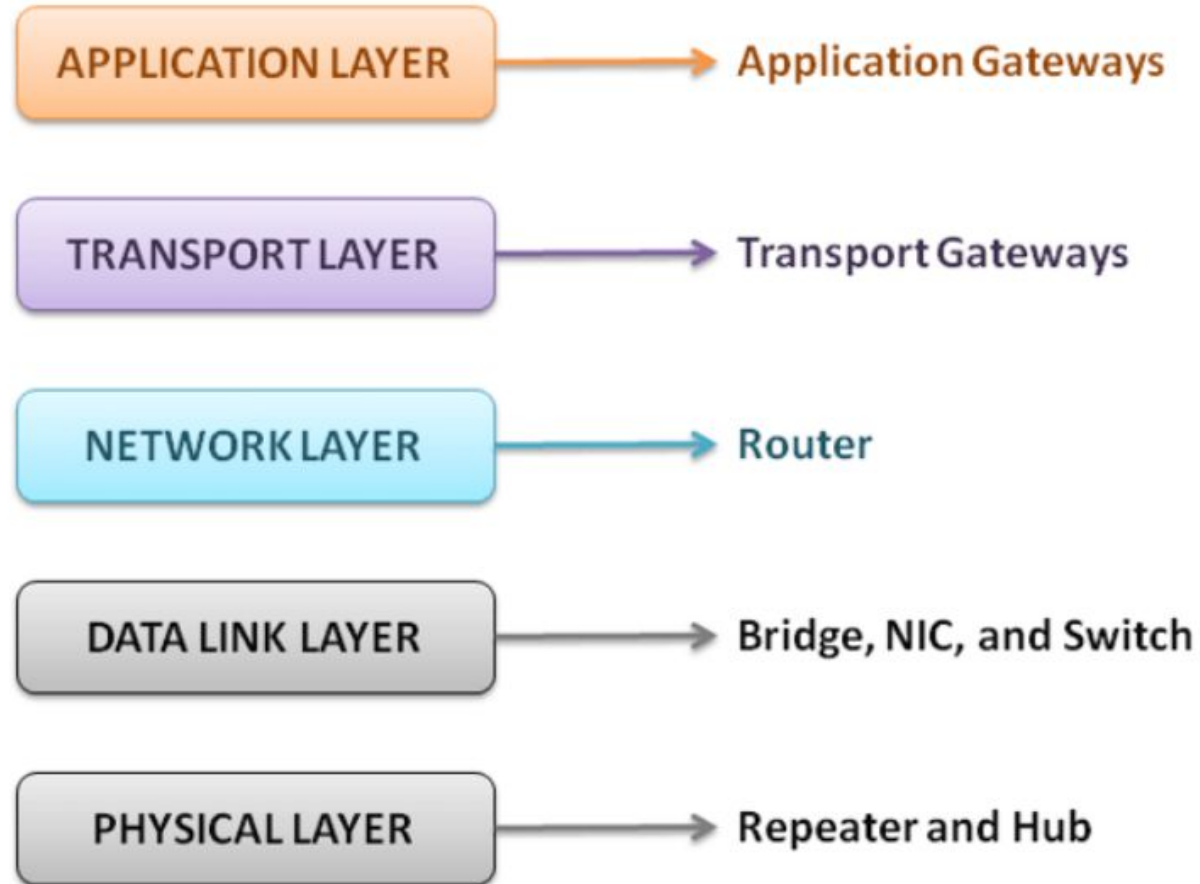




# COMPUTER NETWORKS

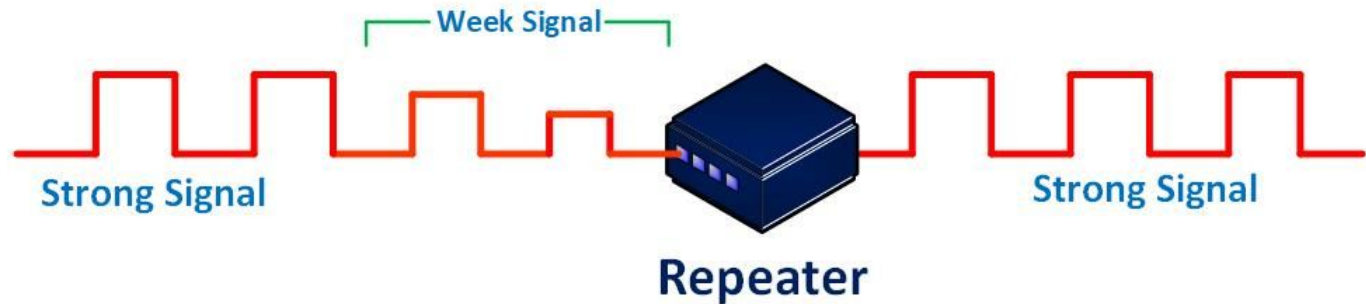
## Introduction to Network Devices

---



### Repeater

- Regenerate the signal over the same network before the signal becomes too weak or corrupted to extend the length to which the signal can be transmitted over the same network
- Works at the Physical Layer



### Working of a Repeater:-

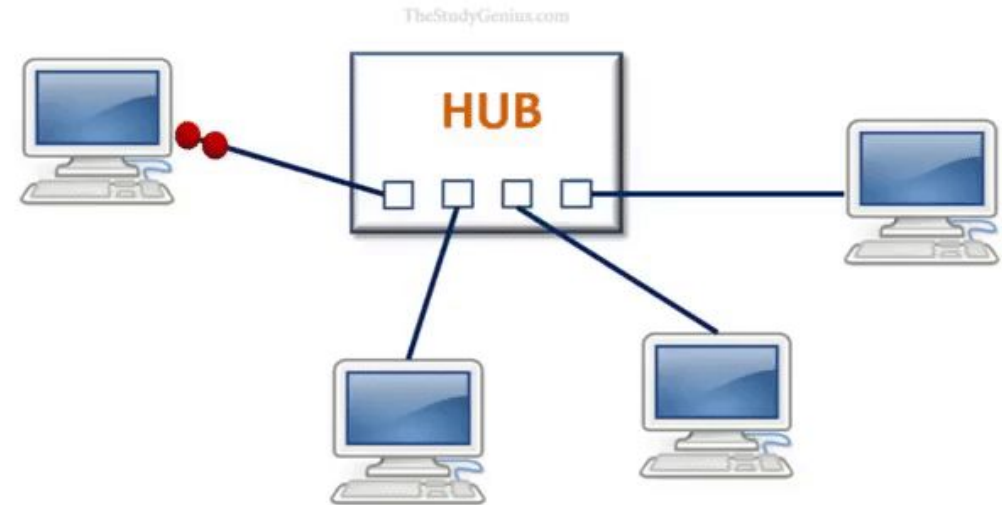
1. Signal Reception: Captures incoming data signal (electrical voltages or electromagnetic waves).
2. Signal Amplification: Strengthens weakened signals due to attenuation and interference.
3. Signal Regeneration: Reconstructs the signal to ensure data integrity and network compliance.
4. Signal Transmission: Sends the amplified and regenerated signal to its target, minimizing further deterioration.

### Hub

- Basic networking device that is used to connect multiple devices together on a local network.
- It shares bandwidth among all connected devices and broadcasts data sent by one device to all others, giving each device equal access to the network
- Operates at Physical Layer

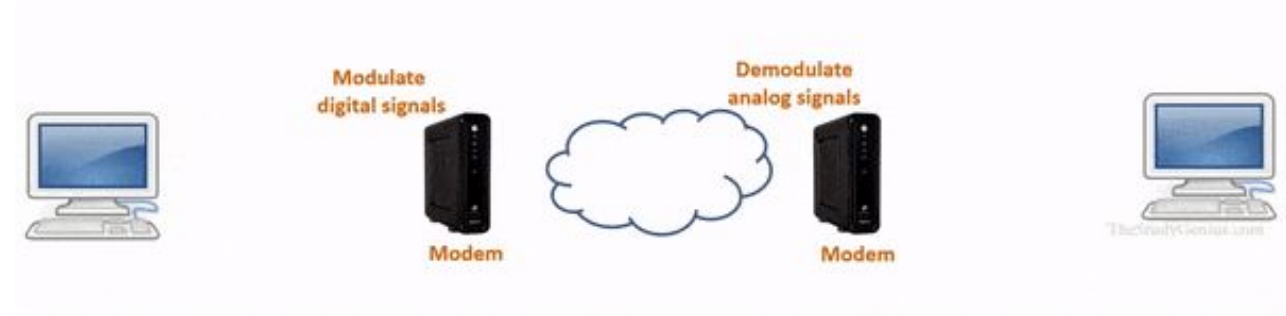
#### Features of a Hub:-

1. Always Broadcast: Hubs broadcast data to all connected devices, regardless of the intended recipient.
2. Half-duplex Communication: Only one device can transmit at once.
3. Cannot Store MAC Addresses: Hubs lack intelligence and cannot store MAC addresses or maintain tables.
4. No Data Filtering: Hubs transmit all data to all connected devices without filtering.



### Modem

- Device that connects a computer to the internet by converting digital signals to analog and vice versa for data transmission.
- Operates at the Physical Layer

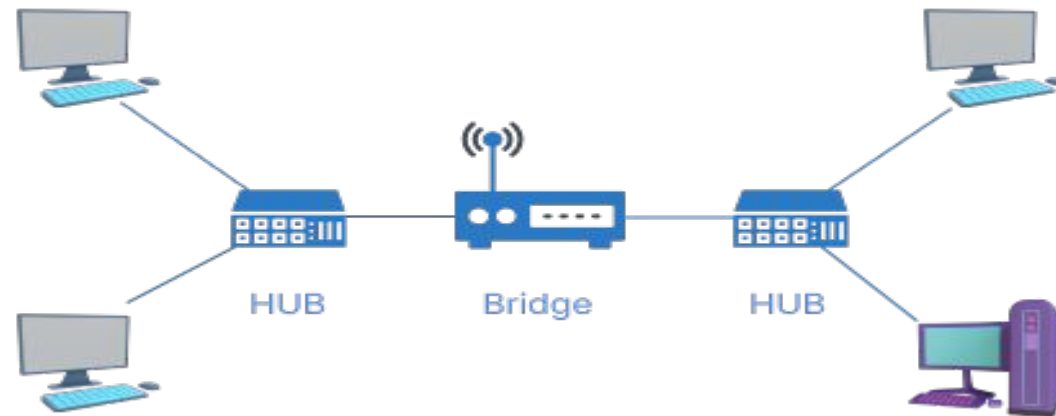


### Modem Data Transmission Process:-

1. Modulation: The modem converts digital data from the computer into an analog signal for transmission.
2. Transmission: The modulated signal is sent over the communication line to the receiving modem.
3. Demodulation: The receiving modem converts the analog signal back into digital data.
4. Decoding: The digital data is then sent to the computer for use.

### Bridge

- A 2-port device that joins two or more network segments and routes data between them according to MAC addresses
- Operates at the Data Link Layer



### Functions :-

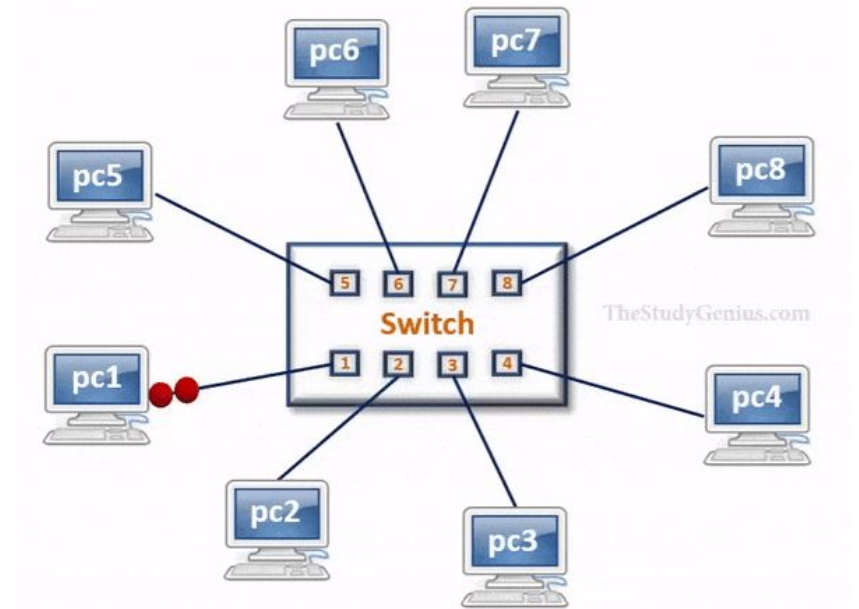
1. Segmentation: Divides a large network into smaller segments to reduce congestion and improve performance.
2. Filtering: Uses MAC addresses to selectively forward packets, enhancing security and bandwidth usage.
3. Learning: Tracks MAC addresses and port mappings to optimize packet forwarding.
4. Forwarding: Directs packets between segments, reducing unnecessary traffic and ensuring proper delivery.

### Switch

- An intelligent network device, functioning as a multiport bridge, uses MAC addresses to forward data packets to the appropriate destination ports.
- Operates at Data Link Layer

#### Features of a Switch:-

1. Employs packet switching to receive and route data from source to destination.
2. Can perform some error checking before forwarding data to the destined port.
3. Can operate in full-duplex mode, enabling simultaneous data transmission & reception in network
4. Store MAC addresses of connected devices, routing communication only to the intended destination, reducing collisions and eliminating broadcast domains.



### NIC - Network Interface Card

- It is a physical card or chip, which contains MAC addresses, helps to identify the device on the network.
- Used at the Data Link Layer

#### Functions of a NIC:-

1. Connectivity: Links the device to the network (wired or wireless).
2. Data Transmission: Sends and receives data between devices.
3. Data Conversion: Converts data for transmission and reception.
4. Error Handling: Detects and corrects transmission errors.
5. Traffic Management: Prioritizes or limits data to avoid congestion.
6. Security: Provides encryption and authentication for protection.

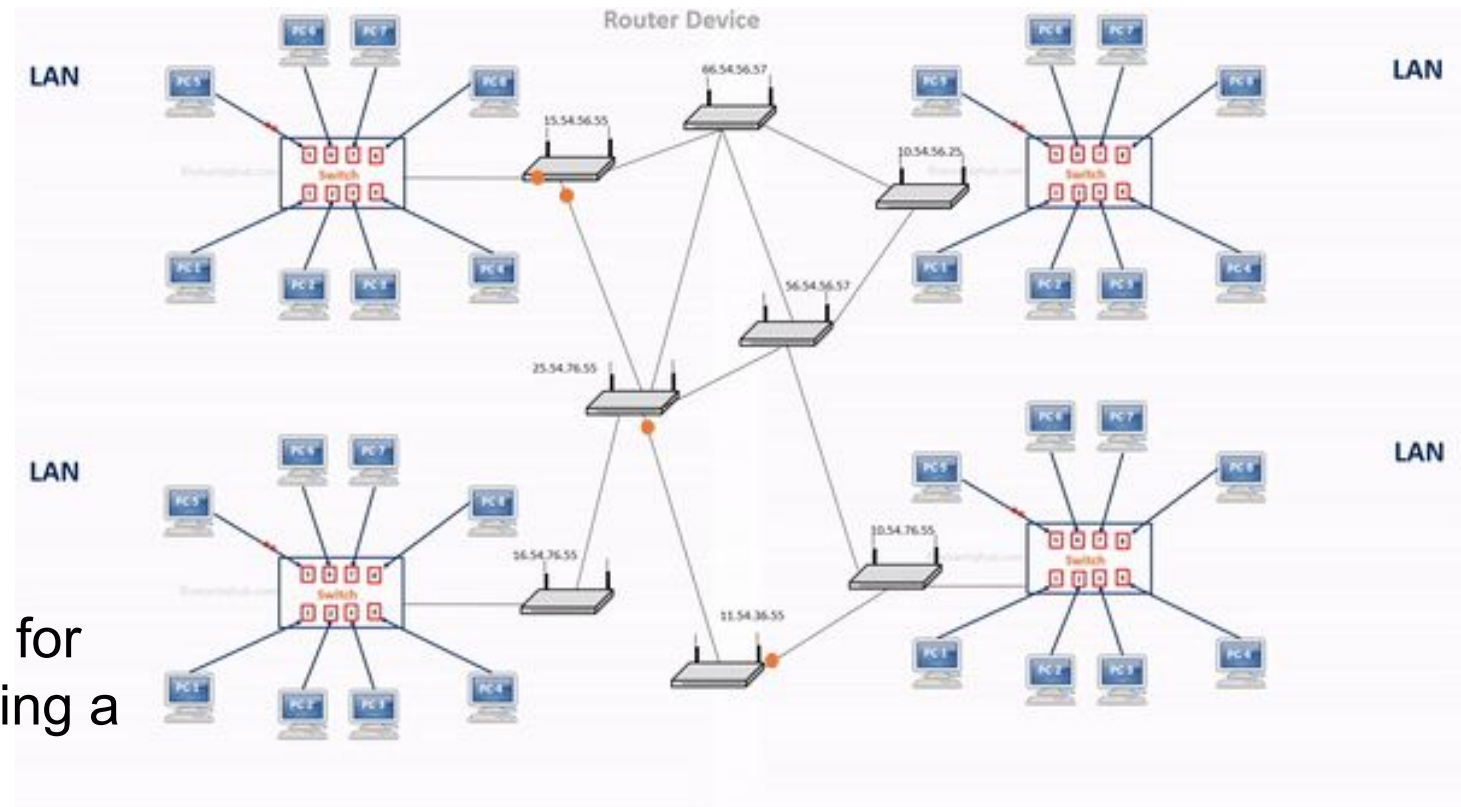


### Router

- Device that routes/forwards data packets based on their IP addresses and have a dynamically updating routing table based on which they make decisions on routing the incoming packets.
- Operates at the Network Layer

### Functions of a Router:-

1. Forwarding: Receives packets, checks headers, performs basic functions, and forwards packets to the appropriate output port.
1. Routing: Determines the best path for a packet to reach its destination using a routing table built with algorithms.



## Router

### Functions of a Router:-

3. Network Address Translation (NAT): Translates between private and public IP addresses, enabling devices on a private network to access the internet.

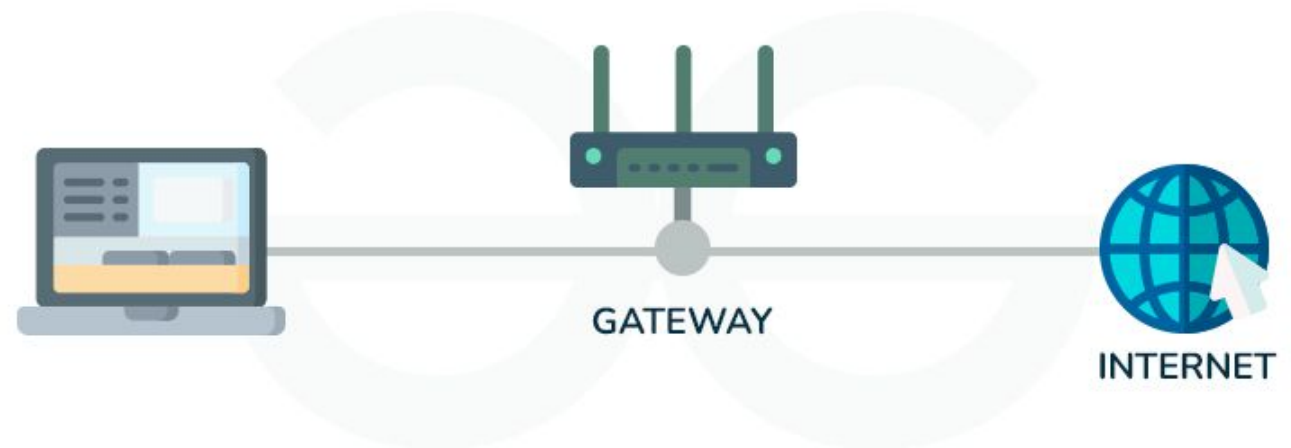


### Gateway

- Network connectivity device that connects two different configuration networks
- The gateway acts as a portal between two applications via protocol communications, allowing them to share data on the same or other systems. Hence, a gateway is also a protocol converter that can operate at any OSI model layer.

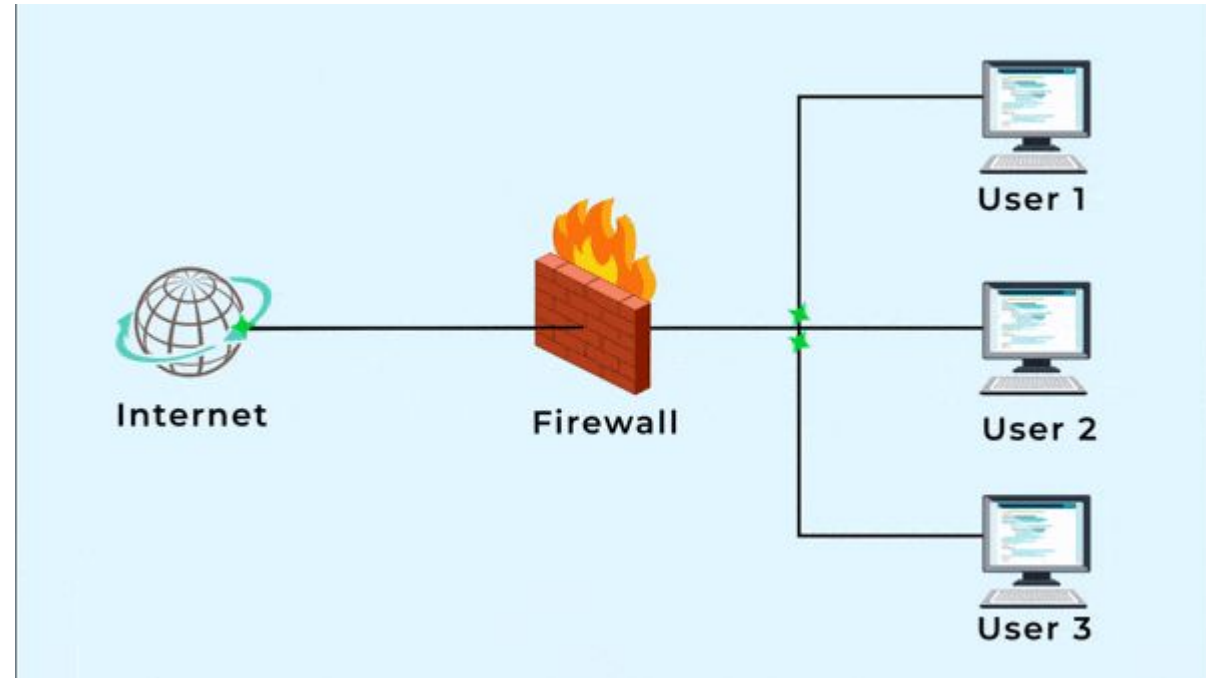
#### Types of Gateway:-

1. Unidirectional Gateway: Data flows in one direction, with changes replicated from source to destination only.
2. Bidirectional Gateway: Data flows in both directions, used for synchronization.



### Firewall

- Network security device that monitors incoming and outgoing network traffic and decides whether to allow or block specific traffic based on a defined set of security rules
- It can be used in the transport layer or the application layer



# COMPUTER NETWORKS

## Introduction to Network Devices

| Feature          | Hub                                   | Switch                                                                |
|------------------|---------------------------------------|-----------------------------------------------------------------------|
| Function         | Broadcasts data to all devices        | Forwards data to the specific destination device based on MAC address |
| Layer            | Operates at Physical Layer            | Operates at Data Link Layer                                           |
| Data Handling    | Sends data to all connected devices   | Sends data only to the intended device                                |
| Collision Domain | Single collision domain               | Multiple collision domains (one per port)                             |
| Bandwidth        | Shared bandwidth among all devices    | Dedicated bandwidth for each device                                   |
| Intelligence     | Basic, no intelligence                | More intelligent, stores MAC addresses and makes forwarding decisions |
| Data Filtering   | No filtering, all data is broadcasted | Filters data based on MAC addresses                                   |
| Usage            | Suitable for small networks           | Suitable for larger, more complex networks                            |
| Performance      | Lower performance due to collisions   | Higher performance with reduced collisions                            |

| Feature           | Switch                                                                   | Router                                                                         |
|-------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Definition        | A network device that connects multiple devices within the same network. | A network device that connects multiple networks and routes data between them. |
| Data Transmission | Uses MAC addresses to forward data within the network.                   | Uses IP addresses to route data between networks.                              |
| Function          | Operates at Data Link Layer                                              | Operates at Network Layer                                                      |
| Broadcast Domain  | Typically operates within a single broadcast domain.                     | Breaks up broadcast domains and creates separate ones.                         |
| Usage             | Used in LANs to expand network capacity.                                 | Used to connect LANs to the internet or other LANs.                            |
| Speed             | High-speed data transfer within the network.                             | Typically slower due to complex routing processes.                             |

# Computer Networks and the Internet

---

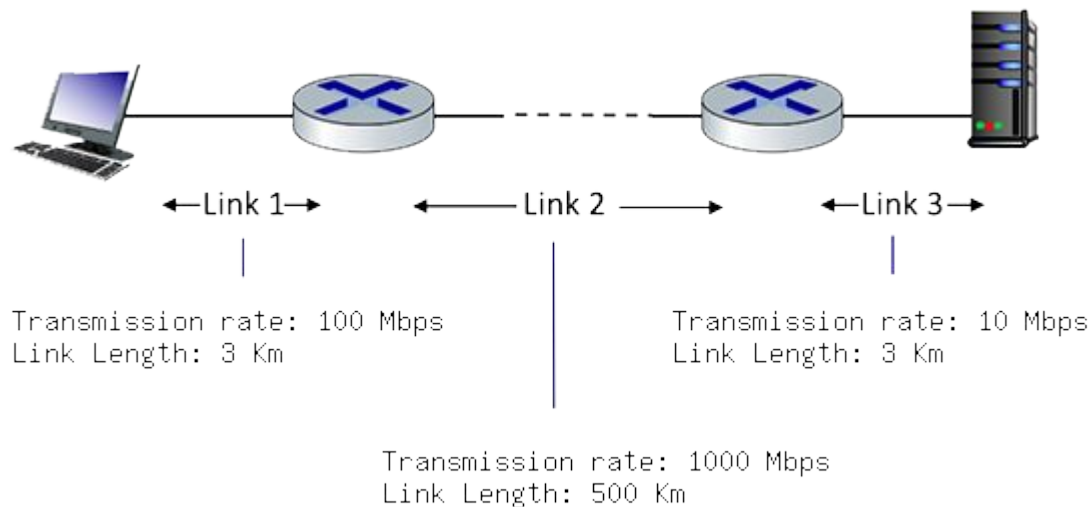
## Interactive Exercises

### Team Networks

Department of Computer Science and Engineering

### Computing end-end delay (transmission and propagation delay)

Consider the figure below, with three links, each with the specified transmission rate and link length.



**Find the end-to-end delay (including the transmission delays and propagation delays on each of the three links, but ignoring queueing delays and processing delays) from when the left host begins transmitting the first bit of a packet to the time when the last bit of that packet is received at the server at the right.**

The speed of light propagation delay on each link is  $3 \times 10^8$  m/sec.

Note that the transmission rates are in Mbps and the link distances are in Km.

Assume a packet length of 8000 bits. Give your answer in milliseconds.

Link 1 transmission delay =  $L/R = 8000 \text{ bits} / 100 \text{ Mbps} = 80 \text{ microsec}$

Link 1 propagation delay =  $d/s = 3 \text{ Km} / 3 \times 10^8 \text{ m/sec} = 10 \text{ microsec}$

Link 1 total delay =  $80 + 10 = 90 \text{ microsec}$

Link 2 transmission delay =  $L/R = 8000 \text{ bits} / 1000 \text{ Mbps} = 8 \text{ microsec}$

Link 2 propagation delay =  $d/s = 500 \text{ Km} / 3 \times 10^8 \text{ m/sec} = 1670 \text{ microsec}$

Link 2 total delay =  $8 + 1670 = 1678 \text{ microsec}$

Link 3 transmission delay =  $L/R = 8000 \text{ bits} / 10 \text{ Mbps} = 800 \text{ microsec}$

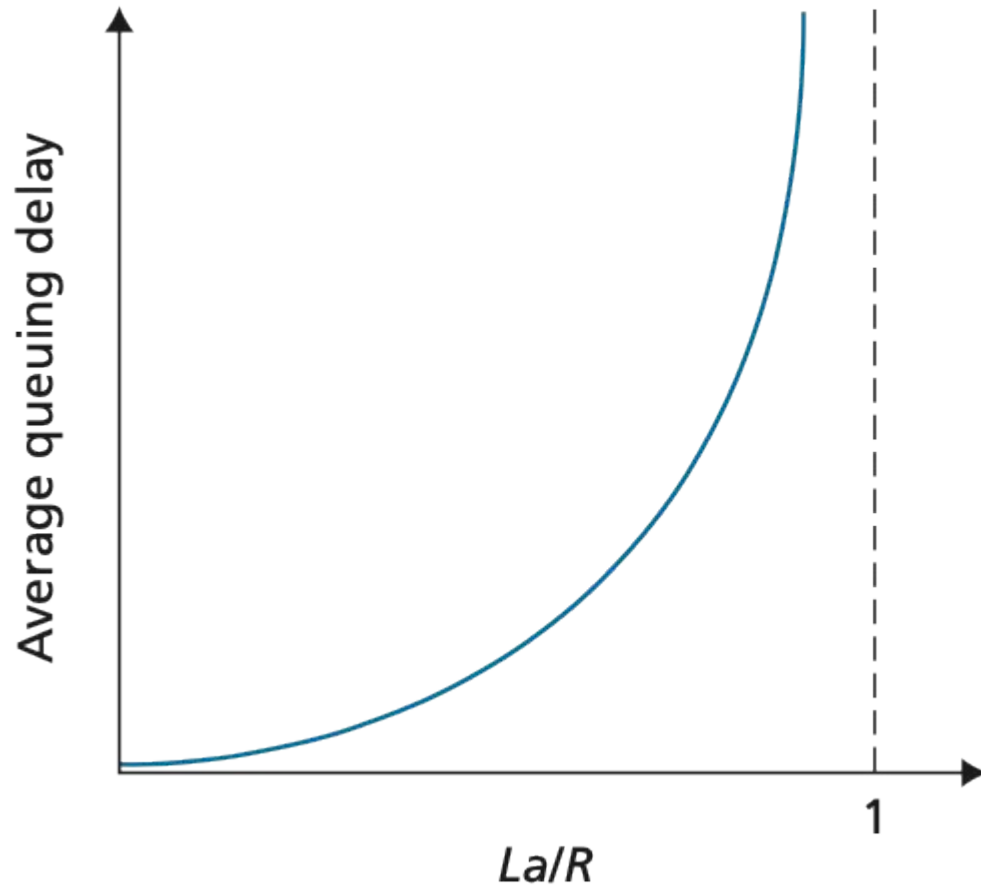
Link 3 propagation delay =  $d/s = 3 \text{ Km} / 3 \times 10^8 \text{ m/sec} = 10 \text{ microsec}$

Link 3 total delay =  $800 + 10 = 810 \text{ microsec}$

Total end-to-end delay is the sum of these six delays:  $90 + 1678 + 810 = 2.578 \text{ ms}$

### Computing Queuing Delay

Consider the queuing delay in a router buffer.



Assume:

a constant transmission rate of  $R = 1800000$  bps,  
a constant packet-length  $L = 7300$  bits, and  
 $a$  is the average rate of packets/second.

Traffic intensity  $I = La/R$ , and  
Queuing delay is calculated as:  
 **$IL/(R(1 - I))$**  for  $I < 1$ .

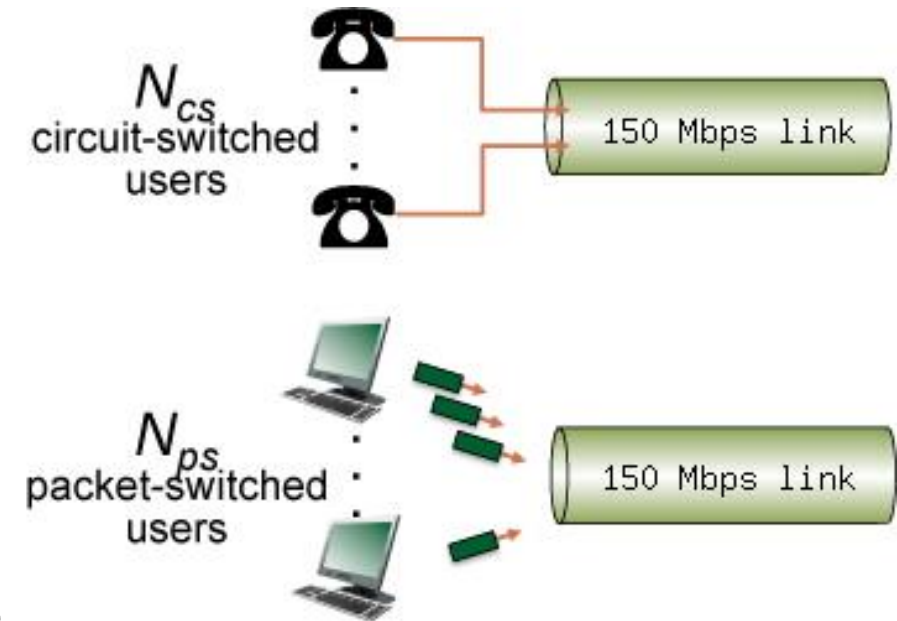
1. In practice, does the queuing delay tend to vary a lot? Answer with Yes or No
2. Assuming that  $a = 30$ , what is the queuing delay?
3. Assuming that  $a = 76$ , what is the queuing delay?
4. Assuming the router's buffer is infinite, the queuing delay is 0.8647 ms, and 1762 packets arrive. How many packets will be in the buffer 1 second later?
5. If the buffer has a maximum size of 956 packets, how many of the packets would be dropped upon arrival from the previous question?

1. Yes, in practice, queuing delay can vary significantly. We use the above formulas as a way to give a rough estimate, but in a real-life scenario it is much more complicated.
2. Queuing Delay =  $\frac{IL}{R(1 - I)} * 1000 = \frac{0.1217 * 7300}{(1800000 * (1 - 0.1217))} * 1000 = 0.561 \text{ ms.}$
3. Queuing Delay =  $\frac{IL}{R(1 - I)} * 1000 = \frac{0.3082 * 7300}{(1800000 * (1 - 0.3082))} * 1000 = 1.806 \text{ ms.}$
4. Packets left in buffer =  $a - \text{floor}(1000/\text{delay}) = 1762 - \text{floor}(1000/0.8647) = 606 \text{ packets.}$
5. Packets dropped = packets - buffer size =  $1762 - 956 = 806 \text{ dropped packets.}$

## Quantitative Comparison of Packet Switching and Circuit Switching

Consider the two scenarios below:

- A circuit-switching scenario in which  $N_{cs}$  users, each requiring a bandwidth of 10 Mbps, must share a link of capacity 150 Mbps.
- A packet-switching scenario with  $N_{ps}$  users sharing a 150 Mbps link, where each user again requires 10 Mbps when transmitting, but only needs to transmit 30 percent of the time.



**Round your answer to two decimals after leading zeros**

### Answer the following questions:

1. When circuit switching is used, what is the maximum number of circuit-switched users that can be supported? Explain your answer.
2. For rest of the questions, suppose packet switching is used. Suppose there are 29 packet-switching users (i.e.,  $N_{ps} = 29$ ). Can this many users be supported under circuit-switching? Explain.

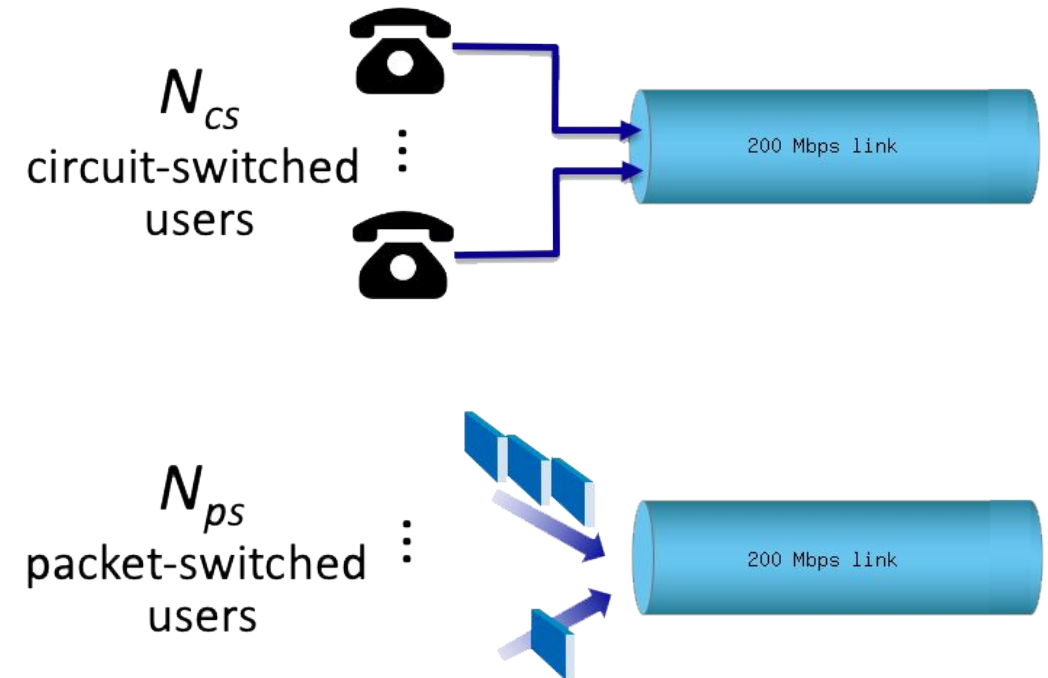
### Answers:

1. 15
2. No.

### Quantitative Comparison of Packet Switching and Circuit Switching

Consider the two scenarios below:

- A circuit-switching scenario in which  $N_{cs}$  users, each requiring a bandwidth of 25 Mbps, must share a link of capacity 200 Mbps.
- A packet-switching scenario with  $N_{ps}$  users sharing a 200 Mbps link, where each user again requires 25 Mbps when transmitting, but only needs to transmit 20 percent of the time.



### Answer the following questions:

1. When circuit switching is used, what is the maximum number of users that can be supported?
2. Suppose packet switching is used. If there are 15 packet-switching users, can this many users be supported under circuit-switching? Yes or No.

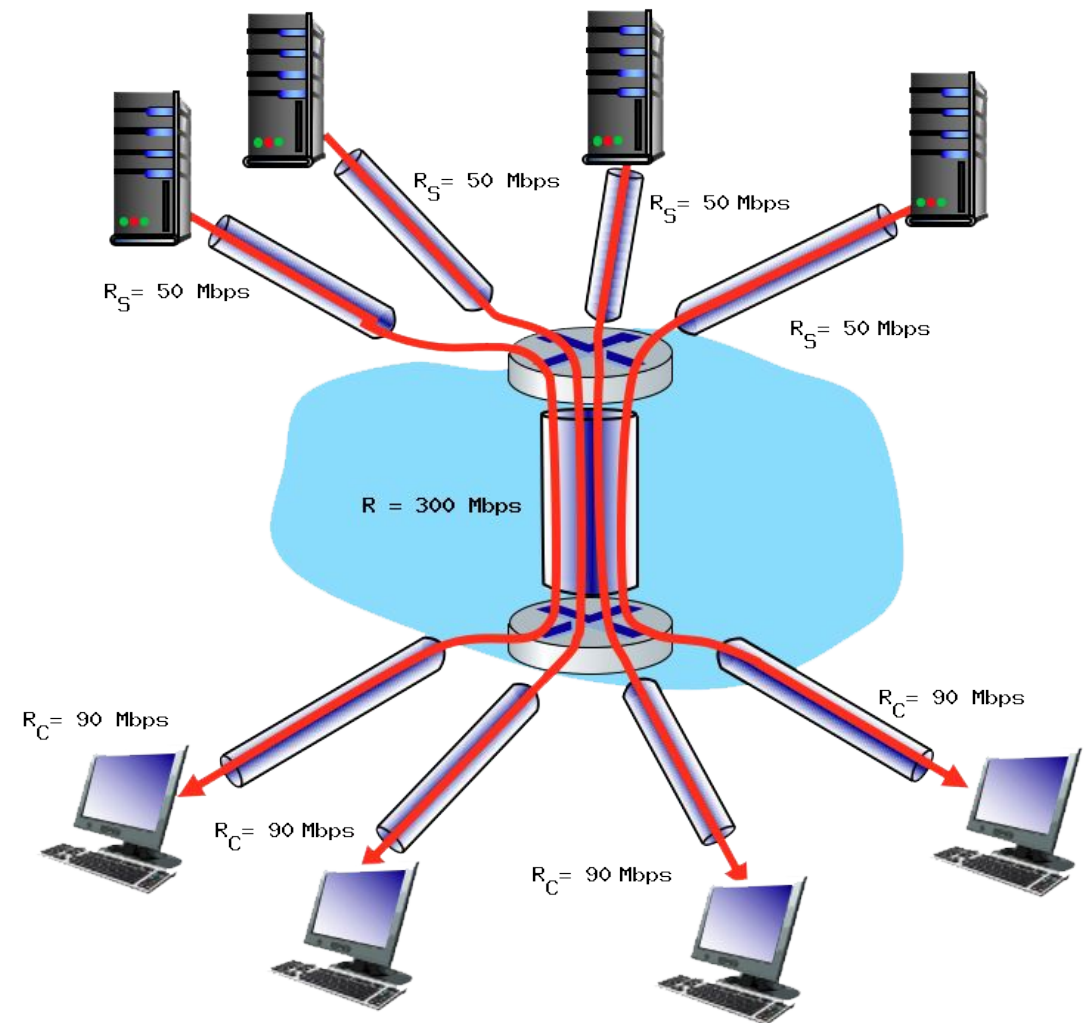
### END TO END THROUGHPUT AND BOTTLENECK LINKS

Consider the scenario shown below, with four different servers connected to four different clients over four three-hop paths.

The four pairs share a common middle hop with a transmission capacity of  $R = 300$  Mbps.

The four links from the servers to the shared link have a transmission capacity of  $R_S = 50$  Mbps.

Each of the four links from the shared middle link to a client has a transmission capacity of  $R_C = 90$  Mbps.



### Answer the following questions:

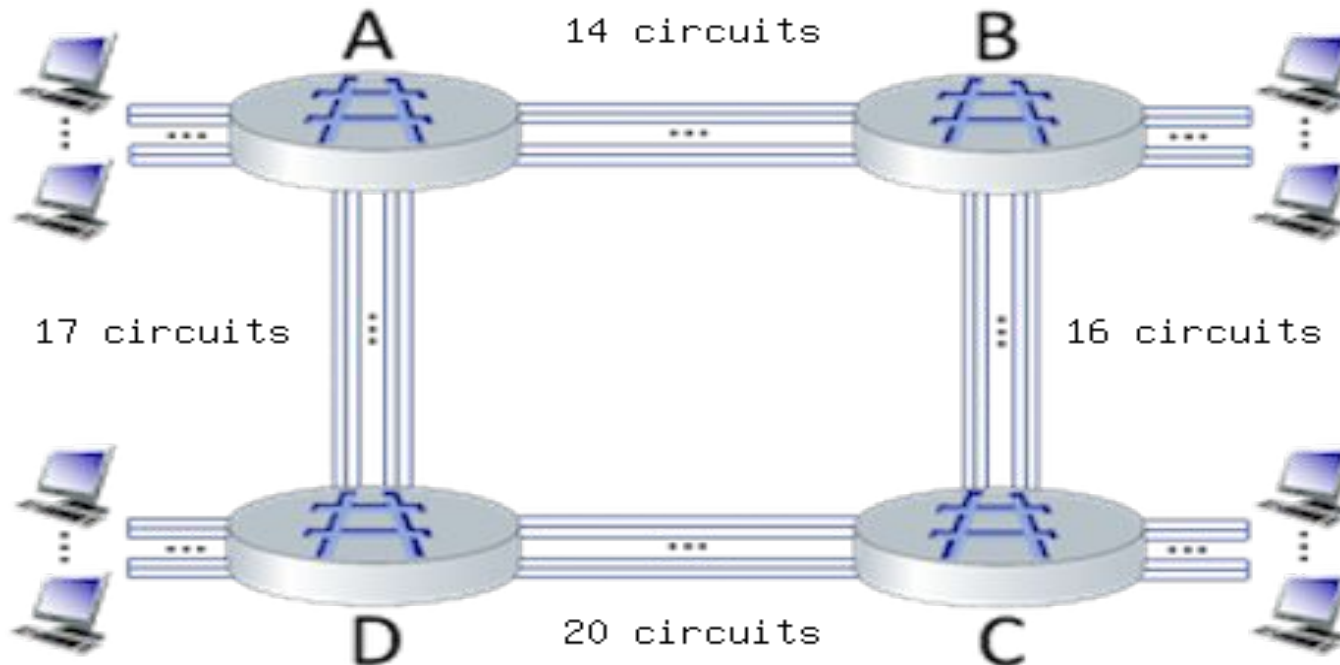
1. What is the maximum achievable end-end throughput (in Mbps) for each of four client-to-server pairs, assuming that the middle link is fairly shared (divides its transmission rate equally)?
2. Which link is the bottleneck link? Format as  $R_c$ ,  $R_s$ , or  $R$
3. Assuming that the servers are sending at the maximum rate possible, what are the link utilizations for the server links ( $R_s$ )? Answer as a decimal
4. Assuming that the servers are sending at the maximum rate possible, what are the link utilizations for the client links ( $R_c$ )? Answer as a decimal
5. Assuming that the servers are sending at the maximum rate possible, what is the link utilizations for the shared link ( $R$ )? Answer as a decimal

### Solutions:

1. The maximum achievable end-end throughput is the capacity of the link with the minimum capacity, which is 50 Mbps
2. The bottleneck link is the link with the smallest capacity between  $R_S$ ,  $R_C$ , and  $R/4$ . The bottleneck link is  $R_S$ .
3. The server's utilization =  $R_{\text{bottleneck}} / R_S = 50 / 50 = 1$
4. The client's utilization =  $R_{\text{bottleneck}} / R_C = 50 / 90 = 0.56$
5. The shared link's utilization =  $R_{\text{bottleneck}} / (R / 4) = 50 / (300 / 4) = 0.67$

## Packet Switching vs Circuit Switching – Numerical Example

Consider the circuit-switched network shown in the figure below, with circuit switches A, B, C, and D. Suppose there are 14 circuits between A and B, 16 circuits between B and C, 20 circuits between C and D, and 17 circuits between D and A



### Question List

1. What is the maximum number of connections that can be ongoing in the network at any one time?
2. Suppose that these maximum number of connections are all ongoing. What happens when another call connection request arrives to the network, will it be accepted? Answer Yes or No
3. Suppose that every connection requires 2 consecutive hops, and calls are connected clockwise. For example, a connection can go from A to C, from B to D, from C to A, and from D to B. With these constraints, what is the maximum number of connections that can be ongoing in the network at any one time?
4. Suppose that 12 connections are needed from A to C, and 15 connections are needed from B to D. Can we route these calls through the four links to accommodate all 27 connections? Answer Yes or No



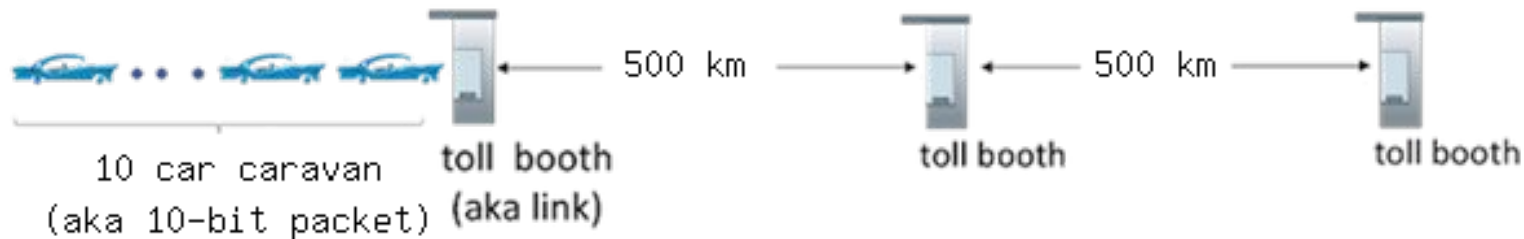
### Solution

1. The maximum number of connections that can be ongoing at any one time is the sum of all circuits, which happens when 14 connections go from A to B, 16 connections go from B to C, 20 connections go from C to D, and 17 connections go from D to A. This sum is 67.
2. No, it will be blocked because there are no free circuits.
3. There can be a maximum of 33 connections. Consider routes A->C and C->A, sum the bottleneck links, consider any leftover capacity that would allow for B->D and D->B connections, and compare that value to the equivalent of B->D and D->B.
4. Using our answer from question 4, the sum of our needed connections is 27, and we have 33 available connections, so it is possible.



## Car - Caravan Analogy

Consider the figure below, adapted from Figure 1.17 in the text, which draws the analogy between store-and-forward link transmission and propagation of bits in packet along a link, and cars in a caravan being serviced at a toll booth and then driving along a road to the next tollbooth.



Suppose the caravan has 10 cars, and that the tollbooth services (that is, transmits) a car at a rate of one car per 2 seconds. Once receiving serving a car proceeds to the next toll booth, which is 500 kilometers away at a rate of 10 kilometers per second. Also assume that whenever the first car of the caravan arrives at a tollbooth, it must wait at the entrance to the tollbooth until all of the other cars in its caravan have arrived, and lined up behind it before being serviced at the toll booth. (That is, the entire caravan must be stored at the tollbooth before the first car in the caravan can pay its toll and begin driving towards the next tollbooth).



### Question List

1. Once a car enters service at the tollbooth, how long does it take until it leaves service?
2. How long does it take for the entire caravan to receive service at the tollbooth (that is the time from when the first car enters service until the last car leaves the tollbooth)?
3. Once the first car leaves the tollbooth, how long does it take until it arrives at the next tollbooth?
4. Once the last car leaves the tollbooth, how long does it take until it arrives at the next tollbooth?
5. Once the first car leaves the tollbooth, how long does it take until it enters service at the next tollbooth?
6. Are there ever two cars in service at the same time, one at the first toll booth and one at the second toll booth?  
Answer Yes or No
7. Are there ever zero cars in service at the same time, i.e., the caravan of cars has finished at the first toll both but not yet arrived at the second tollbooth? Answer Yes or No



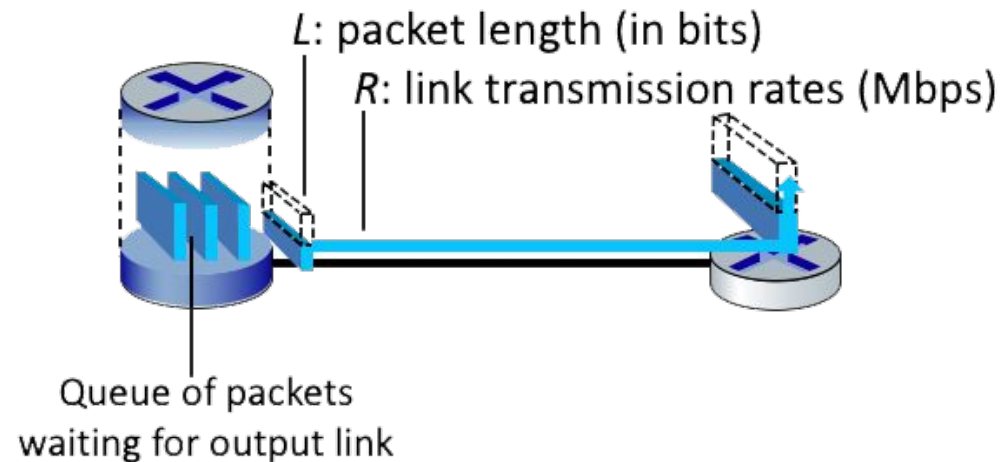
### Solution

1. Service time is 2 seconds
2. It takes 20 seconds to service every car, (10 cars \* 2 seconds per car)
3. It takes 50 seconds to travel to the next toll booth (500 km / 10 km/s)
4. Just like in the previous question, it takes 50 seconds, regardless of the car
5. It takes 68 seconds until the first car gets serviced at the next toll booth (10-1 cars \* 2 seconds per car + 500 km / 10 km/s)
6. No, because cars can't get service at the next tollbooth until all cars have arrived
7. Yes, one notable example is when the last car in the caravan is serviced but is still travelling to the next toll booth; all other cars have to wait until it arrives, thus no cars are being serviced



## Computing the one-hop transmission delay

Consider the figure below, in which a single router is transmitting packets, each of length  $L$  bits, over a single link with transmission rate  $R$  Mbps to another router at the other end of the link.



Suppose that the packet length is  $L = 8000$  bits, and that the link transmission rate along the link to router on the right is  $R = 1$  Mbps.

Round your answer to two decimals after leading zeros



### Question List

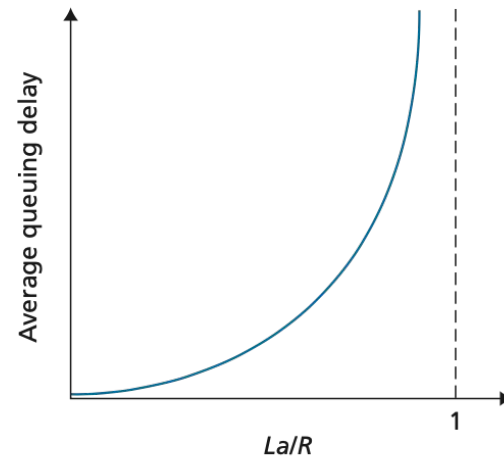
1. What is the transmission delay?
2. What is the maximum number of packets per second that can be transmitted by this link?

### Solution

1. The transmission delay =  $L/R = 8000 \text{ bits} / 1000000 \text{ bps} = 0.008 \text{ seconds}$
1. The number of packets that can be transmitted in a second into the link =  $R / L = 1000000 \text{ bps} / 8000 \text{ bits} = 125 \text{ packets}$



Consider the queuing delay in a router buffer, where the packet experiences a delay as it waits to be transmitted onto the link. The length of the queuing delay of a specific packet will depend on the number of earlier-arriving packets that are queued and waiting for transmission onto the link. If the queue is empty and no other packet is currently being transmitted, then our packet's queuing delay will be zero. On the other hand, if the traffic is heavy and many other packets are also waiting to be transmitted, the queuing delay will be long.



Assume a constant transmission rate of  $R = 1900000$  bps, a constant packet-length  $L = 4700$  bits, and  $a$  is the average rate of packets/second. Traffic intensity  $I = La/R$ , and the queuing delay is calculated as  $\frac{I}{R(1 - I)}$  for  $I < 1$ .

### Question List

1. In practice, does the queuing delay tend to vary a lot? Answer with Yes or No
2. Assuming that  $a = 35$ , what is the queuing delay? Give your answer in milliseconds (ms)
3. Assuming that  $a = 77$ , what is the queuing delay? Give your answer in milliseconds (ms)
4. Assuming the router's buffer is infinite, the queuing delay is 0.3815 ms, and 613 packets arrive. How many packets will be in the buffer 1 second later?
5. If the buffer has a maximum size of 627 packets, how many of the 613 packets would be dropped upon arrival from the previous question?



### Solution

1. Yes, in practice, queuing delay can vary significantly. We use the above formulas as a way to give a rough estimate, but in a real-life scenario it is much more complicated.

$$I = \lambda a / R = 4700 * 35 / 19000000 = 0.865$$

$$2. \text{ Queuing Delay} = \frac{I \lambda}{R(1 - I)} * 1000 = \frac{0.0865 * 4700}{(19000000 * (1 - 0.0865))} * 1000 = 0.23 \text{ ms.}$$

$$I = \lambda a / R = 4700 * 77 / 19000000 = 0.19$$

$$3. \text{ Queuing Delay} = \frac{I \lambda}{R(1 - I)} * 1000 = \frac{0.19 * 4700}{(19000000 * (1 - 0.19))} * 1000 = 0.58 \text{ ms.}$$

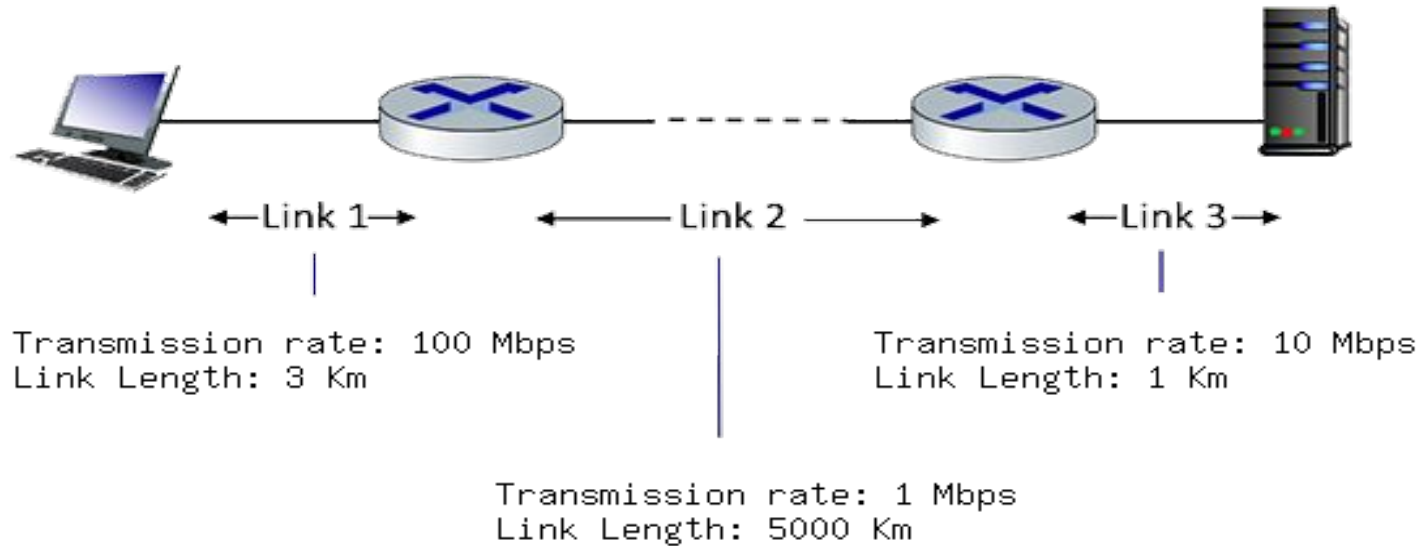
$$4. \text{ Packets left in buffer} = a - \text{floor}(1000 / \text{delay}) = 613 - \text{floor}(1000 / 0.3815) = 0 \text{ packets.}$$

$$5. \text{ Packets dropped} = \text{packets} - \text{buffer size} = 613 - 627 \text{ hence } 0 \text{ dropped packets.}$$



## Computing end-end delay (transmission and propagation delay)

Consider the figure below, with three links, each with the specified transmission rate and link length.



Assume the length of a packet is 12000 bits. The speed of light propagation delay on each link is  $3 \times 10^8$  m/sec

Round your answer to two decimals after leading zeros

## Computing end-end delay (transmission and propagation delay)

---

### Question List

1. What is the transmission delay of link 1?
2. What is the propagation delay of link 1?
3. What is the total delay of link 1?
4. What is the transmission delay of link 2?
5. What is the propagation delay of link 2?
6. What is the total delay of link 2?
7. What is the transmission delay of link 3?
8. What is the propagation delay of link 3?
9. What is the total delay of link 3?
10. What is the total delay?



## Computing end-end delay (transmission and propagation delay)

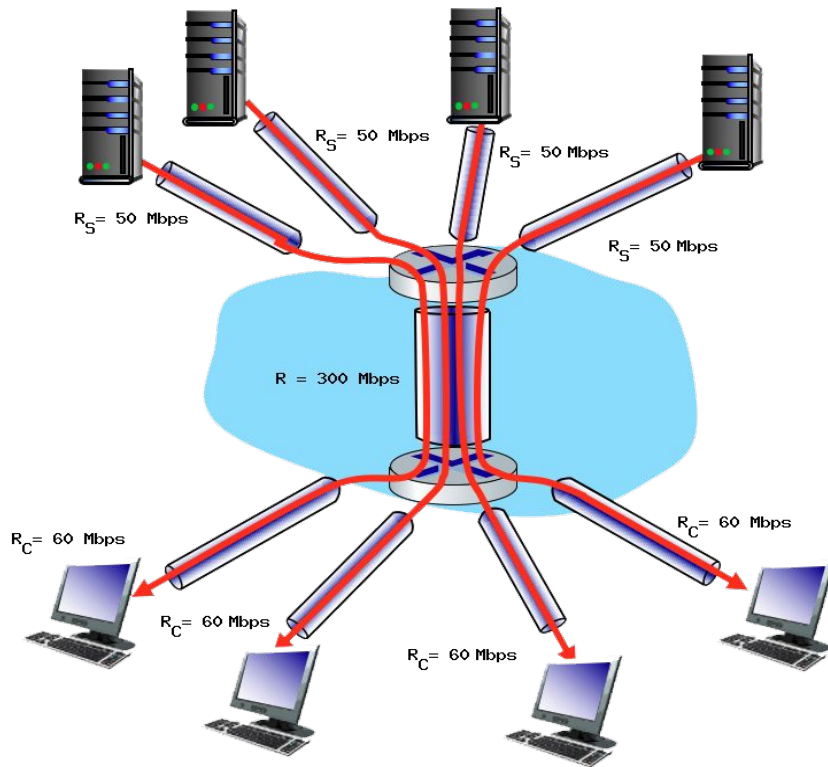
### Solution

1. Link 1 transmission delay =  $L/R = 12000 \text{ bits} / 100 \text{ Mbps} = 0.00012 \text{ seconds}$
2. Link 1 propagation delay =  $d/s = (3 \text{ Km}) * 1000 / 3*10^8 \text{ m/sec} = 1.00\text{E-}5 \text{ seconds}$
3. Link 1 total delay =  $d_t + d_p = 0.00012 \text{ seconds} + 1.00\text{E-}5 \text{ seconds} = 0.00013 \text{ seconds}$
4. Link 2 transmission delay =  $L/R = 12000 \text{ bits} / 1 \text{ Mbps} = 0.012 \text{ seconds}$
5. Link 2 propagation delay =  $d/s = (5000 \text{ Km}) * 1000 / 3*10^8 \text{ m/sec} = 0.017 \text{ seconds}$
6. Link 2 total delay =  $d_t + d_p = 0.012 \text{ seconds} + 0.017 \text{ seconds} = 0.029 \text{ seconds}$
7. Link 3 transmission delay =  $L/R = 12000 \text{ bits} / 10 \text{ Mbps} = 0.0012 \text{ seconds}$
8. Link 3 propagation delay =  $d/s = (1 \text{ Km}) * 1000 / 3*10^8 \text{ m/sec} = 3.33\text{E-}6 \text{ seconds}$
9. Link 3 total delay =  $d_t + d_p = 0.0012 \text{ seconds} + 3.33\text{E-}6 \text{ seconds} = 0.0012 \text{ seconds}$
10. The total delay =  $d_{L1} + d_{L2} + d_{L3} = 0.00013 \text{ seconds} + 0.029 \text{ seconds} + 0.0012 \text{ seconds} = 0.03 \text{ seconds}$



## End to End Throughput and Bottleneck Links

Consider the scenario shown below, with four different servers connected to four different clients over four three-hop paths. The four pairs share a common middle hop with a transmission capacity of  $R = 300$  Mbps. The four links from the servers to the shared link have a transmission capacity of  $R_S = 50$  Mbps. Each of the four links from the shared middle link to a client has a transmission capacity of  $R_C = 60$  Mbps.



### Question List

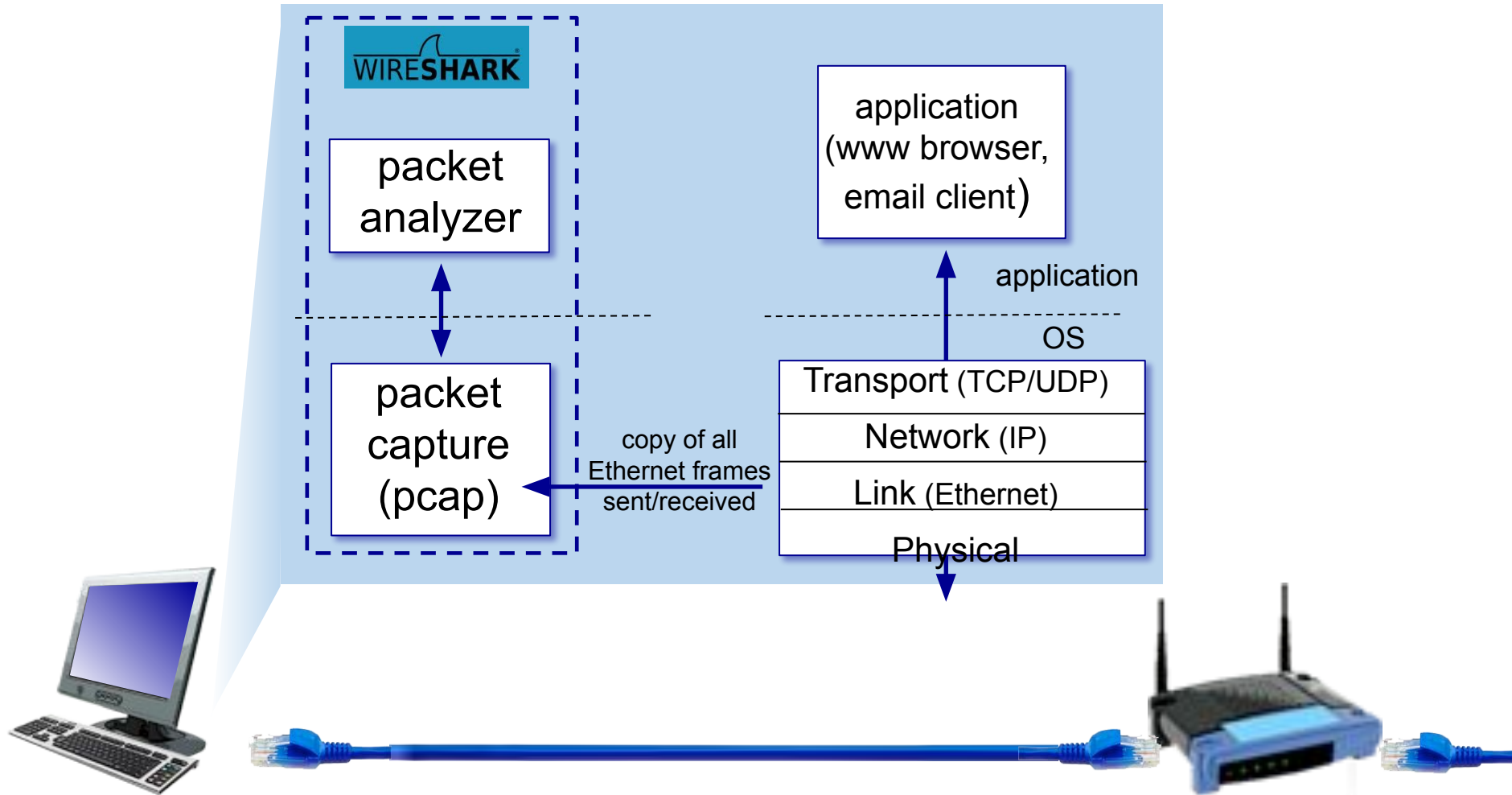
1. What is the maximum achievable end-end throughput (in Mbps) for each of four client-to-server pairs, assuming that the middle link is fairly shared (divides its transmission rate equally)?
2. Which link is the bottleneck link? Format as  $R_c$ ,  $R_s$ , or  $R$
3. Assuming that the servers are sending at the maximum rate possible, what are the link utilizations for the server links ( $R_s$ )? Answer as a decimal
4. Assuming that the servers are sending at the maximum rate possible, what are the link utilizations for the client links ( $R_c$ )? Answer as a decimal
5. Assuming that the servers are sending at the maximum rate possible, what is the link utilizations for the shared link ( $R$ )? Answer as a decimal



### Solution

1. The maximum achievable end-end throughput is the capacity of the link with the minimum capacity, which is 50 Mbps
2. The bottleneck link is the link with the smallest capacity between RS, RC, and R/4. The bottleneck link is Rs.
3. The server's utilization =  $R_{\text{bottleneck}} / R_S = 50 / 50 = 1$
4. The client's utilization =  $R_{\text{bottleneck}} / R_C = 50 / 60 = 0.83$
5. The shared link's utilization =  $R_{\text{bottleneck}} / (R / 4) = 50 / (300 / 4) = 0.67$







**THANK YOU**

---

**Team Networks**

Department of Computer Science and Engineering



**PES**  
**UNIVERSITY**

CELEBRATING 50 YEARS

# COMPUTER NETWORKS

---

## TEAM NETWORKS

Department of Computer Science and Engineering

# COMPUTER NETWORKS

---

## Application Layer

Department of Computer Science and Engineering

## Unit – 1 Application Layer

### 2.1 Principles of Network Applications

### 2.2 Web, HTTP and HTTPS

### Our goals:

- Conceptual *and* implementation aspects of application-layer protocols
  - transport-layer service models
  - client-server paradigm
  - peer-to-peer paradigm
- Learn about protocols by examining popular application-layer protocols
  - HTTP

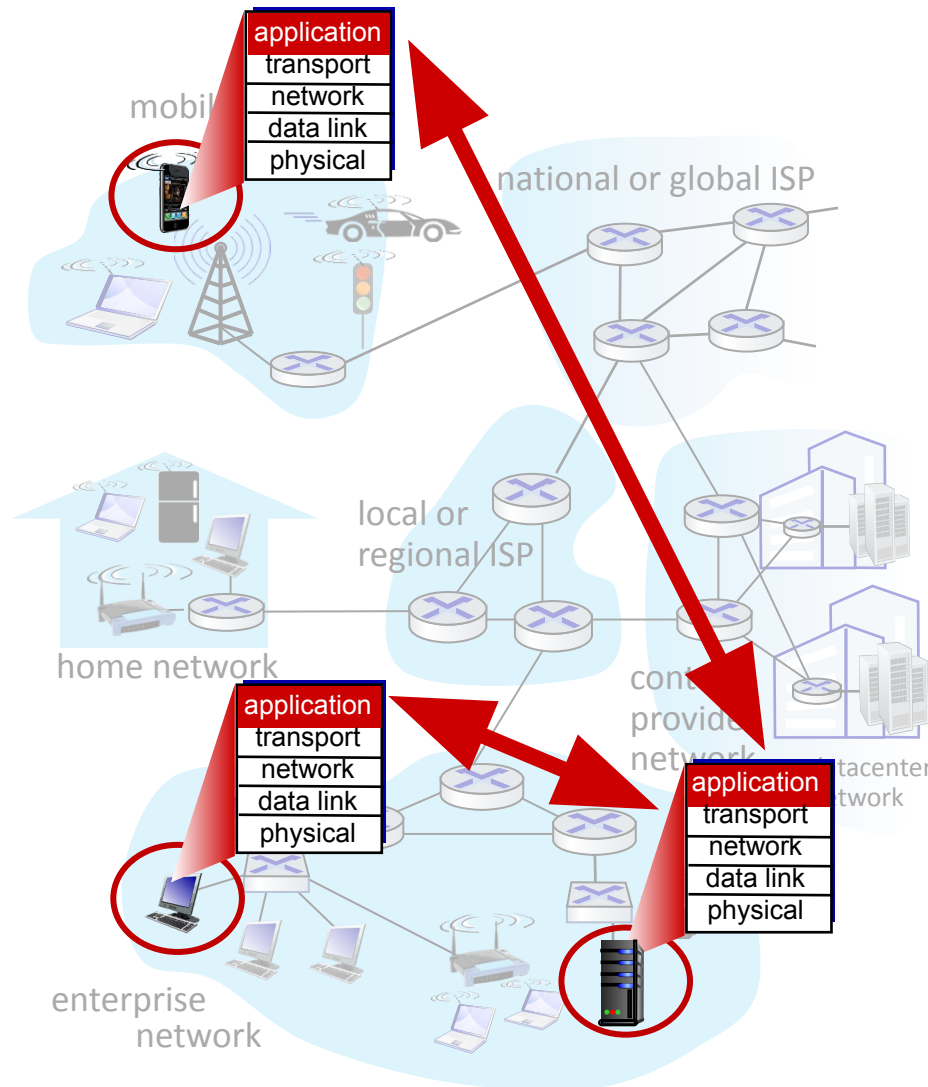
- social networking
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP
- real-time video conferencing (e.g., Skype, Hangouts)
- Internet search
- remote login
- ...

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation

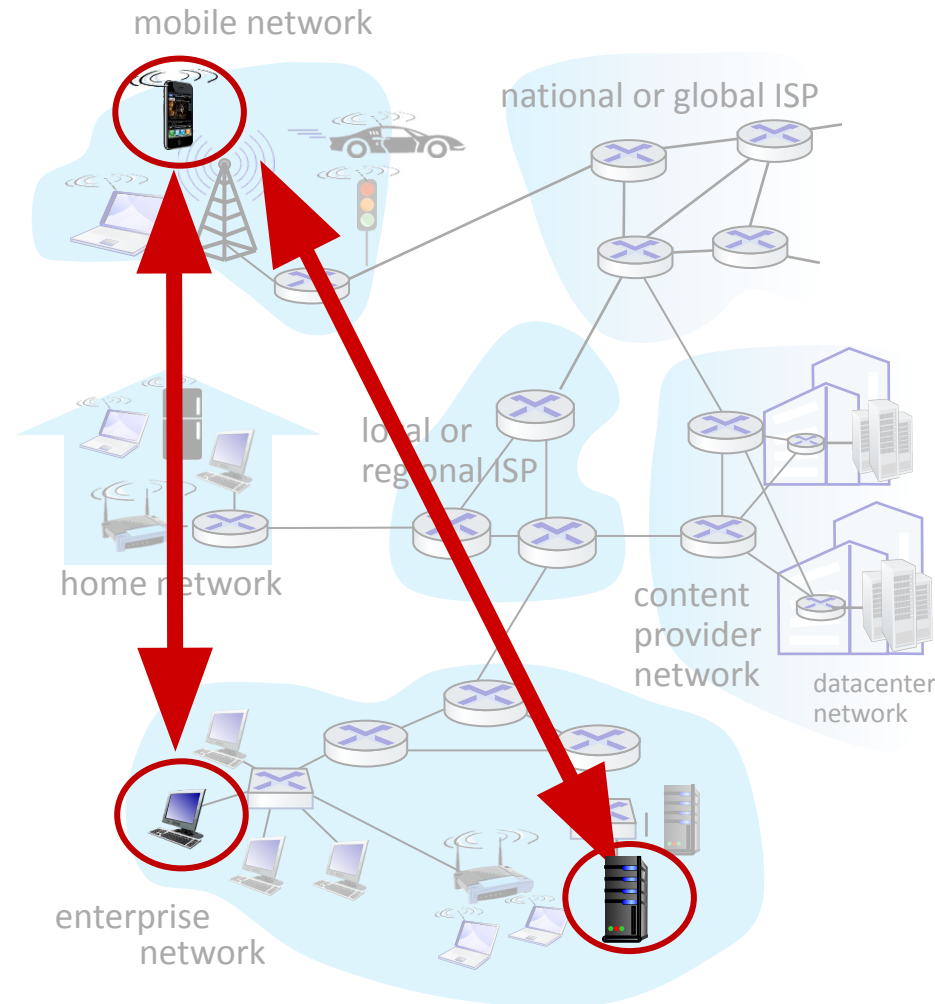


### server:

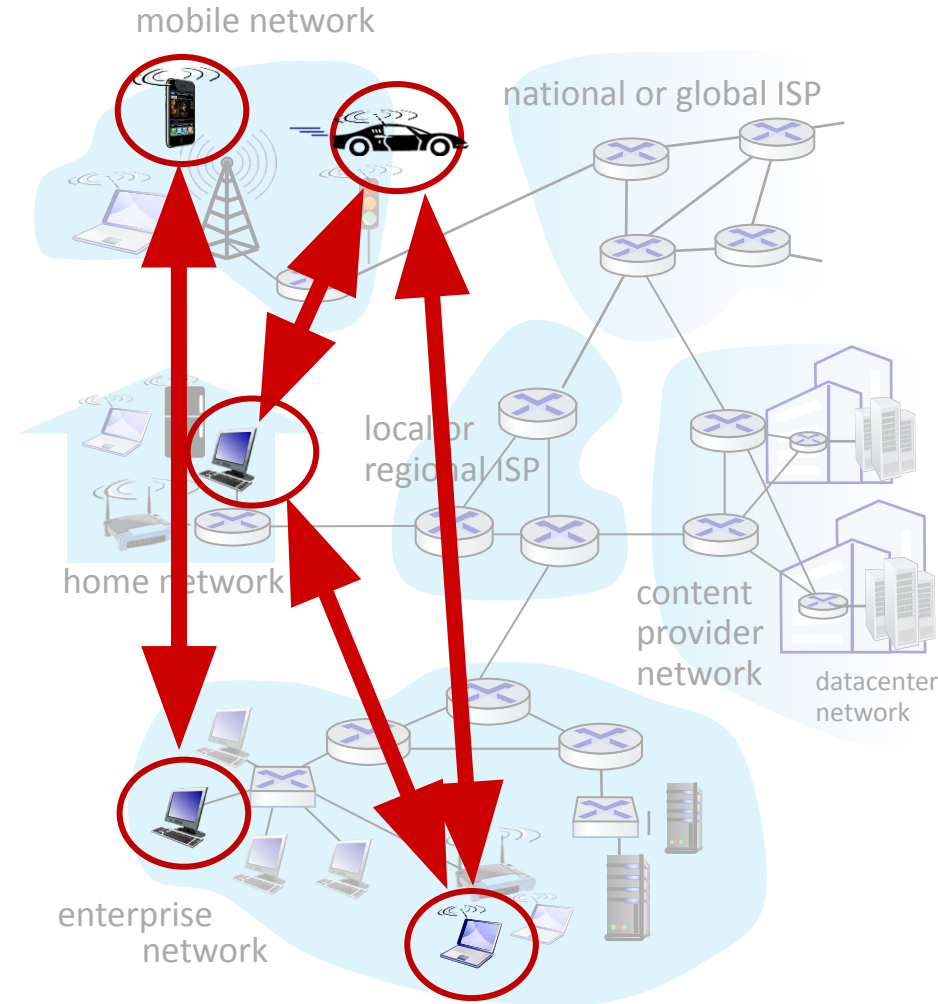
- always-on host
- permanent IP address
- often in data centers, for scaling

### clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other
- examples: HTTP, IMAP, FTP



- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- example: P2P file sharing



*process*: program running within a host

- within same host, two processes communicate using **inter-process communication** (defined by OS)
- processes in different hosts communicate by exchanging **messages**

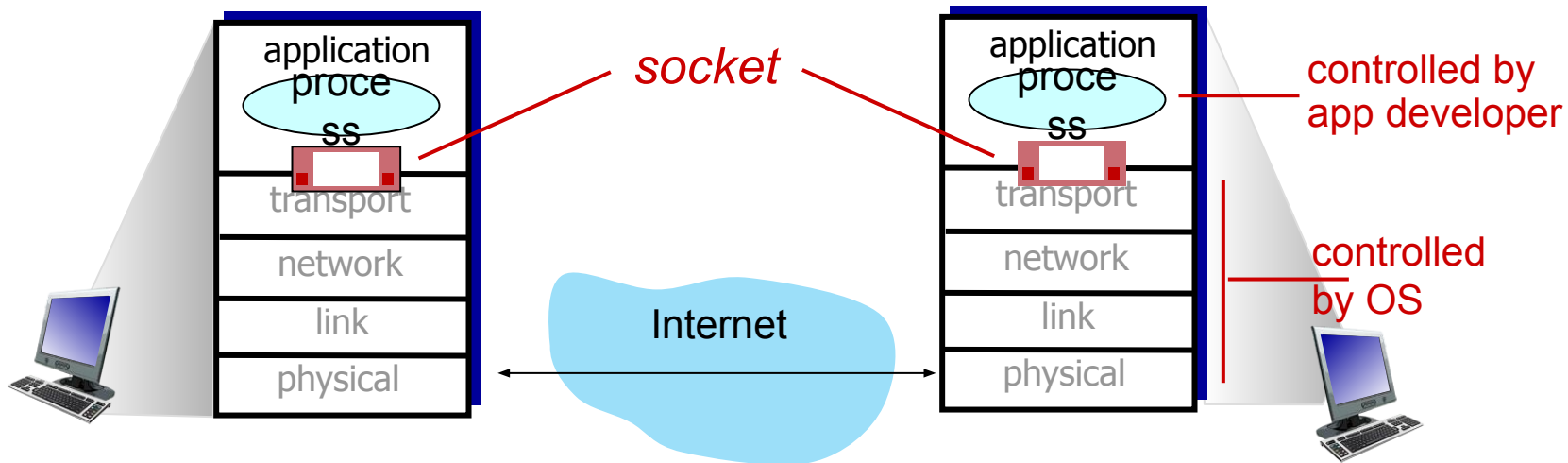
clients, servers

*client process*: process that initiates communication

*server process*: process that waits to be contacted

- note: applications with P2P architectures have client processes & server processes

- process sends/receives messages to/from its **socket**
- socket analogous to door
  - sending process shoves message out door
  - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
  - two sockets involved: one on each side



- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- Q: does IP address of host on which process runs suffice for identifying the process?
  - A: no, *many* processes can be running on same host
- *identifier* includes both **IP address** and **port numbers** associated with process on host.
- example port numbers:
  - HTTP server: 80
  - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
  - **IP address**: 128.119.245.12
  - **port number**: 80
- more shortly...

- **types of messages exchanged**,
  - e.g., request, response
- **message syntax**:
  - what fields in messages & how fields are delineated
- **message semantics**
  - meaning of information in fields
- **rules** for when and how processes send & respond to messages

### open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

### proprietary protocols:

- e.g., Skype

## What transport service does an App needs?

---

### data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

### timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

### throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

### security

- encryption, data integrity, ...

| application            | data loss     | throughput                               | time sensitive? |
|------------------------|---------------|------------------------------------------|-----------------|
| file transfer/download | no loss       | elastic                                  | no              |
| e-mail                 | no loss       | elastic                                  | no              |
| Web documents          | no loss       | elastic                                  | no              |
| real-time audio/video  | loss-tolerant | audio: 5Kbps-1Mbps<br>video:10Kbps-5Mbps | yes, 10's msec  |
| streaming audio/video  | loss-tolerant | same as above                            | yes, few secs   |
| interactive games      | loss-tolerant | Kbps+                                    | yes, 10's msec  |
| text messaging         | no loss       | elastic                                  | yes and no      |

### *TCP service:*

- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *does not provide*: timing, minimum throughput guarantee, security
- *connection-oriented*: setup required between client and server processes

### *UDP service:*

- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

| application            | application layer protocol                             | transport protocol |
|------------------------|--------------------------------------------------------|--------------------|
| file transfer/download | FTP [RFC 959]                                          | TCP                |
| e-mail                 | SMTP [RFC 5321]                                        | TCP                |
| Web documents          | HTTP 1.1 [RFC 7320]                                    | TCP                |
| Internet telephony     | SIP [RFC 3261], RTP [RFC 3550], or proprietary (Skype) | TCP or UDP         |
| streaming audio/video  | DASH                                                   | TCP                |
| interactive games      | WOW, FPS (proprietary)                                 | UDP or TCP         |

## Unit – 1 Application Layer

2.1 Principles of Network Applications

**2.2** Web, HTTP and HTTPS

*First, a quick review...*

- web page consists of *objects*, each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *URL*, e.g.,
- If a Web page contains HTML text and 5 JPEG images, then the Web page has 6 objects: the base HTML file plus the 5 images.

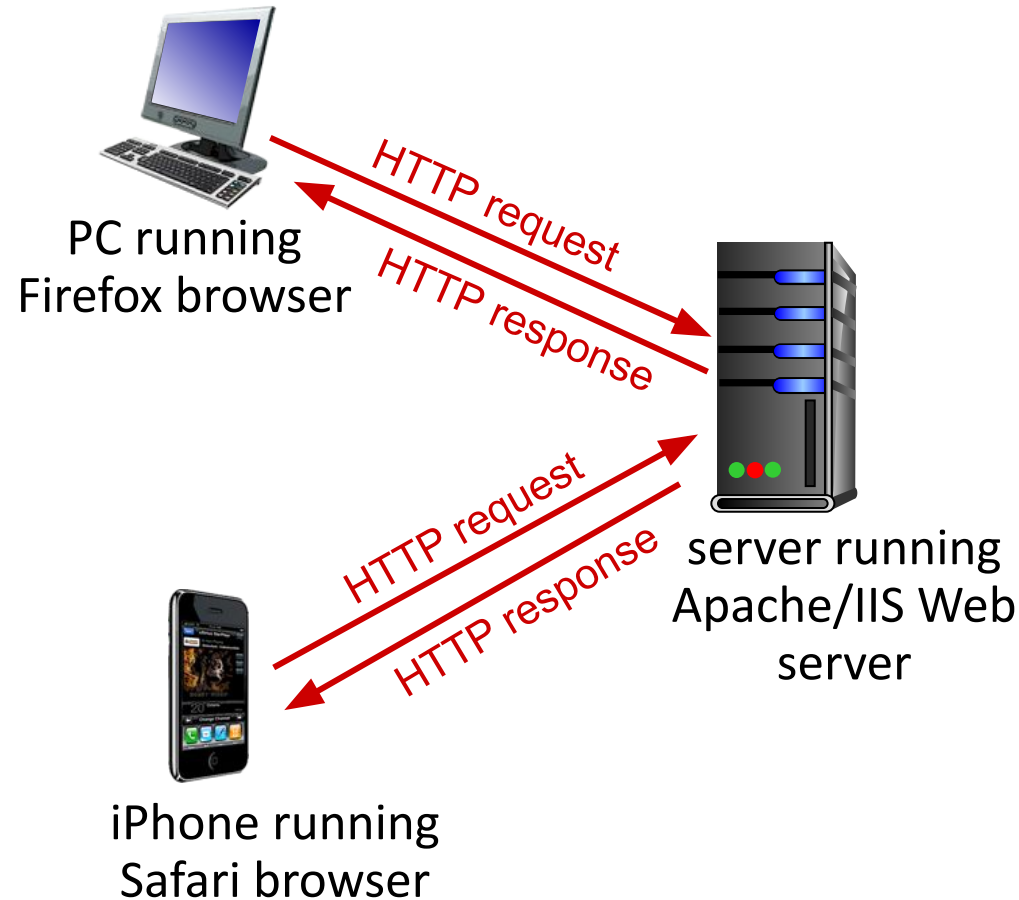
`www.someschool.edu/someDept/pic.gif`

host name

path name

### HTTP: hypertext transfer protocol

- Web's application layer protocol
- client/server model:
  - *client*: browser that requests, receives, (using HTTP protocol) and “displays” Web objects
  - *server*: Web server sends (using HTTP protocol) objects in response to requests



**Defined in RFC 1945; RFC 2616**

### *HTTP uses TCP:*

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

### *HTTP is “stateless”*

- server maintains *no* information about past client requests

*—aside—*

#### protocols that maintain “state” are complex!

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled

### *Non-persistent HTTP*

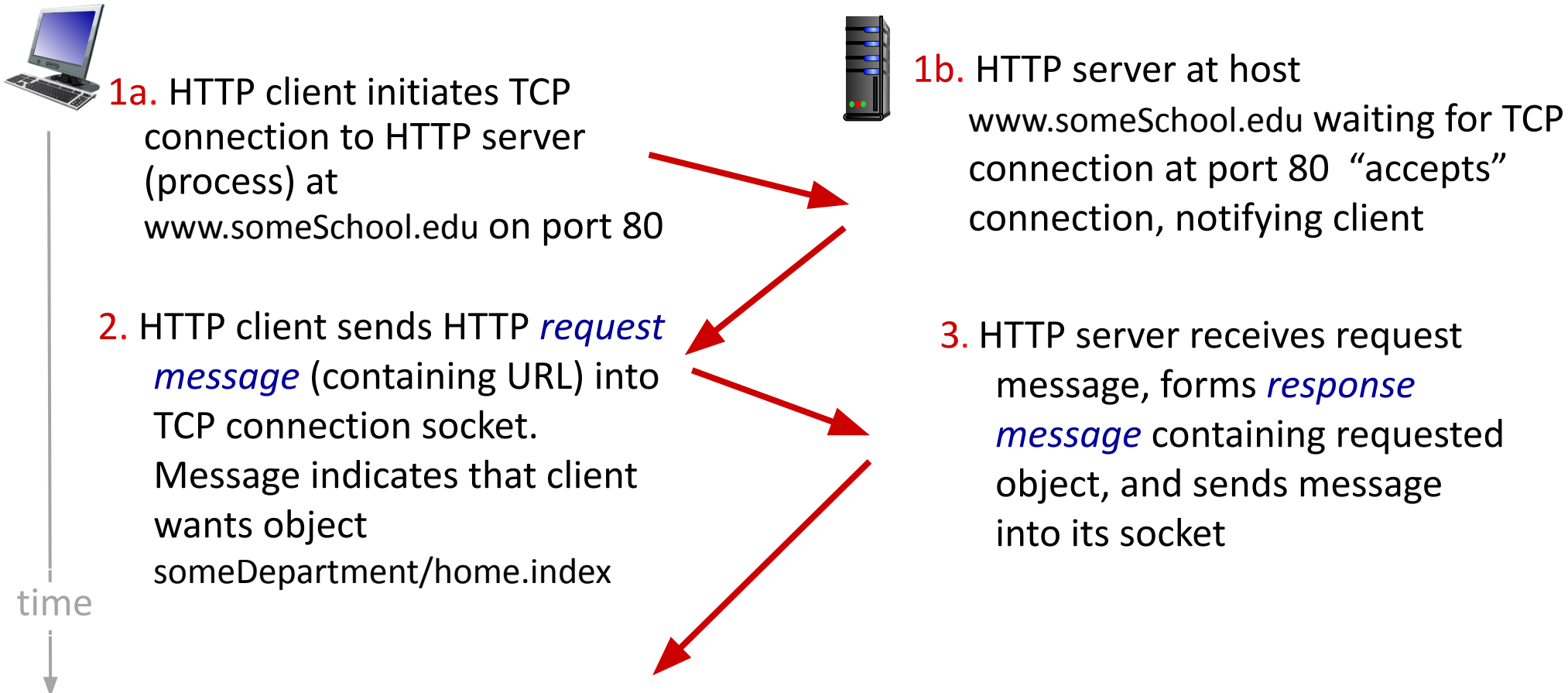
1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

downloading multiple objects  
required multiple connections

### *Persistent HTTP*

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

User enters URL: `www.someSchool.edu/someDepartment/home.index`  
(base HTML file containing text, references to 10 jpeg images)



User enters URL: `www.someSchool.edu/someDepartment/home.index`  
(containing text, references to 10 jpeg images)



4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

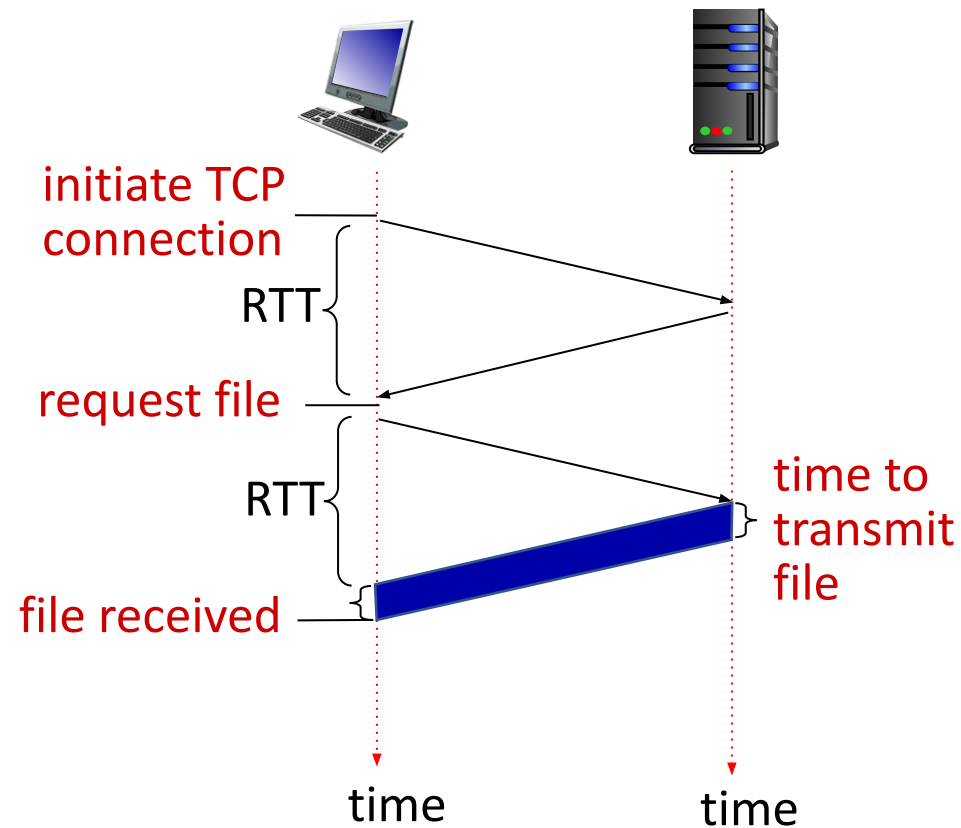
time



**RTT (definition):** time for a small packet to travel from client to server and back

**HTTP response time (per object):**

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



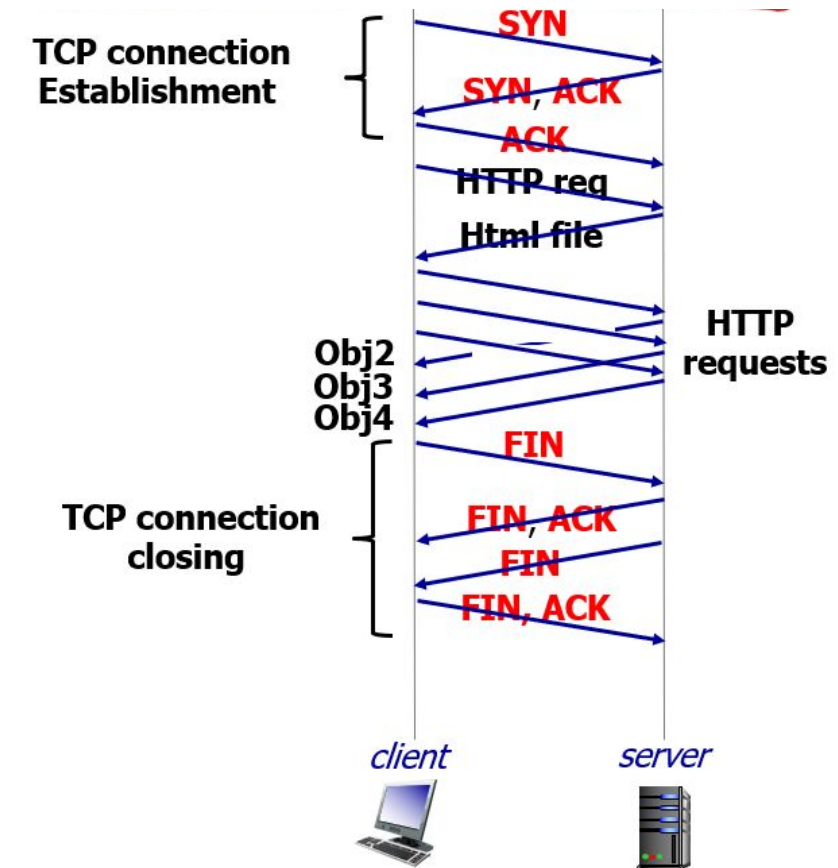
*Non-persistent HTTP response time = 2RTT + file transmission time*

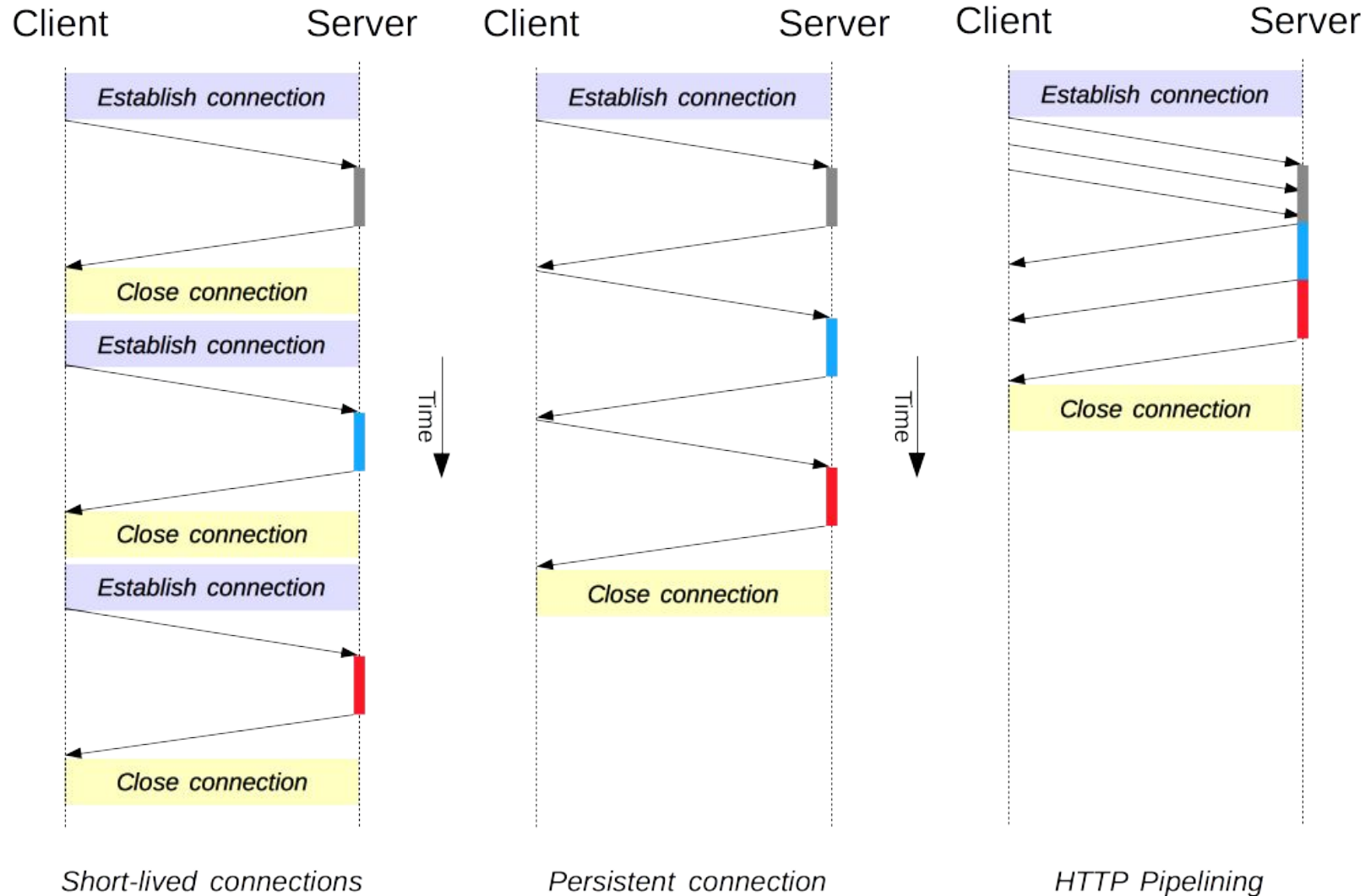
### *Non-persistent HTTP issues:*

- requires 2 RTTs per object
- OS overhead for *each* TCP connection (TCP buffer and variables)
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

### *Persistent HTTP (HTTP1.1):*

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)





## Unit – 1 Application Layer

2.1 Principles of Network Applications

**2.2** Web, HTTP and HTTPS

- two types of HTTP messages: *request, response*
- **HTTP request message:**
  - ASCII (human-readable format)

request line (GET, POST,  
HEAD commands)

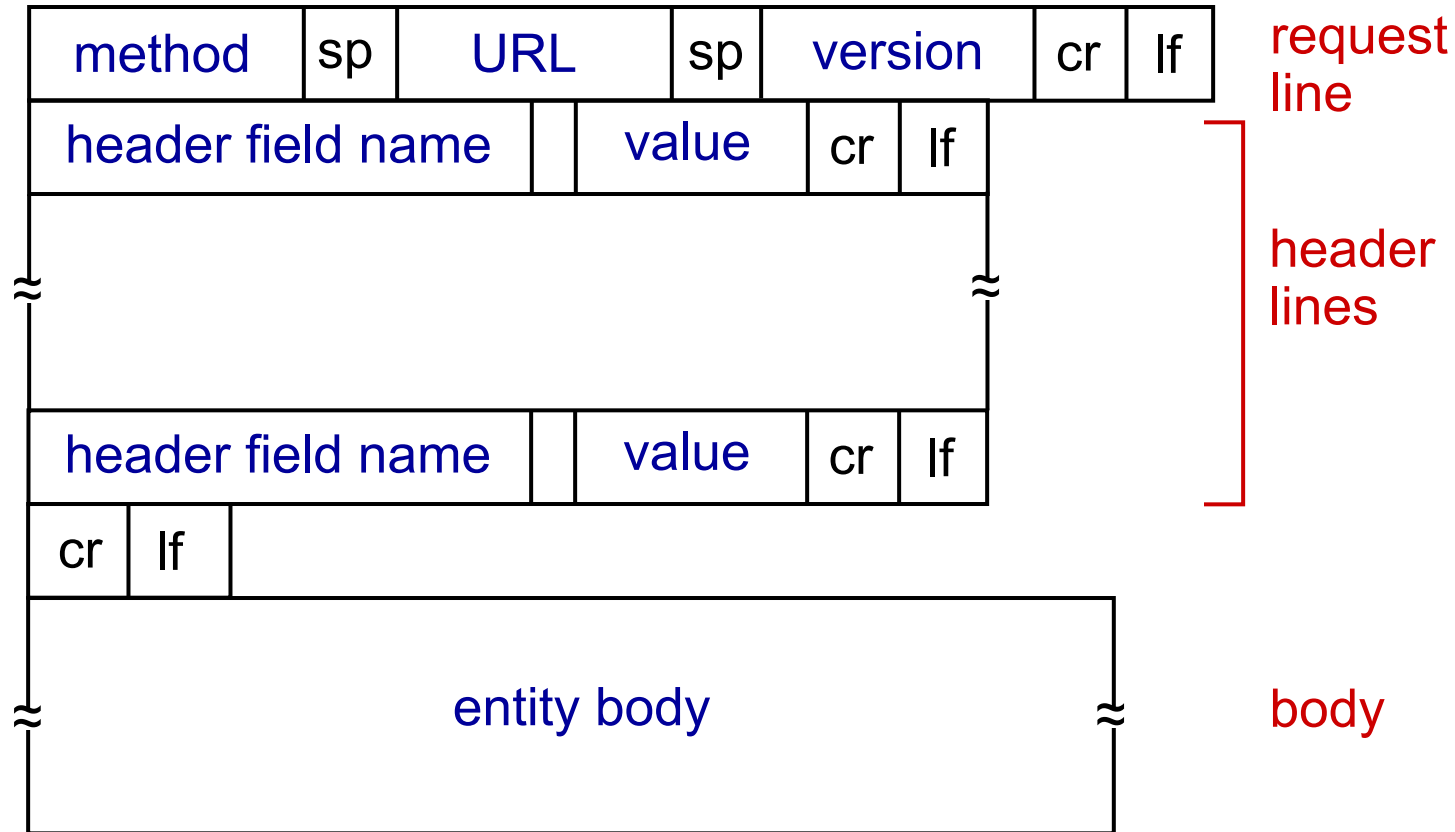
header  
lines

carriage return, line feed  
at start of line indicates  
end of header lines

carriage return character  
line-feed character

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

\* Check out the online interactive exercises for more  
examples: [http://gaia.cs.umass.edu/kurose\\_ross/interactive/](http://gaia.cs.umass.edu/kurose_ross/interactive/)



**HTTP specifications [RFC 1945; RFC 2616; RFC 7540]**

# COMPUTER NETWORKS

## HTTP Request Message – Wireshark Capture

Capturing from any

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

| No. | Time        | Source       | Destination  | Protocol | Length | Info                                      |
|-----|-------------|--------------|--------------|----------|--------|-------------------------------------------|
| 44  | 1.734232835 | 192.168.1.95 | 63.33.73.205 | HTTP     | 677    | GET /file1.html HTTP/1.1                  |
| 50  | 1.831703556 | 63.33.73.205 | 192.168.1.95 | HTTP     | 326    | HTTP/1.1 200 OK (text/html)               |
| 52  | 1.874581455 | 192.168.1.95 | 63.33.73.205 | HTTP     | 558    | GET /favicon.ico HTTP/1.1                 |
| 54  | 1.970307494 | 63.33.73.205 | 192.168.1.95 | HTTP     | 326    | HTTP/1.1 404 Not Found (application/json) |

Frame 44: 677 bytes on wire (5416 bits), 677 bytes captured (5416 bits) on interface 0

Linux cooked capture

Internet Protocol Version 4, Src: 192.168.1.95, Dst: 63.33.73.205

Transmission Control Protocol, Src Port: 33862, Dst Port: 80, Seq: 1, Ack: 1, Len: 609

Hypertext Transfer Protocol

GET /file1.html HTTP/1.1\r\n

Host: wireshark.grydeske.net\r\n

Connection: keep-alive\r\n

Pragma: no-cache\r\n

Cache-Control: no-cache\r\n

Upgrade-Insecure-Requests: 1\r\n

User-Agent: Mozilla/5.0 (X11; Linux x86\_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3809.87 Safari/537.36\r\n

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,\*/\*;q=0.8,application/signed-exchange;v=b3\r\n

Referer: http://localhost:8080/wireshark/wireshark-http.html\r\n

Accept-Encoding: gzip, deflate\r\n

Accept-Language: en-US,en;q=0.9,da;q=0.8\r\n

Cookie: \_\_utma=231828553.1458339319.1536683537.1537430811.1537432942.3\r\n\r\n

[Full request URI: http://wireshark.grydeske.net/file1.html]

[HTTP request 1/2]

[Response in frame: 50]

[Next request in frame: 52]

0000 00 04 00 01 00 06 9c eb e8 19 0a 4a 00 00 08 00 .....J....  
0010 45 00 02 95 53 6b 40 00 40 06 9a 02 c0 a8 01 5f E...Sk@. @.....  
0020 3f 21 49 cd 84 46 00 50 68 d6 3c f0 c3 38 b6 49 ?!I..F.P h<..8.I  
0030 80 18 01 06 4d 7d 00 00 01 01 08 0a e0 c1 51 3b ...M}... ..Q;  
0040 5f f3 8b d1 47 45 54 20 2f 66 69 6c 65 31 2e 68 ....GET /file1.h

### POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

### HEAD method:

- requests headers (only) that would be returned *if* specified URL were requested with an HTTP GET method.

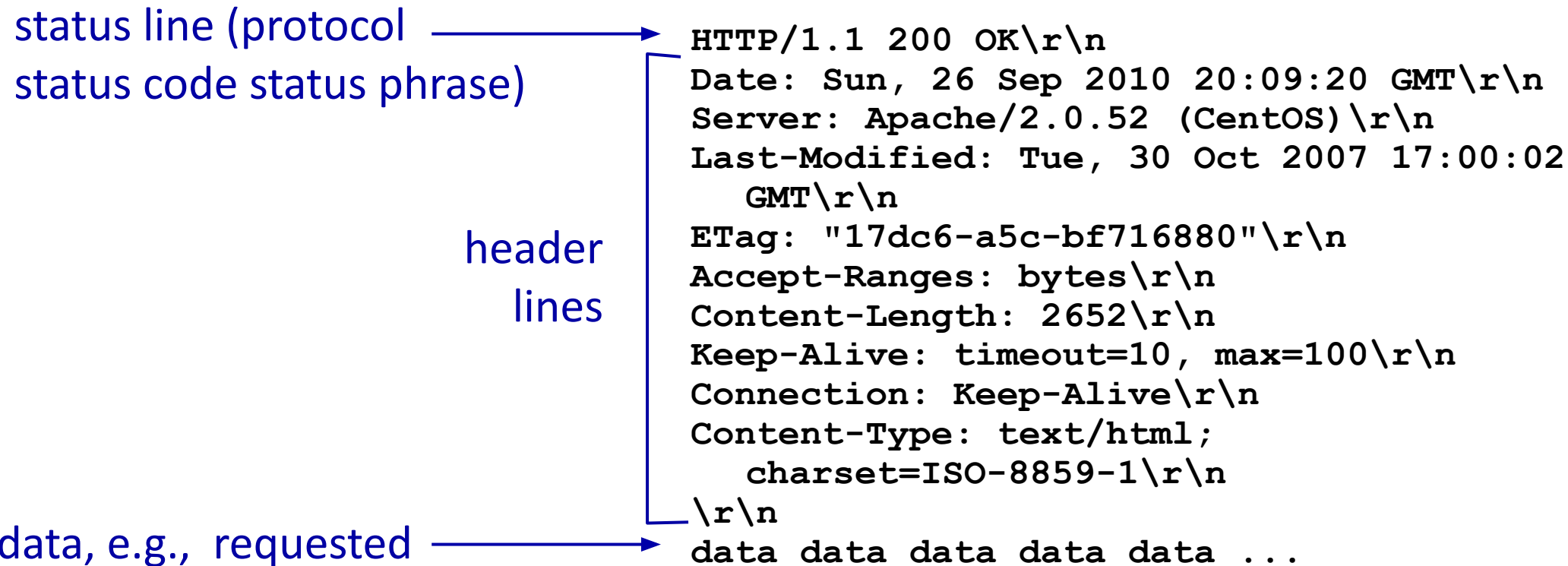
### GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

### PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

`www.somesite.com/animalsearch?monkeys&banana`



# COMPUTER NETWORKS

## HTTP Response Message – Wireshark Capture

Capturing from Microsoft: \Device\NPF\_{483C83F4-DCBA-4863-B523-3C4E1B03D06F} [Wireshark 1.8.5 (SVN Rev 47350 from /trunk-1.8)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: http Expression... Clear Apply Save

| No. | Time               | Source                     | Destination     | Protocol | Length | Info                                                   |
|-----|--------------------|----------------------------|-----------------|----------|--------|--------------------------------------------------------|
| 222 | 21:13:58.590670000 | 10.36.40.181               | 239.255.255.250 | SSDP     | 528    | NOTIFY * HTTP/1.1                                      |
| 223 | 21:13:58.590877000 | fe80::4195:59f3:544ff02::c |                 | SSDP     | 556    | NOTIFY * HTTP/1.1                                      |
| 233 | 21:13:59.117254000 | 10.36.40.181               | 128.119.245.12  | HTTP     | 473    | GET /wireshark-labs/HTTP-wireshark-file3.html HTTP/1.1 |
| 241 | 21:13:59.150482000 | 128.119.245.12             | 10.36.40.181    | HTTP     | 452    | HTTP/1.1 200 OK (text/html)                            |
| 242 | 21:13:59.190558000 | 10.36.40.181               | 239.255.255.250 | SSDP     | 556    | NOTIFY * HTTP/1.1                                      |
| 243 | 21:13:59.190758000 | fe80::4195:59f3:544ff02::c |                 | SSDP     | 584    | NOTIFY * HTTP/1.1                                      |
| 245 | 21:13:59.443994000 | 10.36.40.181               | 128.119.245.12  | HTTP     | 384    | GET /favicon.ico HTTP/1.1                              |
| 246 | 21:13:59.462702000 | 128.119.245.12             | 10.36.40.181    | HTTP     | 532    | HTTP/1.1 404 Not Found (text/html)                     |
| 247 | 21:13:59.467414000 | 10.36.40.181               | 239.255.255.250 | SSDP     | 542    | NOTIFY * HTTP/1.1                                      |
| 248 | 21:13:59.467605000 | fe80::4195:59f3:544ff02::c |                 | SSDP     | 570    | NOTIFY * HTTP/1.1                                      |

Frame 241: 452 bytes on wire (3616 bits), 452 bytes captured (3616 bits) on interface 0

- Ethernet II, Src: Cisco\_4c:61:3f (00:1e:f7:4c:61:3f), Dst: HonHaiPr\_0a:de:6b (cc:af:78:0a:de:6b)
- Internet Protocol Version 4, Src: 128.119.245.12 (128.119.245.12), Dst: 10.36.40.181 (10.36.40.181)
- Transmission Control Protocol, Src Port: http (80), Dst Port: 55990 (55990), Seq: 4381, Ack: 420, Len: 398
- [5 Reassembled TCP Segments (4778 bytes): #234(1423), #237(1460), #239(1460), #235(37), #241(398)]
- Hypertext Transfer Protocol
  - HTTP/1.1 200 OK\r\n
  - Date: wed, 27 Feb 2013 02:14:00 GMT\r\n
  - Server: Apache/2.2.3 (CentOS)\r\n
  - Last-Modified: wed, 27 Feb 2013 02:13:01 GMT\r\n
  - ETag: "d6c97-1194-50408540"\r\n
  - Accept-Ranges: bytes\r\n
  - Content-Type: text/html; charset=UTF-8\r\n
  - Content-Length: 4500\r\n
  - Connection: Keep-Alive\r\n
  - Age: 0\r\n
  - \r\n

0000 cc af 78 0a de 6b 00 1e f7 4c 61 3f 08 00 45 00 ..x..k.. .La?..E.  
0010 01 b6 5f cd 00 00 3a 06 77 18 80 77 f5 0c 0a 24 .....: w..w...\$  
0020 28 b5 00 50 da b6 70 e7 fd 49 a7 2b b3 a2 50 18 (.P.p. .I.+..P.  
0030 ff ff 16 ab 00 00 70 3e 3c 2f 70 3e 3c 70 3e 54 .....p> </p><p>T  
0040 68 65 20 65 6e 75 6d 65 72 61 74 69 6f 6e 20 69 he enume ration i  
0050 6e 20 74 68 65 20 43 6f 6e 73 74 69 74 75 74 69 n the Co nstituti  
0060 6f 6e 2c 20 6f 66 20 63 65 72 74 61 69 6e 20 72 on of c ertain r

Frame (452 bytes) Reassembled TCP (4778 bytes)

Frame (frame), 452 bytes Packets: 336 Displayed: 40 Marked: 0 Profile: Default

- status code appears in 1st line in server-to-client response message.
- some sample codes:

### 200 OK

- request succeeded, requested object later in this message

### 301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

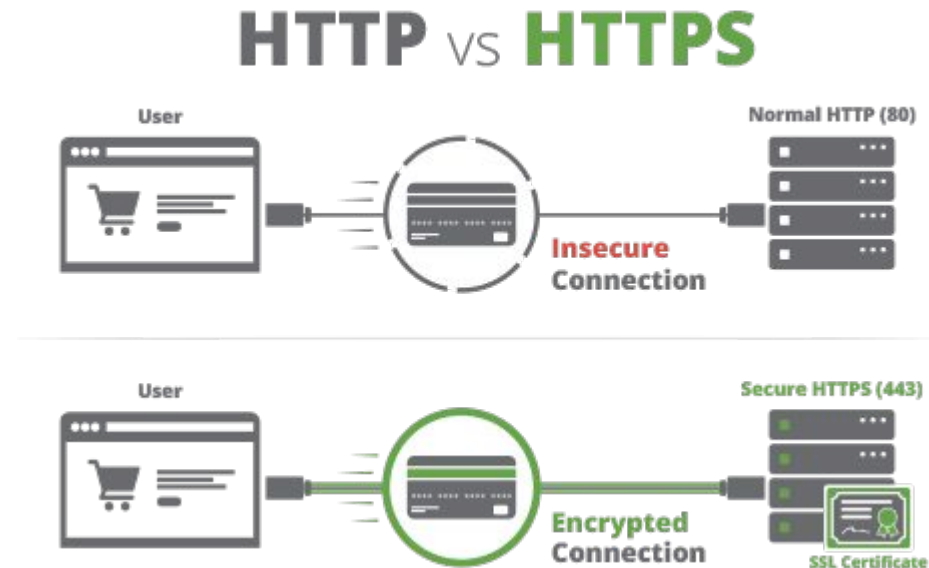
### 400 Bad Request

- request msg not understood by server

### 404 Not Found

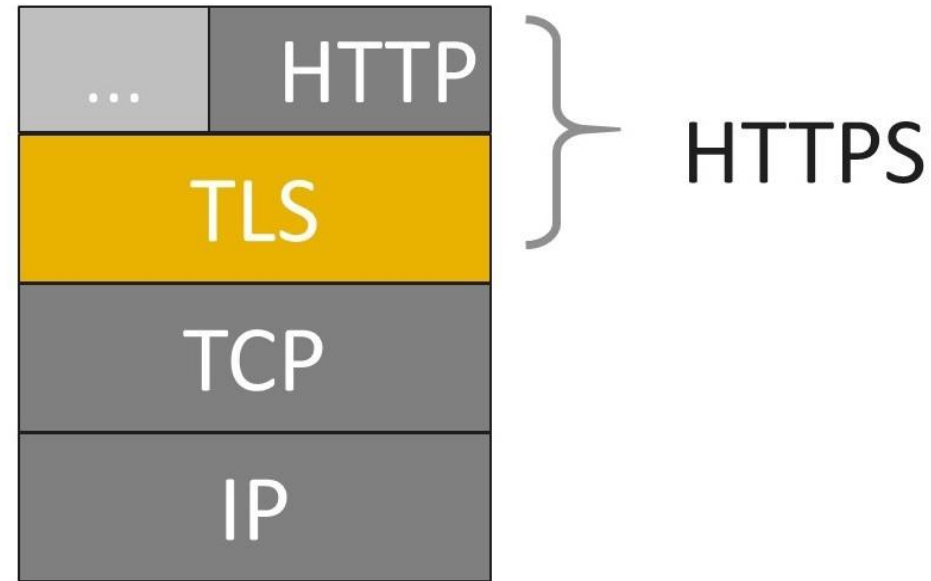
- requested document not found on this server

### 505 HTTP Version Not Supported



- HTTPS is HTTP with encryption – All communications between browser and server are encrypted (bi-directional).
- 'S' refers 'Secure' or HTTP over Secure Socket Layer.
- Uses TLS (SSL) to encrypt normal HTTP requests and responses.
- Attackers can't read the data crossing the wire and you know you are talking to the server you think you are talking too.

- HTTP + TLS -> Encrypted
- Uses port no. 443 for data communication.
- HTTPS is based on public/private-key cryptography.
  - The public key is used for encryption
  - The secret private key is required for decryption.
- SSL certificate is a web server's digital certificate issued by a third party CA.
  - Create an encrypted connection and establish trust.
- Is my certificate SSL or TLS?

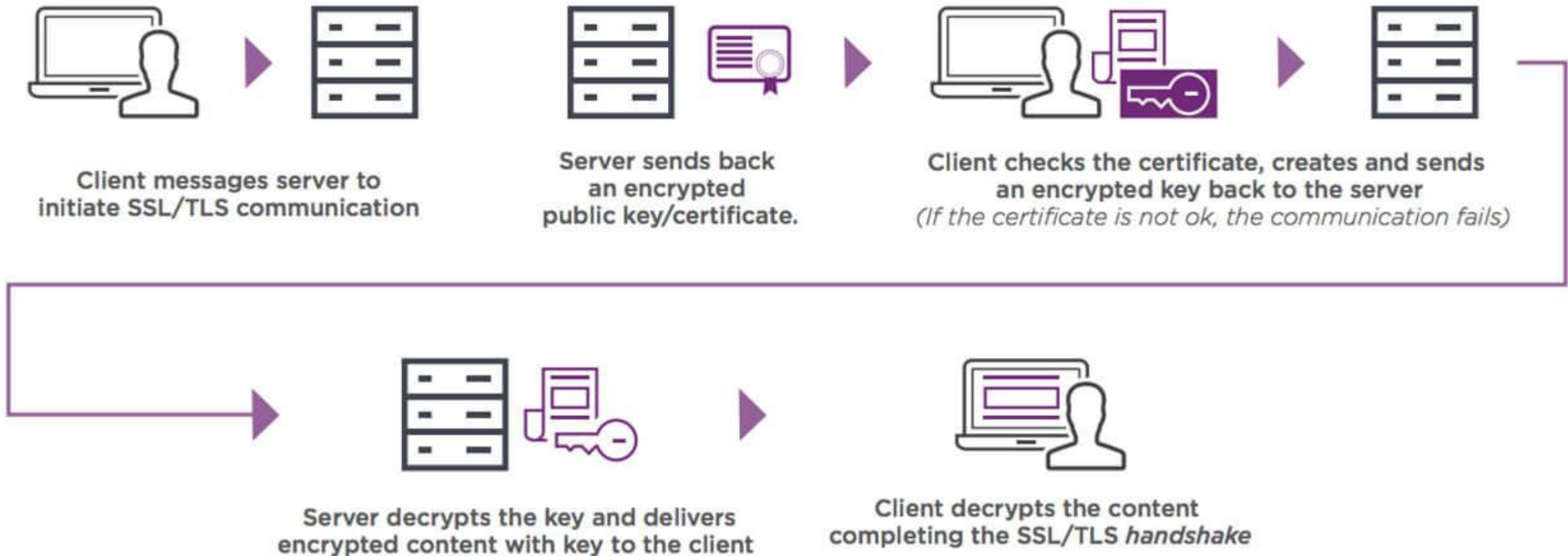


Any message encrypted with Bob's public key can be only decrypted with Bob's private key.

- Step 1: Browser requests secure pages (HTTPS) from a server.
  - Step 2: Server sends its public key with its SSL certificate (digitally signed by a third party – CA).
  - Step 3: On receipt of certificate, browser verifies issuer's digital signature. (green padlock key)
  - Step 4: Browser creates a symmetric key (shared key), keeps one and gives a copy to server. Encrypts it using server's public key.
  - Step 5: On receipt of encrypted secret key, decrypts it using its private key and gets browser's secret key.
- Asymmetric and Symmetric key algorithms work together.
  - Asymmetric key algorithm – verify identity of the owner & its public key -> Establish trust.
  - Once connection is established, Symmetric key algorithm is used to encrypt and decrypt the traffic.

# COMPUTER NETWORKS

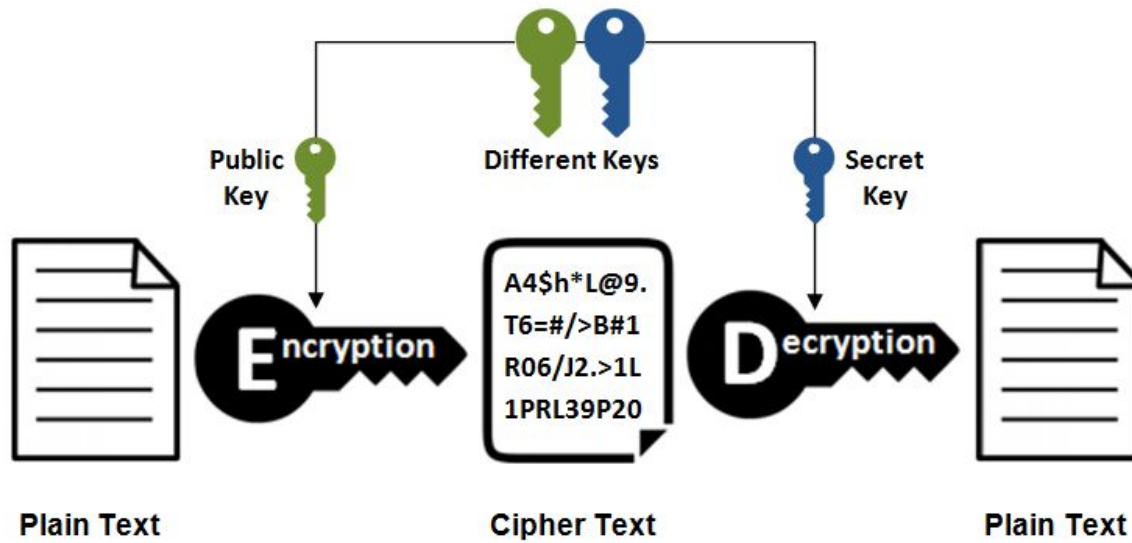
## How does SSL works?



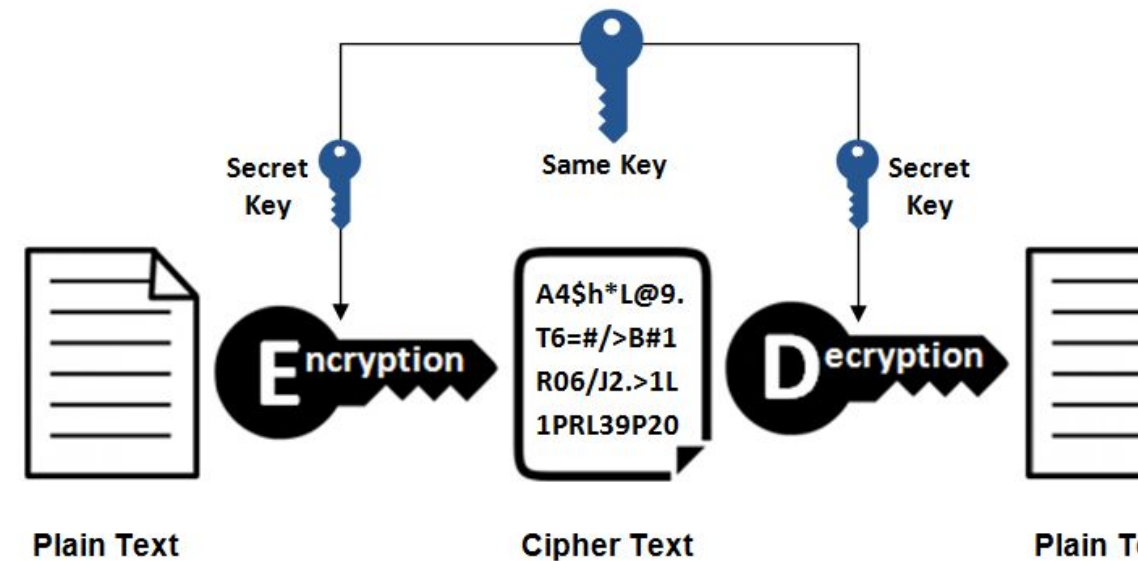


- Stronger Google ranking.
- Updated browser labels.
- Improved security.
- Increased customer confidence / safer experience.
- Build customer trust and improve conversions.

### Asymmetric Encryption



### Symmetric Encryption



- Website/HTTP/Internet cookies
- Piece of data from a specific website
- Stored on a user's computer
- Allows sites to keep track of users
- Eg: language selection



## Cookies

This site uses cookies to offer you a better browsing experience. Find out more on [how we use cookies and how you can change your settings](#).

I accept cookies

I refuse cookies

This website uses cookies to ensure you get the best experience on our website.

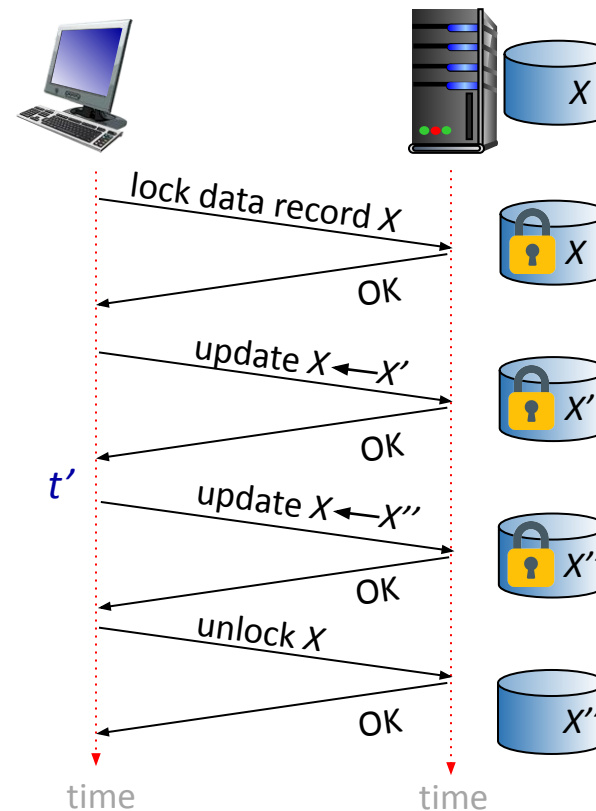
[Learn more](#)

Got it!

Recall: HTTP GET/response interaction is *stateless*

- no notion of multi-step exchanges of HTTP messages to complete a Web “transaction”
  - no need for client/server to track “state” of multi-step exchange
  - all HTTP requests are independent of each other
  - no need for client/server to “recover” from a partially-completed-but-never-completely-completed transaction

a *stateful* protocol: client makes two changes to X, or none at all



**Q:** what happens if network connection or client crashes at  $t'$  ?

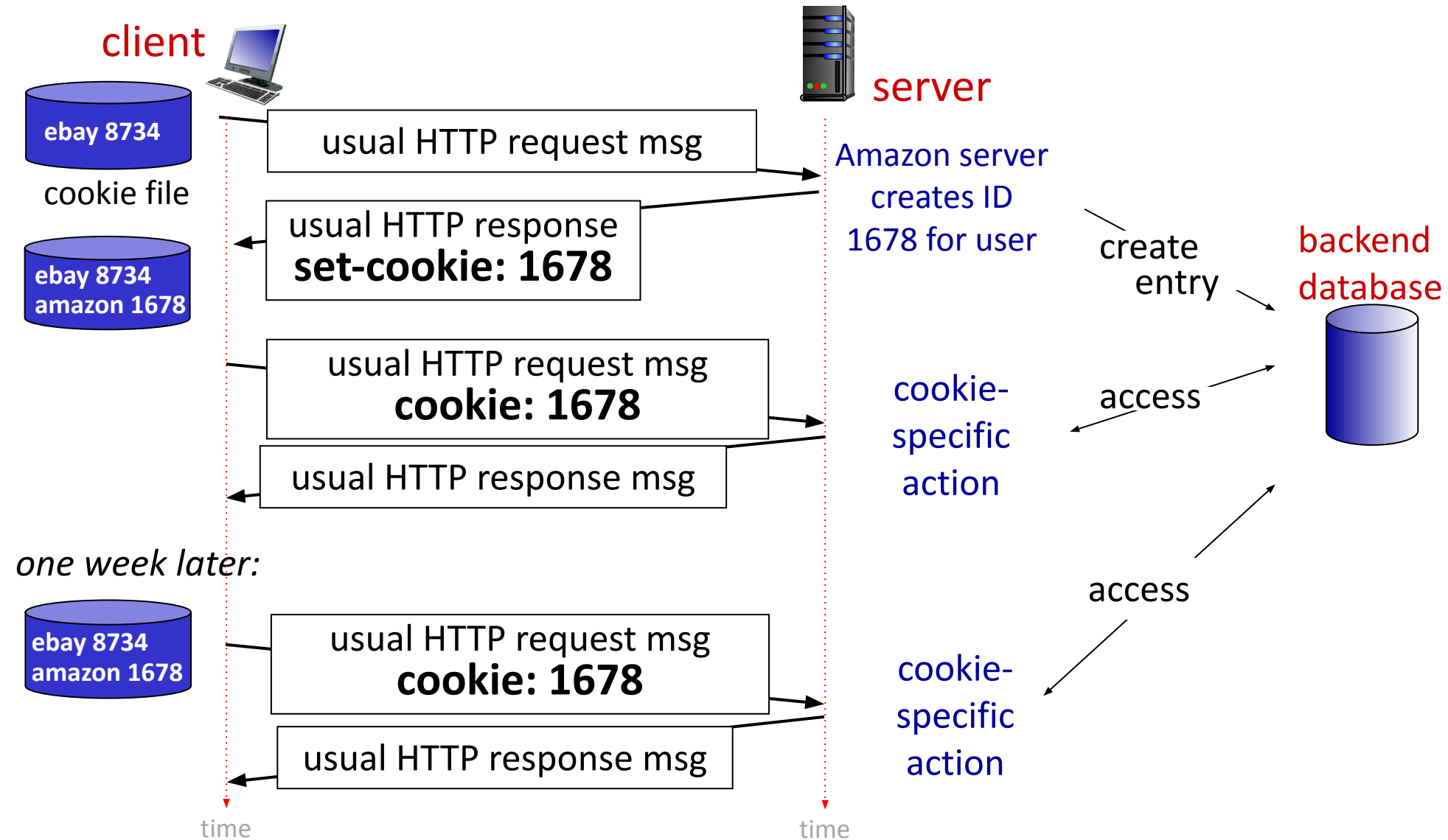
Web sites and client browser use *cookies* to maintain some state between transactions

### *four components:*

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

### Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
  - unique ID (aka “cookie”)
  - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan



### *What cookies can be used for:*

- track user's browsing history
- remembering login details
- track visitor count
- shopping carts
- recommendations
- save coupon codes for you

### *Challenge: How to keep state:*

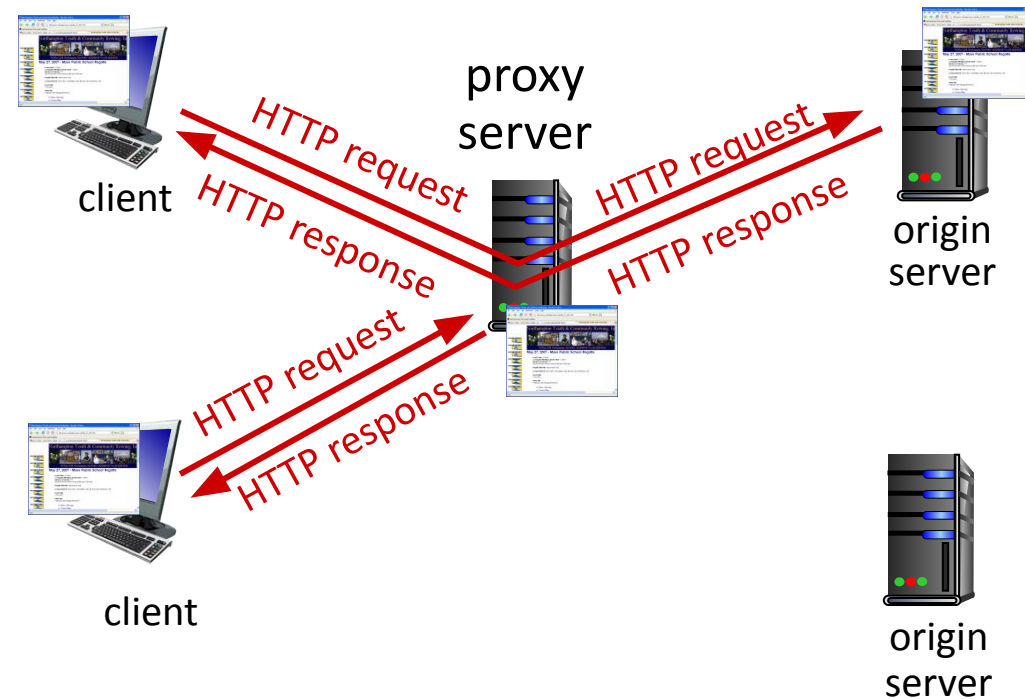
- protocol endpoints: maintain state at sender/receiver over multiple transactions
- cookies: HTTP messages carry state

### *cookies and privacy:* aside

- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

*Goal:* satisfy client request without involving origin server

- user configures browser to point to a *Web cache*
- browser sends all HTTP requests to cache
  - *if* object in cache: cache returns object to client
  - *else* cache requests object from origin server, caches received object, then returns object to client



- Web cache acts as both client and server
  - server for original requesting client
  - client to origin server
- typically cache is installed by ISP (university, company, residential ISP)

### *Why* Web caching?

- reduce response time for client request (speed)
  - cache is closer to client
- reduce traffic on an institution's access link (saves bandwidth)
- internet is dense with caches
  - enables “poor” content providers to more effectively deliver content
- privacy – surf the internet anonymously
- activity logging

# COMPUTER NETWORKS

## Caching example

$$(15 \text{ req/sec}) * (100 \text{ Kbits/req}) / (1.54 \text{ Mbps}) = 0.974$$

$$(15 \text{ req/sec}) * (100 \text{ Kbits/req}) / (1 \text{ Gbps}) = 0.0015$$

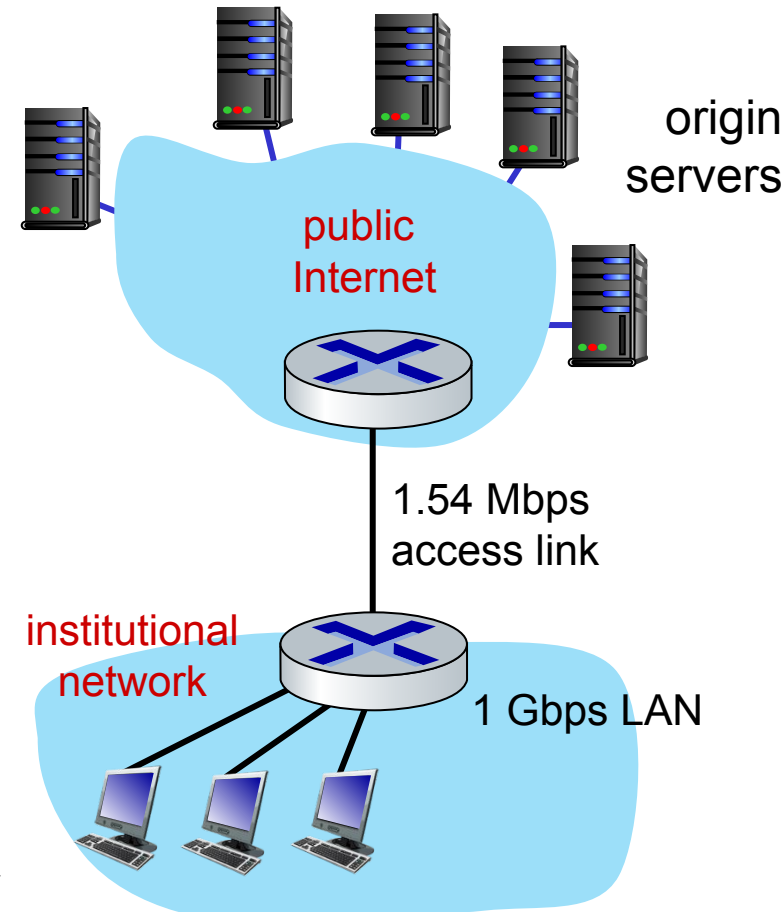
### Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- Web object size: 100K bits
- Average request rate from browsers to origin servers: 15/sec
  - average data rate to browsers: 1.50 Mbps

### Performance:

- LAN utilization: .0015
- access link utilization = .97
- end-end delay = Internet delay +  
access link delay + LAN delay  
= 2 sec + minutes + usecs

*problem: large  
delays at high  
utilization!*



# COMPUTER NETWORKS

## Caching example: buy a faster access link

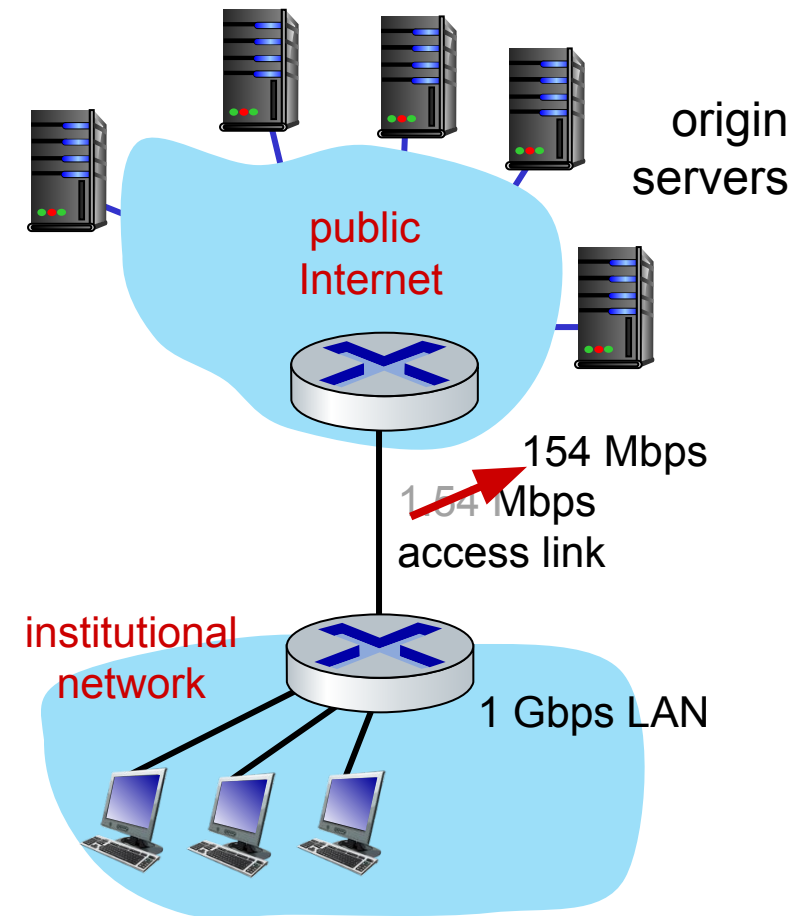
### Scenario:

- access link rate: ~~1.54~~ 154 Mbps
- RTT from institutional router to server: 2 sec
- Web object size: 100K bits
- Avg request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

### Performance:

- LAN utilization: .0015
- access link utilization = ~~.97~~ .0097
- end-end delay = Internet delay +  
access link delay + LAN delay  
= 2 sec + ~~minutes~~ + usecs

**Cost:** faster access link (expensive!) → msec



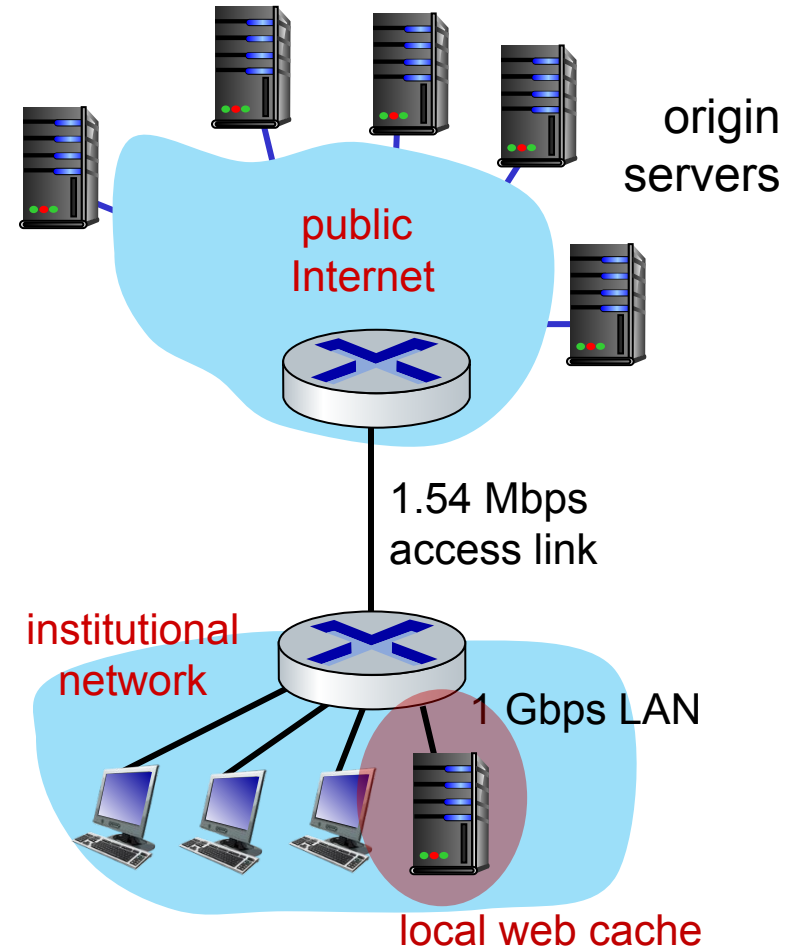
### Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- Web object size: 100K bits
- Avg request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

### Performance: *How to compute link utilization, delay?*

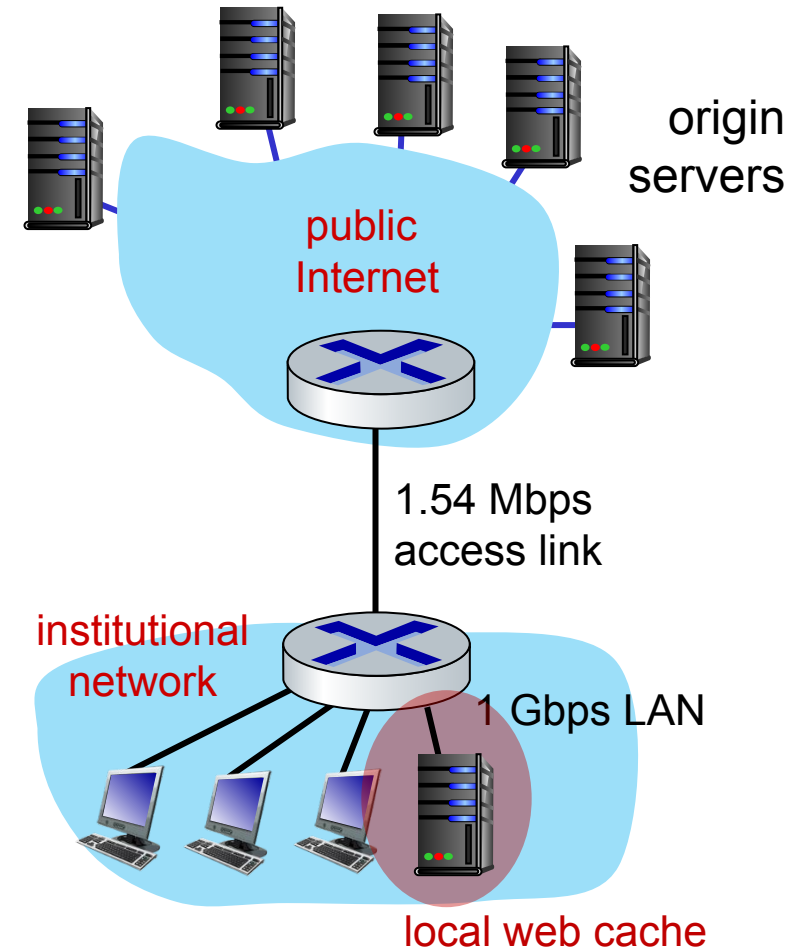
- LAN utilization: .?
- access link utilization = ?
- average end-end delay = ?

*Cost:* web cache (cheap!)



### Calculating access link utilization, end-end delay with cache:

- suppose cache hit rate is 0.4: 40% requests satisfied at cache, 60% requests satisfied at origin
- access link: 60% of requests use access link
- data rate to browsers over access link  
 $= 0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$
- utilization  $= 0.9 / 1.54 = .58$
- average end-end delay  
 $= 0.6 * (\text{delay from origin servers})$   
 $+ 0.4 * (\text{delay when satisfied at cache})$   
 $= 0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$



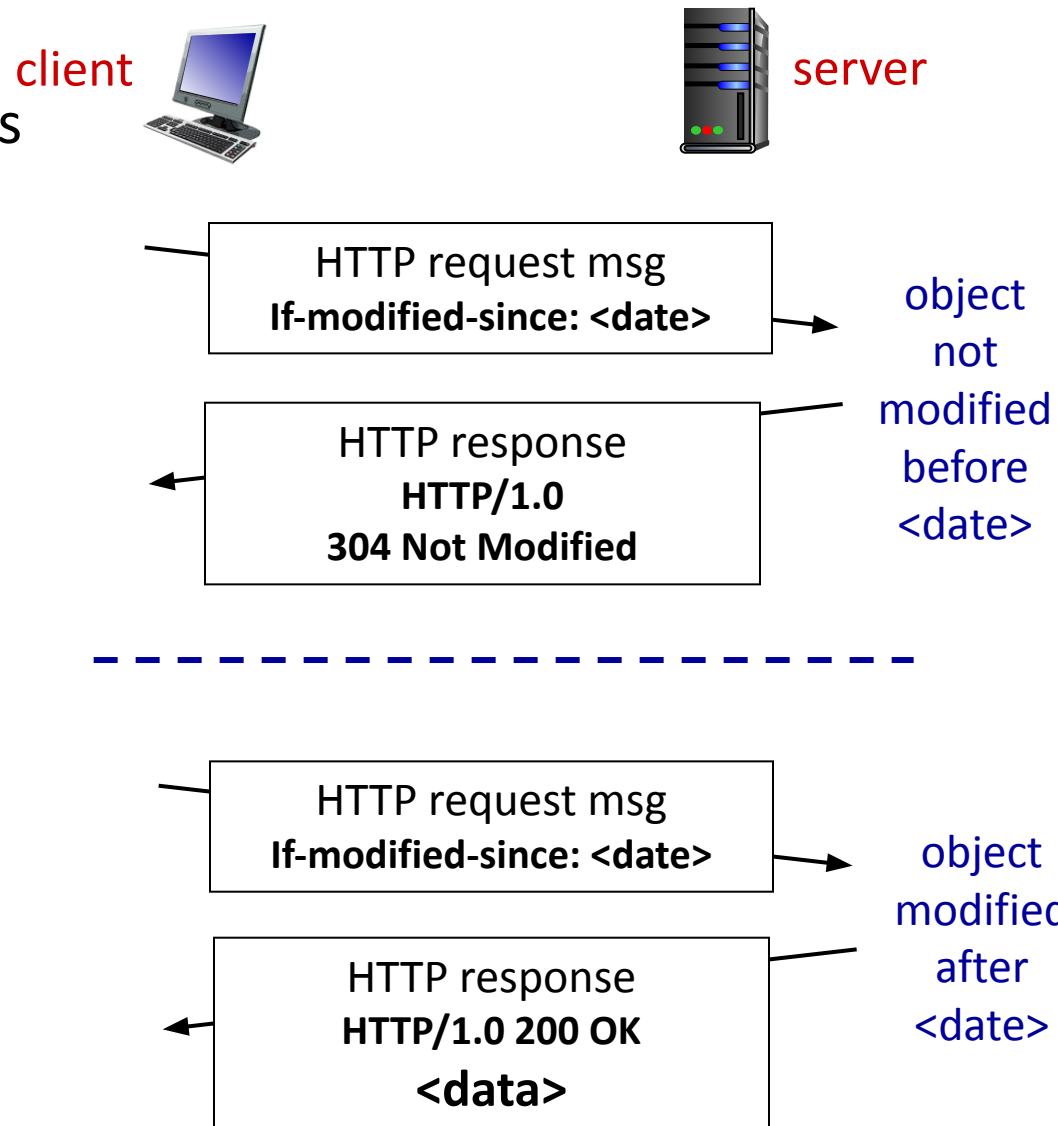
*lower average end-end delay than with 154 Mbps link (and cheaper too!)*

**Goal:** don't send object if cache has up-to-date cached version

- no object transmission delay
- lower link utilization

■ **cache:** specify date of cached copy in HTTP request  
**If-modified-since: <date>**

■ **server:** response contains no object if cached copy is up-to-date:  
**HTTP/1.0 304 Not Modified**



# COMPUTER NETWORKS

## Conditional Get (more)

Microsoft: \\Device\\NPF\_{483C83F4-DCBA-4863-B523-3C4E1B03D06F} [Wireshark 1.8.5 (SVN Rev 47350 from /trunk-1.8)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: http Expression... Clear Apply Save

| No. | Time               | Source         | Destination    | Protocol | Length | Info                                                                                        |
|-----|--------------------|----------------|----------------|----------|--------|---------------------------------------------------------------------------------------------|
| 4   | 20:33:03.198438000 | 10.36.40.181   | 149.152.32.102 | HTTP     | 715    | GET /ssp_director/p.php?a=UUFRxiqyPSEqYHT1pZ04JzU6Iss7PT4uNio4MTI%2BNjkmky0gPScjKdonNz8x... |
| 321 | 20:33:03.427289000 | 149.152.32.102 | 10.36.40.181   | HTTP     | 1514   | [TCP out-of-order] HTTP/1.1 200 OK (JPEG JFIF image)                                        |
| 340 | 20:33:09.383079000 | 10.36.40.181   | 128.119.245.12 | HTTP     | 473    | GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1                                      |
| 341 | 20:33:09.407557000 | 128.119.245.12 | 10.36.40.181   | HTTP     | 701    | HTTP/1.1 200 OK (text/html)                                                                 |
| 343 | 20:33:09.677244000 | 10.36.40.181   | 128.119.245.12 | HTTP     | 384    | GET /favicon.ico HTTP/1.1                                                                   |
| 344 | 20:33:09.689986000 | 128.119.245.12 | 10.36.40.181   | HTTP     | 532    | HTTP/1.1 404 Not Found (text/html)                                                          |
| 347 | 20:33:14.318343000 | 10.36.40.181   | 128.119.245.12 | HTTP     | 586    | GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1                                      |
| 348 | 20:33:14.331881000 | 128.119.245.12 | 10.36.40.181   | HTTP     | 319    | HTTP/1.1 304 Not Modified                                                                   |
| 349 | 20:33:14.410192000 | 10.36.40.181   | 128.119.245.12 | HTTP     | 384    | GET /favicon.ico HTTP/1.1                                                                   |
| 350 | 20:33:14.426443000 | 128.119.245.12 | 10.36.40.181   | HTTP     | 532    | HTTP/1.1 404 Not Found (text/html)                                                          |

Ethernet II, Src: HonHaiPr\_0a:de:6b (cc:af:78:0a:de:6b), Dst: Cisco\_4c:61:3f (00:1e:f7:4c:61:3f)

Internet Protocol Version 4, Src: 10.36.40.181 (10.36.40.181), Dst: 128.119.245.12 (128.119.245.12)

Transmission Control Protocol, Src Port: 55404 (55404), Dst Port: http (80), Seq: 750, Ack: 1126, Len: 532

Hypertext Transfer Protocol

GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n

Host: gaia.cs.umass.edu\r\n

Connection: keep-alive\r\n

Cache-Control: max-age=0\r\n

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,\*/\*;q=0.8\r\n

User-Agent: Mozilla/5.0 (windows NT 6.1; WOW64) AppleWebKit/537.22 (KHTML, like Gecko) Chrome/25.0.1364.97 Safari/537.22\r\n

Accept-Encoding: gzip,deflate,sdch\r\n

Accept-Language: en-US,en;q=0.8\r\n

Accept-Charset: ISO-8859-1,utf-8;q=0.7,\*;q=0.3\r\n

If-Modified-Since: wed, 27 Feb 2013 01:33:01 GMT\r\n

[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html]

01d0 20 49 53 4f 2d 38 35 39 2d 31 2c 75 74 66 2d ISO-885 9-1,utf-  
01e0 38 3b 71 3d 30 2e 37 2c 2a 3b 71 3d 30 2e 33 0d 8;q=0.7,\*;q=0.3.  
01f0 0a 49 66 2d 4e 6f 6e 65 2d 4d 61 74 63 68 3a 20 .If-None-Match:  
0200 22 64 36 63 39 2d 31 37 33 2d 63 31 33 33 36 "d6c96-1 73-c1336  
0210 64 34 30 22 0d 0a 49 66 2d 4d 6f 64 69 66 69 65 d40".If-Modifie  
0220 64 2d 53 69 6e 63 65 3a 20 57 65 64 2c 20 32 37 d-Since: wed, 27  
0230 20 46 65 62 20 32 30 31 33 20 30 31 3a 33 33 3a Feb 201 3 01:33:  
0240 30 31 20 47 4d 54 0d 0a 0d 0a 01 GMT...

Text item (text), 50 bytes

Packets: 367 Displayed: 10 Marked: 0 Dropped: 0

Profile: Default



**THANK YOU**

---

**TEAM NETWORKS**

Department of Computer Science and Engineering